

User Manual

for S32 ADC Driver

Document Number: UM11ADCASR4.4 Rev0000R4.0.2 Rev. 1.0

1 Revision History	2
2 Introduction	3
2.1 Supported Derivatives	3
2.2 Overview	4
2.3 About This Manual	4
2.4 Acronyms and Definitions	5
2.5 Reference List	5
3 Driver	6
3.1 Requirements	6
3.2 Driver Design Summary	6
3.3 Hardware Resources	7
3.4 Deviations from Requirements	7
3.5 Driver Limitations	17
3.6 Driver usage and configuration tips	17
3.6.1 Calibration function	17
3.6.2 Self-Test function	18
3.6.3 CTU control mode	18
3.6.4 CTU triggered mode	19
3.6.5 CTU triggered mode optimization	20
3.6.6 CTU control mode with reading the FIFO by DMA Transfer	20
3.6.7 DMA transfer	22
3.6.8 Injected conversions	24
3.6.9 Set Channel Optimization	24
3.6.10 Enable/Disable channel	25
3.6.11 Conversion Time Once	26
3.6.12 Bypass Abort Chain Check	26
3.6.13 Set Clock Mode	27
3.6.14 Analog Watchdog feature	29
3.6.15 Optimize OneShot HwTrigger Conversions	30
3.6.16 Without Interrupts group	31
3.6.17 Without Dma group	32
3.6.18 External DMA channel feature	33
3.6.19 Optimize DMA streaming groups	33
3.7 Runtime errors	37
3.8 Symbolic Names Disclaimer	37
4 Tresos Configuration Plug-in	38
4.1 Module Adc	43
4.2 Container AdcConfigSet	43

4.3 Container AdcHwUnit	44
4.4 Parameter AdcHwUnitId	44
4.5 Parameter AdcLogicalUnitId	45
4.6 Parameter AdcTransferType	45
4.7 Parameter AdcClockSource	46
4.8 Parameter AdcPrescale	46
4.9 Parameter AdcAltPrescale	47
4.10 Parameter AdcCalibrationPrescale	48
4.11 Parameter AdcPowerDownDelay	48
4.12 Parameter AdcAltPowerDownDelay	49
4.13 Parameter AdcAutoClockOff	49
4.14 Parameter AdcBypassSampling	49
4.15 Parameter AdcHwUnitOverwriteEn	50
4.16 Parameter AdcPresamplingInternalSignal0	50
4.17 Parameter AdcPresamplingInternalSignal1	51
4.18 Parameter AdcPresamplingInternalSignal2	52
4.19 Parameter AdcHwUnitUsrOffset	52
4.20 Parameter AdcHwUnitUsrGain	53
4.21 Parameter AdcHwUnitDmaClearSource	54
4.22 Reference AdcDmaChannelId	54
4.23 Reference AdcCountingDmaChannelId	54
4.24 Reference AdcHwUnitEcucPartitionRef	55
4.25 Container AdcSelfTestThresholdConfiguration	56
4.26 Parameter AdcSTAW0RSelfTestHighThresholdValue	56
4.27 Parameter AdcSTAW0RSelfTestLowThresholdValue	57
4.28 Parameter AdcSTAW1ARSelfTestHighThresholdValue	57
4.29 Parameter AdcSTAW1ARSelfTestLowThresholdValue	57
4.30 Parameter AdcSTAW1BRSelfTestHighThresholdValue	58
4.31 Parameter AdcSTAW1BRSelfTestLowThresholdValue	58
4.32 Parameter AdcSTAW2RSelfTestLowThresholdValue	59
4.33 Parameter AdcSTAW4RSelfTestHighThresholdValue	59
4.34 Parameter AdcSTAW5RSelfTestHighThresholdValue	60
4.35 Container AdcNormalConvTimings	60
4.36 Parameter AdcSamplingDurationNormal0	61
4.37 Parameter AdcSamplingDurationNormal1	61
4.38 Container AdcAlternateConvTimings	62
4.39 Parameter AdcSamplingDurationAlt0	62
4.40 Parameter AdcSamplingDurationAlt1	63
4.41 Container AdcChannel	63
4.42 Parameter AdcLogicalChannelId	64

4.43 Parameter AdcChannelName	64
4.44 Parameter AdcChannelId	65
4.45 Parameter AdcChannelConvTime	66
4.46 Parameter AdcChannelLimitCheck	66
4.47 Parameter AdcChannelHighLimit	67
4.48 Parameter AdcChannelLowLimit	67
4.49 Parameter AdcChannelRangeSelect	68
4.50 Parameter AdcChannelRefVoltsrcHigh	68
4.51 Parameter AdcChannelRefVoltsrcLow	69
4.52 Parameter AdcChannelResolution	69
4.53 Parameter AdcChannelSampTime	70
4.54 Parameter AdcEnablePresampling	70
4.55 Parameter AdcEnableThresholds	71
4.56 Parameter AdcWdogNotification	71
4.57 Reference AdcThresholdRegister	71
4.58 Container AdcGroup	72
4.59 Parameter AdcGroupAccessMode	72
4.60 Parameter AdcGroupConversionMode	73
4.61 Parameter AdcGroupConversionType	73
4.62 Parameter AdcGroupId	74
4.63 Parameter AdcGroupPriority	74
4.64 Parameter AdcGroupReplacement	75
4.65 Parameter AdcGroupTriggSrc	75
4.66 Parameter AdcHwTrigSignal	76
4.67 Parameter AdcHwTrigTimer	76
4.68 Parameter AdcNotification	77
4.69 Parameter AdcExtraNotification	77
4.70 Parameter AdcStreamingBufferMode	78
4.71 Parameter AdcEnableOptimizeDmaStreamingGroups	78
4.72 Parameter AdcEnableHalfInterrupt	79
4.73 Parameter AdcStreamingNumSamples	79
4.74 Parameter AdcStreamResultGroup	80
4.75 Parameter AdcEnableChDisableChGroup	81
4.76 Parameter AdcWithoutInterrupts	81
4.77 Parameter AdcWithoutDma	82
4.78 Parameter AdcExtDMAChanEnable	82
4.79 Reference AdcGroupHwTriggerSource	83
4.80 Reference AdcGroupDefinition	83
4.81 Reference AdcGroupEcucPartitionRef	84
4.82 Container AdcGroupConversionConfiguration	84

4.83 Parameter	AdcSamplingDuration0	85
4.84 Parameter	AdcSamplingDuration1	85
4.85 Container	AdcAlternateGroupConvTimings	85
4.86 Parameter	AdcAltGroupSamplingDuration0	86
4.87 Parameter	AdcAltGroupSamplingDuration1	86
4.88 Container	AdcThresholdControl	87
4.89 Parameter	AdcThresholdControlRegister	87
4.90 Parameter	AdcHighThreshold	88
4.91 Parameter	AdcLowThreshold	88
4.92 Container	AdcHwTrigger	89
4.93 Parameter	AdcHwTrigSrc	89
4.94 Container	CtuHwUnit	90
4.95 Parameter	CtuHwUnitId	90
4.96 Parameter	CtuLogicalUnitId	91
4.97 Parameter	CtuTGSMODE	91
4.98 Parameter	CtuInputClockPrescaler	92
4.99 Parameter	CtuExtTrigMode	92
4.100 Parameter	CtuTgsCounterCompareVal	93
4.101 Parameter	CtuTgsCounterReloadVal	93
4.102 Parameter	CtuAdcCmdListMode	94
4.103 Parameter	CtuSeqModeMrsInput	94
4.104 Parameter	CtuSeqModeMrsInputEdge	95
4.105 Parameter	CtuDmaDoneGRE	95
4.106 Parameter	CtuDmaReqMRS	96
4.107 Parameter	CtuFifoDmaRawData	96
4.108 Parameter	CtuDisableOutput	97
4.109 Parameter	CtuErrorNotif	97
4.110 Parameter	CtuMrsNotif	97
4.111 Parameter	CtuAdcCmdIssueNotif	98
4.112 Parameter	CtuDigitalFilter	98
4.113 Parameter	CtuControlOnTime	99
4.114 Container	ExpectedConvDurationConfig	99
4.115 Parameter	CtuExpectedValuePortA	100
4.116 Parameter	CtuExpectedNotifPortA	100
4.117 Parameter	CtuExpectedValuePortB	100
4.118 Parameter	CtuExpectedNotifPortB	101
4.119 Parameter	CtuConvDurationCounterRange	101
4.120 Container	CtuInputTrigConfigs	102
4.121 Parameter	CtuInputTrigEdge	102
4.122 Reference	CtuInputTrigSelect	103

4.123 Container CtuTriggerCfg	104
4.124 Parameter CtuTriggerIndex	104
4.125 Parameter CtuCompareVal	104
4.126 Parameter CtuCmdListStartAdr	105
4.127 Parameter CtuOutputTrigEnMask	105
4.128 Parameter CtuTriggerNotif	106
4.129 Container CtuAdcCommandList	107
4.130 Parameter CtuIntEn	107
4.131 Parameter CtuFifoIdx	108
4.132 Parameter CtuConvMode	108
4.133 Parameter CtuLastCmd	109
4.134 Parameter CtuAdcPort	109
4.135 Parameter CtuAdcChanA	109
4.136 Parameter CtuAdcChanB	110
4.137 Container CtuResultFifos	111
4.138 Parameter CtuFifoIndex	111
4.139 Parameter CtuFifoThreshold	111
4.140 Parameter CtuFifoDmaEn	112
4.141 Parameter CtuFifoDmaBuffer	112
4.142 Parameter CtuFifoThresholdNotif	113
4.143 Parameter CtuFifoUnderrunNotif	113
4.144 Parameter CtuFifoOverrunNotif	114
4.145 Parameter CtuFifoFullNotif	114
4.146 Reference CtuFifoDmaChannelId	115
4.147 Container AdcGeneral	115
4.148 Parameter AdcDeInitApi	115
4.149 Parameter AdcDevErrorDetect	116
4.150 Parameter AdcEnableLimitCheck	116
4.151 Parameter AdcEnableQueuing	117
4.152 Parameter AdcPriorityQueueMaxDepth	117
4.153 Parameter AdcEnableStartStopGroupApi	118
4.154 Parameter AdcGrpNotifCapability	118
4.155 Parameter AdcHwTriggerApi	119
4.156 Parameter AdcPriorityImplementation	119
4.157 Parameter AdcReadGroupApi	120
4.158 Parameter AdcResultAlignment	121
4.159 Parameter AdcVersionInfoApi	121
4.160 Parameter AdcLowPowerStatesSupport	122
4.161 Parameter AdcPowerStateAsynchTransitionMode	122
4.162 Reference AdcEcucPartitionRef	122

4.163 Reference	AdcKernelEcucPartitionRef	123
4.164 Container	AdcPowerStateConfig	123
4.165 Parameter	AdcPowerState	124
4.166 Parameter	AdcPowerStateReadyCbRef	124
4.167 Container	AdcInterrupt	125
4.168 Parameter	AdcInterruptSource	125
4.169 Parameter	AdcInterruptEnable	126
4.170 Container	AdcPublishedInformation	126
4.171 Parameter	AdcChannelValueSigned	127
4.172 Parameter	AdcGroupFirstChannelFixed	127
4.173 Parameter	AdcMaxChannelResolution	127
4.174 Container	CommonPublishedInformation	128
4.175 Parameter	ArReleaseMajorVersion	128
4.176 Parameter	ArReleaseMinorVersion	129
4.177 Parameter	ArReleaseRevisionVersion	129
4.178 Parameter	ModuleId	130
4.179 Parameter	SwMajorVersion	130
4.180 Parameter	SwMinorVersion	131
4.181 Parameter	SwPatchVersion	131
4.182 Parameter	VendorApiInfix	132
4.183 Parameter	VendorId	132
4.184 Container	AutosarExt	133
4.185 Parameter	AdcTimeoutMethod	133
4.186 Parameter	AdcTimeoutVal	134
4.187 Parameter	AdcSarIpDevErrorDetect	134
4.188 Parameter	CtuIpDevErrorDetect	135
4.189 Parameter	AdcMulticoreSupport	135
4.190 Parameter	AdcEnableGroupDependentChannelNames	136
4.191 Parameter	AdcBypassAbortChainCheck	136
4.192 Parameter	AdcConvTimeOnce	137
4.193 Parameter	AdcOptimizeOneShotHwTriggerConversions	137
4.194 Parameter	AdcOptimizeDmaStreamingGroups	138
4.195 Parameter	AdcPreSamplingOnce	139
4.196 Parameter	AdcEnableInitialNotification	139
4.197 Parameter	AdcEnableDmaTransferMode	140
4.198 Parameter	AdcUseSoftwareInjectedGroups	140
4.199 Parameter	AdcUseHardwareNormalGroups	141
4.200 Parameter	AdcEnableUserCodeSupport	141
4.201 Parameter	AdcSetHwUnitPowerModeApi	142
4.202 Parameter	AdcEnableChDisableChApi	142

4.203 Parameter AdcGetInjectedConvStatusApi	143
4.204 Parameter AdcEnableThresholdConfigurationApi	143
4.205 Parameter AdcEnableCtuTrigAutosarExtApi	144
4.206 Parameter AdcCtuHardwareTriggerOptimization	144
4.207 Parameter AdcEnableCtuControlModeApi	145
4.208 Parameter CtuEnableDmaTransferMode	146
4.209 Parameter AdcCtuControlModeExtraApis	146
4.210 Parameter AdcEnableWatchdogApi	147
4.211 Parameter AdcEnableSetChannel	147
4.212 Parameter AdcEnableDualClockMode	148
4.213 Parameter AdcEnableCalibration	148
4.214 Parameter AdcEnableSelfTest	149
4.215 Parameter AdcEnableReadRawDataApi	149
4.216 Parameter AdcEnableGroupStreamingResultReorder	150
5 Module Index	151
5.1 Software Specification	151
6 Module Documentation	152
6.1 Adc driver	152
6.1.1 Detailed Description	152
6.1.2 Data Structure Documentation	158
6.1.3 Macro Definition Documentation	160
6.1.4 Types Reference	171
6.1.5 Enum Reference	172
6.1.6 Function Reference	176
6.1.7 Variable Documentation	195
6.2 Adc Sar IPL	197
6.2.1 Detailed Description	197
6.2.2 Data Structure Documentation	200
6.2.3 Macro Definition Documentation	206
6.2.4 Types Reference	207
6.2.5 Enum Reference	208
6.2.6 Function Reference	213
6.3 Ctu IPL	233
6.3.1 Detailed Description	233
6.3.2 Data Structure Documentation	236
6.3.3 Macro Definition Documentation	242
6.3.4 Types Reference	249
6.3.5 Enum Reference	249
6.3.6 Function Reference	253



Chapter 1

Revision History

Revision	Date	Author	Description
1.0	30.06.2023	NXP RTD Team	S32 Real-Time Drivers AUTOSAR 4.4 Version 4.0.2 Release

Chapter 2

Introduction

- [Supported Derivatives](#)
- [Overview](#)
- [About This Manual](#)
- [Acronyms and Definitions](#)
- [Reference List](#)

This User Manual describes NXP Semiconductor AUTOSAR ADC for S32. AUTOSAR ADC driver configuration parameters and deviations from the specification are described in Driver chapter of this document. AUTOSAR ADC driver requirements and APIs are described in the AUTOSAR ADC driver software specification document.

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of NXP Semiconductors:

- s32g274a_bga525
- s32g254a_bga525
- s32g233a_bga525
- s32g234m_bga525
- s32g378a_bga525
- s32g379a_bga525
- s32g398a_bga525
- s32g399a_bga525
- s32g338m_bga525
- s32g339m_bga525
- s32g358a_bga525
- s32g359a_bga525
- s32r45_bga780

All of the above microcontroller devices are collectively named as S32.

2.2 Overview

AUTOSAR (AUTomotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR:

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".
- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About This Manual

This Technical Reference employs the following typographical conventions:

- **Boldface** style: Used for important terms, notes and warnings.
- *Italic* style: Used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

Warning

This is a warning

2.4 Acronyms and Definitions

Term	Definition
API	Application Programming Interface
ASM	Assembler
BSMI	Basic Software Make file Interface
CAN	Controller Area Network
C/CPP	C and C++ Source Code
CS	Chip Select
CTU	Cross Trigger Unit
DEM	Diagnostic Event Manager
DET	Development Error Tracer
DMA	Direct Memory Access
ECU	Electronic Control Unit
FIFO	First In First Out
LSB	Least Significant Bit
MCU	Micro Controller Unit
MIDE	Multi Integrated Development Environment
MSB	Most Significant Bit
RAM	Random Access Memory
SIU	Systems Integration Unit
SWS	Software Specification
VLE	Variable Length Encoding
XML	Extensible Markup Language

2.5 Reference List

#	Title	Version
1	S32G2 Reference Manual	Rev. 7, February 2023
2	S32G3 Reference Manual	Rev. 3, February 2023
3	S32R45 Reference Manual	Rev. 3, December 2021
4	S32G2: Mask Set Errata for Mask 0P77B	Rev. 3.0
5	S32G2: Mask Set Errata for Mask 1P77B	Rev. 1.0
6	S32G3: Mask Set Errata for Mask 1P72B	Rev. 2.1
7	S32R45: Mask Set Errata for Mask P57D	Rev. 2.0
8	S32G2 Data Sheet	Rev. 6, December 2022
9	S32G3 Data Sheet	Rev. 2, RC, February 2023
10	VR5510 Data Sheet	Rev. 5, April 2022
11	S32R45 Data Sheet	Rev. 3, RC — November 2022

Chapter 3

Driver

- [Requirements](#)
- [Driver Design Summary](#)
- [Hardware Resources](#)
- [Deviations from Requirements](#)
- [Driver Limitations](#)
- [Driver usage and configuration tips](#)
- [Runtime errors](#)
- [Symbolic Names Disclaimer](#)

3.1 Requirements

Requirements for this driver are detailed in the Autosar Driver Software Specification document (See [Table Reference List](#)).

It has vendor-specific requirements and implementation.

3.2 Driver Design Summary

The ADC Driver initializes and controls the internal Analog to Digital Converter Unit(s) of the microcontroller. It provides services to start and stop a conversion respectively to enable and disable the trigger source for a conversion. Furthermore it provides services to enable and disable a notification mechanism and routines to query the status and result of a conversion. The ADC Driver shall work on so called ADC channels. An ADC channel combines an analog input pin, the needed ADC circuitry itself and a conversion result register into an entity that can be individually controlled and accessed via the ADC Driver. The driver provides a service for Streaming management results and for De-Initialization of circuits.

3.3 Hardware Resources

This device provides two General-purpose ADC :ADC HW Unit 0 (ADC0), ADC HW Unit 1 (ADC1). The number of ADC hardware units and channels are derivative specific, so please consult the reference manual.

3.4 Deviations from Requirements

The driver deviates from the AUTOSAR ADC Driver software specification in some places. The table identifies the AUTOSAR requirements that are not fully implemented, implemented differently or out of scope for the ADC Driver.

Term	Definition
N/S	Out of scope
N/I	Not implemented
N/F	Not fully implemented

Below table identifies the AUTOSAR requirements that are not fully implemented, implemented differently or out of scope for the driver.

Requirement	Status	Description	Notes
SWS_Adc_00082	N/F	Service name: - IoHwAb_Adc↔ Notification - Syntax: - void IoHw↔ Ab_AdcNotification(void) - Sync/↔ Async: - Synchronous - Reentrancy↔ : - Non Reentrant - Parameters (in): - None - Parameters (inout): - None - Parameters (out): - None - Return value: - None - Description: - Will be called by the ADC Driver when a group conversion is completed for group . - Available via: - IoHwAb↔ _Adc.h -	"IoHwAb_AdcNotification<#group↔ ID>" is not fully implemented, because callback notification with this prototype can be configured per group, according with SWS_Adc_00084 and also in agreement with this statement from the 'Specification of ADC Driver AUTOSAR 4.4.0': "In this chapter all interfaces are listed where the target function could be configured. The target function is usually a call-back function. The names of this kind of interfaces are not fixed because they are configurable."
SWS_Adc_00345	N/S	The ADC module's priority mechanism shall allow suspending and resuming of channel group conversions.	Only abort/restart is supported
SWS_Adc_00341	N/S	If the priority mechanism is supported by the hardware: The ADC module shall support the static configuration option ADC_PRIORITY_HW to enable the priority mechanism using only the hardware priority mechanism.	HW prioritization mechanism is not available on platform
SWS_Adc_00339	N/S	If hardware priority mechanism is supported and selected: The ADC module shall allow the mapping of the configured priority levels (0-255) to the available hardware priority levels.	HW prioritization mechanism is not available on platform

Requirement	Status	Description	Notes
SWS_Adc_00337	N/S	The hardware prioritization mechanism shall be used in case of hardware triggered conversion requests.	HW prioritization mechanism is not available on platform
SWS_Adc_00488	N/S	In case development error reporting is activated: The API shall report the DET error ADC_E_POWER_STATE_E_NOT_SUPPORTED in case this API is called with an unsupported power state or the peripheral does not support low power states at all.	Adc_SetPowerState does not have the Adc_PowerStateType parameter. This parameter is passed and checked (for unsupported power state) through the AdcPreparePowerState function that is called before calling Adc_SetPowerState function.
SWS_Adc_00489	N/S	The API shall report a runtime error ADC_E_TRANSITION_NOT_POSSIBLE in case the requested power state cannot be directly reached from the current power state.	ADC hardware supports only two power states (full power and low power) and all transitions are valid; there is no need to report ADC_E_TRANSITION_NOT_POSSIBLE error.
SWS_Adc_00498	N/S	The API shall report a runtime error ADC_E_TRANSITION_NOT_POSSIBLE in case the requested power state cannot be directly reached from the current power state. All asynchronous operation, needed to reach the target power state, can be executed in background in the context of Adc_Main_PowerTransitionManager.	ADC hardware supports only two power states (full power and low power) and all transitions are valid; there is no need to report ADC_E_TRANSITION_NOT_POSSIBLE error.
SWS_Adc_00479	N/S	Service name: - Adc_Main_PowerTransitionManager - Syntax: - void Adc_Main_PowerTransitionManager(void) - Service ID[hex]: - 0x14 - Description: - This API is cyclically called and supervises the power state transitions, checking for the readiness of the module and issuing the callbacks IoHwAbs_Adc_NotifyReadyForPowerState (see AdcPowerStateReadyCbKRef configuration parameter). - Available via: - SchM_Adc.h -	Asynchronous Power State transition mode is not needed. It's not supported by hardware but still available in driver design and code (without any functionalities) since it's an Autosar API.
SWS_Adc_00499	N/S	This API executes any non-immediate action needed to finalize a power state transition requested by Adc_PreparePowerState().	Asynchronous Power State transition mode is not needed. It's not supported by hardware.
SWS_Adc_00500	N/S	The rate of scheduling shall be defined by Adc_MainSchedulePeriod and shall be variable, as the function only needs to be called if a transition has been requested	Asynchronous Power State transition mode is not needed. It's not supported by hardware.
SWS_Adc_00501	N/S	This API shall also issue callback notifications to the eventually registered users (IoHwAbs) as configured, only in case the asynch mode is chosen.	Asynchronous Power State transition mode is not needed. It's not supported by hardware.

Requirement	Status	Description	Notes
SWS_Adc_00502	N/S	In case the ADC module is not initialized, this function shall simply return without any further elaboration. This is needed to avoid to elaborate uninitialized variables. No DET error shall be entered, because this condition can easily be verified during the startup phase (tasks started before the initialization is complete).Rationale↵: during the startup phase it can happen that the OS already schedules tasks, which call main functions, while some modules are not initialised yet. This is no real error condition, although need handling, i.e. returning without execution.Although the transition state monitoring functionality is mandatory, the implementation of this API is optional, meaning that if the HW allows for other ways to deliver notification and watch the transition state the implementation of this function can be skipped.	Asynchronous Power State transition mode is not needed. It's not supported by hardware.
SWS_Adc_00480	N/S	Service name: - IoHwAb_Adc_↵ NotifyReadyForPowerState - Syntax↵: - void IoHwAb_Adc_NotifyReady↵ ForPowerState(void) - Sync/Async↵: - Synchronous - Reentrancy: - Non Reentrant - Parameters (in): - None - Parameters (inout): - None - Parameters (out): - None - Return value: - None - Description: - The API shall be invoked by the ADC Driver when the requested power state preparation for mode is completed. - Available via: - IoHwAb_Adc.h -	Asynchronous Power State transition mode is not needed. It's not supported by hardware but still available in driver design and code (without any functionalities) since it's an Autosar API.
SWS_Adc_00460	N/S	These requirements are not applicable to this specification.	Not a requirement.

Requirement	Status	Description	Notes
SWS_Adc_91004	N/S	Service name: - Adc_DisableGroupNotification (draft) - Syntax: - void Adc_DisableGroupNotification (Adc_GroupType Group) - Service ID[hex]: - 0x08 - Sync/Async: - Asynchronous - Reentrancy: - Reentrant - Parameters (in): - Group - Numeric ID of requested ADC Channel group. - Parameters (inout): - None - Parameters (out): - None - Return value: - None - Description: - Disables the notification mechanism for the requested ADC Channel group.Tags: atp.Status=draft - Available via: - Adc.h -	[17/05/2019] The function is sync instead of async because the multicore implementation is done with type 2 instead of type 4.
SWS_Adc_91002	N/S	Service name: - Adc_DisableHardwareTrigger (draft) - Syntax: - void Adc_DisableHardwareTrigger (Adc_GroupType Group) - Service ID[hex]: - 0x06 - Sync/Async: - Asynchronous - Reentrancy: - Reentrant - Parameters (in): - Group - Numeric ID of requested ADC Channel group. - Parameters (inout): - None - Parameters (out): - None - Return value: - None - Description: - Disables the hardware trigger for the requested ADC Channel group.Tags: atp.Status=draft - Available via: - Adc.h -	[17/05/2019] The function is sync instead of async because the multicore implementation is done with type 2 instead of type 4.
SWS_Adc_91003	N/S	Service name: - Adc_EnableGroupNotification (draft) - Syntax: - void Adc_EnableGroupNotification (Adc_GroupType Group) - Service ID[hex]: - 0x07 - Sync/Async: - Asynchronous - Reentrancy: - Reentrant - Parameters (in): - Group - Numeric ID of requested ADC Channel group. - Parameters (inout): - None - Parameters (out): - None - Return value: - None - Description: - Enables the notification mechanism for the requested ADC Channel group.Tags: atp.Status=draft - Available via: - Adc.h -	[17/05/2019] The function is sync instead of async because the multicore implementation is done with type 2 instead of type 4.

Requirement	Status	Description	Notes
SWS_Adc_91001	N/S	Service name: - Adc_EnableHardwareTrigger (draft) - Syntax: - void Adc_EnableHardwareTrigger (Adc_GroupType Group) - Service ID[hex]: - 0x05 - Sync/Async: - Asynchronous - Reentrancy: - Reentrant - Parameters (in): - Group - Numeric ID of requested ADC Channel group. - Parameters (inout): - None - Parameters (out): - None - Return value: - None - Description: - Enables the hardware trigger for the requested ADC Channel group. Tags: atp.Status=draft - Available via: - Adc.h -	[17/05/2019] The function is sync instead of async because the multicore implementation is done with type 2 instead of type 4.
SWS_Adc_91000	N/S	Service name: - Adc_SetupResultBuffer (draft) - Syntax: - Std_ReturnType Adc_SetupResultBuffer (Adc_GroupType Group, const Adc_ValueGroupType* DataBufferPtr) - Service ID[hex]: - 0x0c - Sync/Async: - Asynchronous - Reentrancy: - Reentrant - Parameters (in): - Group - Numeric ID of requested ADC channel group. - DataBufferPtr - pointer to result data buffer - Parameters (inout): - None - Parameters (out): - None - Return value: - Std_ReturnType - E_OK: result buffer pointer initialized correctly E_NOT_OK: operation failed or development error occurred - Description: - Initializes ADC driver with the group specific result buffer start address where the conversion results will be stored. The application has to ensure that the application buffer, where DataBufferPtr points to, can hold all the conversion results of the specified group. The initialization with Adc_SetupResultBuffer is required after reset, before a group conversion can be started. Tags: atp.Status=draft - Available via: - Adc.h -	[17/05/2019] The function is sync instead of async because the multicore implementation is done with type 2 instead of type 4.

Requirement	Status	Description	Notes
ECUC_Adc_00461	N/F	Name - AdcPowerState - Parent Container - AdcPowerState↵ Config - Description - Each instance of this parameter describes a different power state supported by the ADC HW. It should be defined by the HW supplier and used by the ADCDriver to reference specific HW configurations which set the ADC HW module in the referenced power state. At least the power mode corresponding to full power state shall be always configured. - Multiplicity - 1 - Type - EcucIntegerParamDef (Symbolic Name generated for this parameter) - Range - 0 .. 18446744073709551615 - &#0160; - Default value - - - Post-Build Variant Value - false - Value Configuration Class - Pre-compile time - X - All Variants - Link time - - - - Post-build time - - - - Scope / Dependency - scope↵ : localdependency: This parameter shall only be configured if the parameter AdcLowPowerStatesSupport is set to true.	This requirement shall be fulfilled to avoid VSMD errors. Asynchronous Power State transition mode doesn't support by hardware. Maximum value of this node is restricted from 18446744073709551615 to 9223372036854775807 due to EB Tresos limitation: "Deviating from the AUTOSAR traceable [TPS_GST_00008], the formula language interpreter of E↵B tresos Studio supports only the signed 64-bit range for integer values, i.e. from 0x8000000000000000 (-9223372036854775808) to 0x7fffffffffffffff (9223372036854775807)." "The formula language parser of EB tresos Studio is implemented in Java. It uses the long primitive type for integer values and therefore can only support the restricted range."

Requirement	Status	Description	Notes
ECUC_Adc_00011	N/F	Name - AdcChannelConvTime - Parent Container - AdcChannel - Description - Configuration of conversion time, i.e. the time during which the analogue value is converted into digital representation, (in clock cycles) for each channel, if supported by hardware.Implementation↵ Type: Adc_ConversionTimeType - Multiplicity - 0..1 - Type - EcucIntegerParamDef - Range - 0 .. 18446744073709551615 - &#0160; - Default value - - - Post-Build Variant Multiplicity - true - Post-Build Variant Value - true - Multiplicity Configuration Class - Pre-compile time - X - VARIANT-PRE-↵ COMPILE - Link time - - - - Post-build time - X - VARIANT-P↵ OST-BUILD - Value Configuration Class - Pre-compile time - X - VARIANT-PRE- COMPILE - Link time - - - - Post-build time - X - VARIANT-P↵ OST-BUILD - Scope / Dependency - scope: local	Maximum value of this node is restricted from 18446744073709551615 to 9223372036854775807 due to EB Tresos limitation: "Deviating from the AUTOSAR traceable [TPS_GST_00008], the formula language interpreter of E↵ B tresos Studio supports only the signed 64-bit range for integer values, i.e. from 0x8000000000000000 (-9223372036854775808) to 0x7fffffffffffffff (9223372036854775807)." "The formula language parser of EB tresos Studio is implemented in Java. It uses the long primitive type for integer values and therefore can only support the restricted range."

Requirement	Status	Description	Notes
ECUC_Adc_00455	N/F	Name - AdcChannelHighLimit - Parent Container - AdcChannel - Description - High limit - used for limit checking. - Multiplicity - 0..1 - Type - EcucIntegerParamDef - Range - 0 .. 18446744073709551615 - &#0160; - Default value - - - Post-Build Variant Multiplicity - false - Post-Build Variant Value - false - Multiplicity Configuration Class - Pre-compile time - X - All Variants - Link time - - - - Post-build time - - - - Value Configuration Class - Pre-compile time - X - All Variants - Link time - - - - Post-build time - - - - Scope / Dependency - scope↵ : localdependency: AdcEnable↵ LimitCheck: not available if limit checking is not globally enabled. AdcChannelLimitCheck: not available if channel specific limit check is not enabled. AdcChannelLowLimit↵ : has to be greater or equal than AdcChannelLowLimit.	Maximum value of this node is restricted from 18446744073709551615 to 9223372036854775807 due to EB Tresos limitation: "Deviating from the AUTOSAR traceable [TPS_GST_00008], the formula language interpreter of E↵B tresos Studio supports only the signed 64-bit range for integer values, i.e. from 0x8000000000000000 (-9223372036854775808) to 0x7fffffffffffffff (9223372036854775807)." "The formula language parser of EB tresos Studio is implemented in Java. It uses the long primitive type for integer values and therefore can only support the restricted range."

Requirement	Status	Description	Notes
ECUC_Adc_00454	N/F	Name - AdcChannelLowLimit - Parent Container - AdcChannel - Description - Low limit - used for limit checking. - Multiplicity - 0..1 - Type - EcucIntegerParamDef - Range - 0 .. 18446744073709551615 - &#0160; - Default value - - - Post-Build Variant Multiplicity - false - Post-Build Variant Value - false - Multiplicity Configuration Class - Pre-compile time - X - All Variants - Link time - - - - Post-build time - - - - Value Configuration Class - Pre-compile time - X - All Variants - Link time - - - - Post-build time - - - - Scope / Dependency - scope↵ : localdependency: AdcEnableLimit↵ Check: not available if limit checking is not globally enabled. AdcChannel↵ LimitCheck: not available if channel specific limit check is not enabled. AdcChannelHighLimit: has to be less or equal than AdcChannelHighLimit.	Maximum value of this node is restricted from 18446744073709551615 to 9223372036854775807 due to EB Tresos limitation: "Deviating from the AUTOSAR traceable [TPS_GST_00008], the formula language interpreter of E↵B tresos Studio supports only the signed 64-bit range for integer values, i.e. from 0x8000000000000000 (-9223372036854775808) to 0x7fffffffffffffff (9223372036854775807)." "The formula language parser of EB tresos Studio is implemented in Java. It uses the long primitive type for integer values and therefore can only support the restricted range."

Requirement	Status	Description	Notes
ECUC_Adc_00290	N/F	Name - AdcChannelSampTime - Parent Container - AdcChannel - Description - Configuration of sampling time, i.e. the time during which the value is sampled, (in clock cycles) for each channel, if supported by hardware.Implementation↵ Type: Adc_SamplingTimeType - Multiplicity - 0..1 - Type - EcucIntegerParamDef - Range - 0 .. 18446744073709551615 - &#0160;- Default value - - - Post-Build Variant Multiplicity - true - Post-Build Variant Value - true - Multiplicity Configuration Class - Pre-compile time - X - VARIANT-PRE-↵ COMPILE - Link time - - - - Post-build time - X - VARIANT-P↵ OST-BUILD - Value Configuration Class - Pre-compile time - X - VARIANT-PRE- COMPILE - Link time - - - - Post-build time - X - VARIANT-P↵ OST-BUILD - Scope / Dependency - scope: local	Maximum value of this node is restricted from 18446744073709551615 to 9223372036854775807 due to EB Tresos limitation: "Deviating from the AUTOSAR traceable [TPS_GST_00008], the formula language interpreter of E↵ B tresos Studio supports only the signed 64-bit range for integer values, i.e. from 0x8000000000000000 (-9223372036854775808) to 0x7fffffffffffffff (9223372036854775807)." "The formula language parser of EB tresos Studio is implemented in Java. It uses the long primitive type for integer values and therefore can only support the restricted range."

Requirement	Status	Description	Notes
ECUC_Adc_00401	N/F	Name - AdcHwTrigTimer - Parent Container - AdcGroup - Description - Reload value of the ADC module embedded timer (only if supported by ADC hardware).ImplementationType↵ : Adc_HwTriggerTimerType - Multiplicity - 0..1 - Type - EcucIntegerParamDef - Range - 0 .. 18446744073709551615 - - Default value - - - Post-Build Variant Multiplicity - true - Post-Build Variant Value - true - Multiplicity Configuration Class - Pre-compile time - X - VARIANT-↵ PRE-COMPILE - Link time - - - - Post-build time - X - VARIANT-P↵ OST-BUILD - Value Configuration Class - Pre-compile time - X - VARIANT-PRE-↵ COMPILE - Link time - - - - Post-build time - X - VARIANT-P↵ OST-BUILD - Scope / Dependency - scope↵ : localdependency: AdcTriggSrcHw: Valid only if the group is configured to be triggered by a hardware event.	Maximum value of this node is restricted from 18446744073709551615 to 9223372036854775807 due to EB Tresos limitation: "Deviating from the AUTOSAR traceable [TPS_GST_00008], the formula language interpreter of E↵ B tresos Studio supports only the signed 64-bit range for integer values, i.e. from 0x8000000000000000 (-9223372036854775808) to 0x7fffffffffffffff (9223372036854775807)." "The formula language parser of EB tresos Studio is implemented in Java. It uses the long primitive type for integer values and therefore can only support the restricted range."

3.5 Driver Limitations

- Limitation of reports: mismatching prototypes in requirements for non-autosar APIs [Adc_CtuEnableNotification\(\)](#), [Adc_CtuDisableNotification\(\)](#), [Adc_CtuReadFifoResult\(\)](#).
- Hard fault error: the value of Internal Dma Buffer overflowed to other memory areas when the following scenario is configured: Transfer type = ADC_DMA; Group access mode = ADC_ACCESS_MODE_STREAMING + ADC_STREAM_BUFFER_CIRCULAR; Conversion mode = ADC_CONV_MODE_CONTINUOUS; Conversion type = ADC_CONV_TYPE_NORMAL; Adc Group Without Interrupt = TRUE; Channel count = 1 (number of channel in the group).

3.6 Driver usage and configuration tips

3.6.1 Calibration function

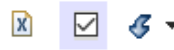
ADC driver provides [Adc_Calibrate\(\)](#) API to eliminate the errors between the actual values and the expected converted values, which might be generated by variations in manufacturing and various runtime environmental

effects. The API can be called explicitly by the user at any time after the driver was initialized and while there are no ongoing conversions. Calibration is not automatically started by initialization function.

Configuration-time-prerequisites:

- Enable `AdcEnableCalibration` in AutosarExt container.

[Adc Enable Calibration API](#)



3.6.2 Self-Test function

For safety applications, it is important to verify correct operation at regular intervals. ADC driver provides [Adc_SelfTest\(\)](#) API for this purpose to check its components and flags errors, if any are found. The API can be called explicitly by the user at any time after the driver was initialized and while there are no ongoing conversions.

Configuration-time-prerequisites:

- Enable `AdcEnableSelfTest` in AutosarExt container.

[Adc Enable Self Test API](#)



3.6.3 CTU control mode

- Control mode is used by enabling the control mode API from the configuration and by calling the [Adc_EnableCtuControlMode\(\)](#) function at runtime.
- In control mode the CTU assumes complete control over the ADC module and groups can no longer be used. If enabled, triggered mode cannot be used in parallel.
- Software triggered groups can be used on different ADC units which are not controlled by CTU. Additionally, software triggered conversions can be used on the ADC unit controlled by CTU using [Adc_CtuStartConversion\(\)](#).
- In control mode the CTU hardware is configured as specified directly by the user in the CTU configuration container and with dedicated APIs at runtime for certain parameters.
- The data can be obtained by calling functions dedicated for reading the FIFOs or data registers from the notifications, or [by DMA transfers](#)
- A notification can be configured for each CTU event.
- The CTU is configured only once when calling [Adc_EnableCtuControlMode\(\)](#) and not every time a hardware trigger is enabled.
- This mode enables the usage of functions prefixed with `Adc_Ctu`, e.g. `Adc_CtuReadConvData()`, [Adc_CtuEnableHwTrigger\(\)](#), etc.
- In Tressos, CTU control mode is enabled by ticking its box and this enables the configuration of the CTU HW unit(see image)

Adc Enable Self Test API



Adc CTU Control Mode API*



Adc Enable Analog Watchdog Functionality



CtuHwUnit

Name CtuHwUnit_0

General

Input Trigger

Ctu Trigger Configurations

Adc Command List

Ctu Result FIFOs

Input Trigger

Index	Name	Ctu Input Signal	Input Trigger Edge
0	CtuInputTrigConfigs_0	PWM_EVEN_1_PIT0_CH6	EDGE_RISING
1	CtuInputTrigConfigs_1	PWM_ODD_2_FTM1_CH0	EDGE_FALLING

3.6.4 CTU triggered mode

- In this mode the CTU is not directly configurable by the users. The CTU configuration is generated based on the group configured information
- Trigger mode is the default mode of triggering (if [Adc_EnableCtuControlMode\(\)](#) is not called)
- In Trigger mode the CTU is used strictly as a trigger source for ADC groups and the result data is read from the ADC module.
- This is done by configuring the CTU list to look exactly like the list of channels in the desired group and activating only the trigger source associated to that group.
- In the end of list interrupt coming from the CTU, the ADC driver copies and process the ADC data from the result registers of the ADC module and optionally calls a user notification mapped for the active group.
- The CTU is reconfigured each time [Adc_EnableHardwareTrigger\(\)](#) is called, so it triggers the channels in the same sequence as they are defined in the groups.
- In Tresos, the triggers must be configured from a separate tab.

☐ Adc Group Replacement
 ADC_GROUP_REPL_ABORT_RESTART

Adc Group Trigger Source
 ADC_TRIGG_SRC_HW

Adc Group Hw Trigger Source
 /Adc/Adc/AdcConfigSet/AdcHwTrigger_3

General | AdcHwUnit | **AdcHwTrigger** | CtuHwUnit | AdcEcucPartitionRef | AdcPowerStateConfig | AdcInterrupt | Published Information

AdcHwTrigger*

Index	Name	Adc Group Hardware Trigger Source
0	AdcHwTrigger_1	CTU_PWM_ODD_3_FTM1_CH1
1	AdcHwTrigger_2	CTU_PWM_ODD_1_FTM0_CH2
2	AdcHwTrigger_3	CTU_PWM_REL_FTM0_CH0
3	AdcHwTrigger_5	CTU_PWM_ODD_0_FTM0_CH1
4	AdcHwTrigger_6	CTU_PWM_ODD_2_FTM1_CH0
5	AdcHwTrigger_7	CTU_PWM_EVEN_0_FTM1_CH2

3.6.5 CTU triggered mode optimization ADC driver provides an optional parameter `AdcCtu←HardwareTriggerOptimization` to configure the CTU channel lists only once at initialization in order to reduce CPU load when calling the `Adc_EnableHardwareTrigger()` and `Adc_EnableCtuTrigger()` APIs. When enabling this optimization, the `Adc_SetChannel` service cannot be used and the total number of channels in all configured groups cannot exceed the size of CTU command list.

Adc Enable/Disable Channels API
 ☐

Adc Enable Configure Threshold API
 ☐

CTU Enable Hardware Trigger Optimization*
☒

Adc CTU Control Mode Enable DMA support
 ☐

Adc Enable Watchdog Api
 ☐

3.6.6 CTU control mode with reading the FIFO by DMA Transfer

In CTU Control Mode, driver provides the DMA transfer mechanism for transferring the conversion results directly from FIFO Data Registers to user buffer. This feature can be configured independently for each FIFO. The interrupts are not intended to be used with DMA.

There are 2 options transfer data from FIFO: Transfer Raw Data with full 32-bit FIFO and Transfer only DATA Field (excluded ADC Port and Channel). This one can be configured by enabling/disabling `CtuFifoDmaRawData` in CTU Container:

Fifo Dma Raw Data
☒
Disable output
☐

Error Notification
 NULL_PTR

MRS Notification
 NULL_PTR

Configuration-time-prerequisites:

- Enable CtuEnableDmaTransferMode and AdcEnableCtuControlModeApi in AutosarExt container.

Adc CTU Control Mode Enable DMA support		☒	
Adc Use Hardware Normal Groups		☐	
Adc Set HW Unit Power Mode API		☐	
Adc Get Injected Conversions Status API		☐	
Adc CTU Control Mode API		☒	

- In the FIFO Tab configuration, enable CtuFifoDmaEn and select the DMA Channel. Fill the Fifo Threshold Notification if needed, this one will be called from Mcl Notification.

Fifo Index		0	
Fifo Threshold (0 -> 8)		3	
DMA Enable		☒	
User Fifo Dma Buffer Pointer		CtuDmaFifo0	
Select Dma Channel for Fifo		/Mcl/Mcl/MclConfig/DMA_CH_FOR_FIFO0	
Fifo threshold notification		Fifo0ThresholdNotification	
Fifo empty notification		NULL_PTR	
Fifo overrun notification		NULL_PTR	
Fifo full notification		NULL_PTR	

- Mcl configuration for the logical channel used for CTU FIFO must have Ctu_Ip_FifoXDmaComplete Notification set as interrupt callback (with X defining which the FIFO Index used, for example Ctu_Ip_Fifo0←DmaComplete). Enable Global Config also needs to be checked.

Logic Channel Configuration		Global	Transfer	ScatterGather	Crc
Logic Channel Name	DMA_LOGIC_CH_0				
Hardware Instance	DMA_IP_HW_INST_0				
Hardware Channel	DMA_IP_HW_CH_0				
Interrupt Callback	Ctu_Ip_Fifo0DmaComplete				
Error Interrupt Callback	NULL_PTR				
Ecuc Partition Ref					
Enable Global Config	☒				
Enable Transfer Config	☐				
Enable Scatter/Gather	☐				
Enable CRC Config	☐				

- The DMA source must be configured as corresponding to FIFO.

Enable DMAMUX Trigger	☐	Enable DMAMUX Source	☒
DMAMUX0 Source	DMA_IP_REQ_MUX0_CTU_FIFO0		

3.6.7 DMA transfer

ADC driver provides the DMA transfer mechanism for transferring the conversion results directly from data registers to system memory via Direct Memory Access (DMA). This is opposed to Interrupt transfer mode, when ADC driver code moves the data from registers to user buffer. This feature can be configured independently for each ADC HW unit. The buffer containing conversion results should be declared within a NO_CACHEABLE data section.

Configuration-time-prerequisites:

- AdcEnableDmaTransferMode needs to be enabled in AutosarExt container.
- Configured AdcTransferType as ADC_DMA in AdcHwUnit container.
- Mcl configuration for the logical channel used for adc must have Adc_Ipw_AdcXDmaTransferComplete← Notification set as interrupt callback (with X defining the Adc instance used, for example Adc_Ipw_Adc0← DmaTransferCompleteNotification()), global config and scatter/gather enabled. The DMA source must be configured as the ADC instance used (SarAdc). Also, the number of ScatterGather elements configured for the Mcl logical channel must be equal to the maximum number of AdcChannels configured in Adc groups. The elements in the ScatterGather configuration must be linked consecutively (element n linked to element n+1), except the last element in the list that should be marked as such. The other parameters for the ScatterGather elements will be overwritten at runtime by the ADC driver.

Logic Channel Configuration

Global Transfer ScatterGather Crc

Logic Channel Name: DMA_LOGIC_CH_0

Hardware Instance: DMA_IP_HW_INST_0

Hardware Channel: DMA_IP_HW_CH_0

Interrupt Callback: Adc_Ipw_Adc0DmaTransferComplete

Error Interrupt Callback: NULL_PTR

Enable Global Config: ☒ Enable Transfer Config: ☐

Enable Scatter/Gather: ☒ Enable CRC Config: ☐

Note

- When using multi-core feature, the DMA channel should have the same partition as the ADC module.
- When using Enable Config Time Support feature, if ADC is configured as VariantPreCompile, Mcl cannot be configured as VariantPostBuild.
- AdcEnableDmaTransferMode shouldn't be configured simultaneously with Adc_SetChannel() or Adc_↔ EnableChannel()/Adc_DisableChannel() because there is a risk that DMA TCD size will be modified at runtime.
- When using DMA, the results transferred are 15-bit wide, same as Adc CDATA register, for any resolution selected.
- For groups having only one channel, the driver does not use the Scatter-Gather mode. So Scatter-Gather must be disabled in DMA Channel Configuration.

Logic Channel

Name  CHANNEL_FOR_ADC_0

Logic Channel Configuration Global Transfer ScatterGather

Logic Channel Name  DMA_LOGIC_CH_0 

Hardware Instance  DMA_IP_HW_INST_0  

Hardware Channel  DMA_IP_HW_CH_0  

Interrupt Callback  Adc_Ipw_Adc0DmaTransferCompleteNotificator 

Error Interrupt Callback  NULL_PTR 

 Ecuc Partition Ref 

Enable Global Config  ☒ 

Enable Transfer Config  ☐ 

Enable Scatter/Gather  ☐ 

3.6.8 Injected conversions

Adc driver supports to configure groups with conversion type Injected for high priority conversions. An injected conversion is a conversion chain that can be injected at any time, including into an ongoing normal conversion chain. Injected conversion can only be run in One-Shot mode, and can only interrupt a normal conversion. When an injected chain is inserted into an ongoing normal chain, the ongoing normal channel is suspended and the injected channel request is processed. After completing the last channel in the injected chain, normal conversion resumes from the channel at which it was suspended. Depending on the hardware capabilities, a Software Normal Group can be active in the Queue at the same time as a Hardware Injected group, or a Software Injected Group can be active at the same time with a Hardware Normal Group, so that 2 groups can be converted almost in parallel. For more details, please refer to the Reference Manual of the microcontroller, chapter Injected conversion.

Configuration-time-prerequisites:

- In AdcGroup container, configure AdcGroupConversionType as ADC_CONV_TYPE_INJECTED, AdcGroupConversionMode as ADC_CONV_MODE_ONESHOT and the AdcGroupPriority must be 255 (highest)
- If it is a Software group, AdcUseSoftwareInjectedGroups needs to be enabled in AutosarExt container

3.6.9 Set Channel Optimization

ADC driver provides an optional configuration parameter that allows the user to change the configuration of a group at runtime. According to Autosar specification, the group definition (list of channels) should be fixed at configuration time; if a different set of channels needs to be converted, another group must be started. However, in some applications stopping the group and starting another takes too long for the given time constraints. For this type of use case, Adc_SetChannel() can be used to quickly change the configuration of a group at runtime: the list of channels, and if applicable on the hardware platform - the set of associated conversion timing information for each channel.

Configuration-time-prerequisites:

- The `AdcEnableSetChannel` parameter needs to be enabled in AutosarExt container

Adc Set Channel API



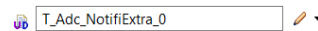
- If channels need to be updated in ISR notification, `AdcEnableInitialNotification` parameter can be enabled in AutosarExt container

Adc Initial Notification Capability



- The initial notification function needs to be enable for each Group where it needs to be used

Adc Group Extra Notification



`Adc_SetChannel()` API can be called by the user application whenever it is needed. In the interrupt routine that handled the conversion complete event, Adc driver will check if `Adc_SetChannel()` has been called and will update the required hardware registers to match the new configuration. The API can be called from the standard Autosar group conversion complete notification, but this notification has the disadvantage that it's called at the very end of ADC interrupt processing, leaving no time to do the updates in ADC hardware registers. So the very next conversion will not be affected by the changes, but the one after that will be.

Note

- If the `Adc_SetChannel()` is called before ADC receives the first hardware trigger signal and after calling `Adc_EnableHardwareTrigger()`, the results of this trigger will not be correct since the new list of channels will be updated in the interrupt of first trigger. To start a conversion with the new list, another signal needs to be provided.
- In case of calling `Adc_SetChannel()` after `Adc_StartGroupConversion()`, the list that will be converted depends on total conversion time. If the interrupt event happened before executing `Adc_SetChannel()`, the old list will be converted. Otherwise, if the total conversion time is longer than the time required to execute both functions (`Adc_SetChannel()` and `Adc_StartGroupConversion()`), then the results of the new list will be recorded.
- Sample index will be reset and result data will be overwritten starting from the first sample after runtime channels are updated in ISR.
- The feature `AdcEnableInitialNotification` can be used by the user application to call `Adc_SetChannel()` API before ADC driver updates the hardware configuration for the next conversion.
- Since Channel index field of CTU command list only supports 4 bits width, physical channel index cannot be greater than 15 if `Adc_SetChannel()` is used for CTU HW trigger group. Ctu Dev Error Detect could also be enabled to check this issue at runtime

3.6.10 Enable/Disable channel

The ADC driver provides optional functions `Adc_EnableChannel()` and `Adc_DisableChannel()` to enable/disable one ADC channel of a given software triggered (non-injected) ADC group.

Configuration-time-prerequisites:

- The Adc Enable/Disable Channels API parameter needs to be enabled in AutosarExt container

[Adc Enable/Disable Channels API](#)



Note

- The user should use generated symbolic channel name defines (e.g. `AdcChannel_0_0`), because the API assumes that `ChannelId` has the following format: logical channel ID on bits until position defined by `ADC_CHANNEL_SYMBOLIC_NAME_SHIFT_HW_UNIT_ID_U16`, and logical unit ID from that position
- The driver will not update the values in result buffers corresponding to disabled channels, keeping in the buffer the last results from when the channel was enabled.
- Adc Enable/Disable Channels API and Adc Set Channel API cannot be enabled simultaneously

3.6.11 Conversion Time Once

ADC driver provides Conversion Time Once feature to enable/disable configuring conversion time only when initializing the ADC module. If this feature is enabled, this setting will be done only once in `Adc_Init()` with values configured per unit, in `AdcHwUnit` container. If disabled, this setting will be done whenever a group is started, using values from `AdcGroup` container. When the feature is enabled, the latency of APIs that start group conversions will be reduced. If `Adc_SetClockMode()` API is also used, the driver allows the user to configure alternate timing settings (for ALTERNATE mode), both at Unit and Group level.

Configuration-time-prerequisites:

- Enable `AdcConvTimeOnce` in AutosarExt container
- Configure the values will be used in `AdcNormalConvTiming` or `AdcNormalAlternateTiming` (if `Adc_SetClockMode()` was invoked before with Alternate mode) in `AdcHwUnit` container

Note

- The `AdcPreSamplingOnce` feature is always enabled from configurator but readonly. `Adc_Init()` will initialize once the presample for all channels in `AdcChannel` container per unit.

3.6.12 Bypass Abort Chain Check

When a conversion chain is aborted, ADC driver will block program execution to wait for the conversion chain to be stopped. This wait can be skipped by enabling `AdcBypassAbortChainCheck`. Doing this will speed up program execution at the cost of HW-SW coherency no longer being guaranteed. In this case, the user must make sure he does not call an ADC service before the hardware reaches the correct state.

Configuration-time-prerequisites:

- Enable `AdcBypassAbortChainCheck` in AutosarExt container

[Adc Bypass Abort Chain Check](#)



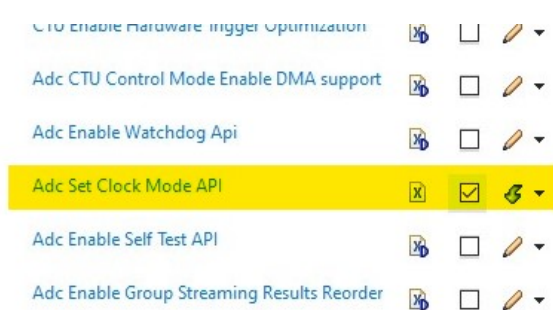
Note

This can be enabled if no queues are being used.

3.6.13 Set Clock Mode

ADC driver provides an optional API [Adc_SetClockMode\(\)](#) and configuration parameters to compensate the changes of the input clock of the controlled hardware. By enabling this feature, beside the normal clock configuration set, user can configure an alternate clock configuration set and can switch between normal and alternate at runtime using [Adc_SetClockMode\(\)](#). One clock configuration set includes the ADC parameters affected by clock configuration, such as: ADC power down delay, ADC prescale and sampling duration. Configuration-time-prerequisites:

- To use the set clock mode feature, "Adc Set Clock Mode API" needs to be enabled in AutosarExt container first:



- Enable and configure the alternate clock configuration set at HW unit level. If the Conversion Time Once feature is enabled, normal and alternate sampling duration can be configured as well.

AdcHwUnit

Name  AdcHwUnit_0

General | AdcChannel | AdcGroup | AdcThresholdControl | AdcHwUnitEcucPartitionRef

 Adc Prescaler Value  1  

Adc Alternate Prescale  2  

Adc Calibration Prescale  2  

Adc Power Down Delay (0 -> 255)  15  

Adc Alternate Power Down Delay (0 -> 255)  15  

Adc Auto Clock Off  ☐   Adc Bypass Sampling  ☐  

Adc Presampling channel (0 - 31)  DVDD  

Adc Presampling channel (32 - 63)  DVDD  

Adc Presampling channel (64 - 95)  DVDD 

User Offset (0 -> 255)  0  

User Gain (0 -> 65535)  0  

DMA Clear Source  ADC_SAR_IP_DMA_REQ_CLEAR_ON_ACK  

AdcNormalConvTimings

 Name  AdcNormalConvTimings

Adc Unit Normal Sampling Duration 0 (8 -> 255)  100  

Adc Unit Normal Sampling Duration 1 (8 -> 255)  200  

AdcAlternateConvTimings

 Name  AdcAlternateConvTimings

Adc Unit Alternate Sampling Duration 0 (8 -> 255)  255  

Adc Unit Alternate Sampling Duration 1 (8 -> 255)  8  

- Enable and configure alternate clock configuration set at group level, if Conversion Time Once feature is disabled.

▼

AdcGroupConversionConfiguration

Name

AdcGroupConversionConfiguration

Adc Group Sampling Duration 0 (8 -> 255)

255

↺

Adc Group Sampling Duration 1 (8 -> 255)

20

↺

▼

AdcAlternateGroupConvTimings

Name

AdcAlternateGroupConvTimings

Adc Group Alternate Sampling Duration 0 (8 -> 255)

20

↺

Adc Group Alternate Sampling Duration 1 (8 -> 255)

10

↺

3.6.14 Analog Watchdog feature

ADC driver provides the Analog Watchdog feature for determining whether the result of a conversion lies within a given guard area specified by an upper and a lower threshold value named THRH and THRL, respectively. The upper/lower values are set at configuration time and can be changed at runtime via the API [Adc_ConfigureThreshold\(\)](#). Each channel can be configured to use this feature or not. If the result of a conversion does not lie within the configured range, an interrupt will occur and a user configured notification will be called. For more details about Analog Watchdog feature see the reference manual of the microcontroller. ADC driver also provide optional functions to enable/disable the analog watchdog event notification of each configured ADC channel: [Adc_EnableWdgNotification\(\)](#) and [Adc_DisableWdgNotification\(\)](#)

Configuration-time-prerequisites:

- To use the Analog Watchdog feature, AdcEnableWatchdogFunctionality needs to be enabled in AutosarExt container first:

Adc Cfg Control Mode API

☐

↺

Adc Enable Analog Watchdog Functionality

☒

↺

Adc Use Hardware Normal Groups

☐

↺

- Configure the channel to use Analog Watchdog feature in AdcChannel container. Enable AdcEnableThresholds and configure the AdcThresholdRegister and AdcWdgNotification:

Adc Enable Presampling

☐

↺

Adc Threshold Control Register

/Adc/Adc/AdcConfigSet/AdcHwUnit_0/AdcThresholdControl_0

Adc Channel Watchdog Notification

Notification_WDG_0

↺

Adc Enable Threshold

☒

↺

- The upper/lower threshold values need to be configured in AdcThresholdControl container:

AdcThresholdControl

Name  AdcThresholdControl_0

General

Adc Threshold Register  ADC_THRESHOLD_REG_1 

 Adc High Threshold value  4095 

 Adc Low Threshold value  0 

- To use [Adc_ConfigureThreshold\(\)](#), AdcEnableThresholdConfiguration in AutosarExt container needs to be enabled:

[Adc Configure Threshold](#)   

[Adc Enable Self Test API](#)   

Note

- When using [Adc_EnableWdgNotification\(\)](#) and [Adc_DisableWdgNotification\(\)](#), symbolic channel name defines should be used (e.g. AdcChannel_0_0), because these functions assumes that ChannelId has the following format: logical channel ID on bits until position defined by ADC_CHANNEL_SYMBOLIC_NAME_SHIFT_HW_UNIT_ID_U16, and logical unit ID from that position
- When using [Adc_ConfigureThreshold\(\)](#), symbolic threshold control name defines should be used (e.g. AdcThresholdControl_0_0), because the function assumes that ThresholdControlIndex has the following format: logical threshold ID on bits until position defined by ADC_THRESHOLD_SYMBOLIC_NAME_SHIFT_HW_UNIT_ID_U16, and logical unit ID from that position

3.6.15 Optimize OneShot HwTrigger Conversions

ADC driver provides a configuration option to enable maximum speed optimizations for conversion processing of ADC groups configured with ADC_CONV_MODE_ONESHOT and ADC_TRIGG_SRC_HW. When this parameter is enabled, no other type of ADC group may be configured.

Configuration-time-prerequisites:

- Enable AdcOptimizeOneShotHwTriggerConversions in AutosarExt container:

[Adc Optimize OneShot HwTrigger Conversions](#)   

Note

Some restrictions are applied when using this feature:

- Only hardware trigger, single access mode groups could be configured.

3.6.16 Without Interrupts group

ADC driver provides this feature as a mechanism to reduce the CPU load. The basic functionality of ADC driver requires that an interrupt event occurs after each group conversion, during which the software will copy the data from data registers to the user buffer and update the Group status. If Without Interrupts feature is configured, the conversions can run without software intervention (without any interrupt events generated).

Configuration-time-prerequisites:

- Enable `AdcWithoutInterrupts` for the group using without interrupt feature in `AdcGroup` container:

Adc Group Without Interrupts



For the case of groups configured with `ADC_ACCESS_MODE_SINGLE` the application can retrieve the results by calling `Adc_ReadGroup()`. The result buffer configured by user via `Adc_SetupResultBuffer()` is no longer used to store the results, but must still be registered to avoid raising DET required by AUTOSAR Standard. If HW unit transfer type is configured as `INTERRUPT` (not with DMA), `Adc_ReadGroup()` reads the results directly from hardware registers (if the flags indicate that the group conversion was completed) and updates the group state afterwards. If HW unit transfer type is set to `DMA`, the results are transferred via DMA into an internal buffer. `Adc_ReadGroup()` reads the results from this buffer and updates the group state afterwards.

Note

- The Without Interrupt optimization is also supported for groups configured with `ADC_ACCESS_MODE_STREAMING`, with following restrictions: HW unit transfer type is `DMA`, group buffer mode is `CIRCULAR` and group has enabled Streaming Results Reorder (requires Group Streaming Results Reorder to be enabled in General AutosarExt). In this case a second DMA channel is required to be configured (named Select Adc Streaming Dma Channel) and the results are transferred in 2 stages:
 1. Conversion results for 1 sample of all channels are transferred into an internal buffer via the first DMA channel configured with ScatterGather. IMPORTANT NOTE: this channel must be configured as described in [DMA transfer](#), with only one difference: the last ScatterGather element in the array must be linked to the first one, creating a circular scatter gather list. This introduces the limitation that on the same HW unit groups having without interrupt `DISABLED` cannot be used. Also all groups having without interrupt `ENABLED` must have the same number of channels as elements in ScatterGather list.

ScatterGather Element

Name dmaLogicChannelConfig_ScatterGatherArrayType_1

General

▼ Element

Name dmaLogicChannelConfig_ScatterGatherElementConfigType_1

Element Name DMA_LOGIC_CH_0_SGA_ELEMENT_1

Last Element of the Link

Element Link /Mcl/MclConfig/CHANNEL_FOR_ADC_0/dmaLogicChannel_ConfigType/dmaLogicChannel_ScatterGatherConfigType/dmaLogicChannelConfig_ScatterGatherArrayType_0

Enable Scatter/Gather Config ☐

2. Transfer complete of the last scatter gather element (when last ADC result for the current sample is available) triggers Streaming Dma Channel which transfers results for 1 sample from the internal buffer into the buffer registered by user via `Adc_SetupResultBuffer()`. The ADC driver configures the DMA channel to also support Streaming Number Samples > 1. The configuration required for the Streaming Dma Channel is shown in picture below:

Name COUNTING_CHANNEL_FOR_ADC_0

Logic Channel Configuration Global Transfer ScatterGather Crc

▼ dmaLogicChannel_GlobalConfigType

Name dmaLogicChannel_GlobalConfigType

▼ Control

Name dmaLogicChannelConfig_GlobalControlType

Enable Master Id Replication ☐ Enable Buffered Writes ☐

▼ Request

Name dmaLogicChannelConfig_GlobalRequestType

Enable DMAMUX Trigger ☐ Enable DMAMUX Source ☒

DMAMUX0 Source DMA_IP_REQ_MUX0_DISABLED

DMAMUX1 Source DMA_IP_REQ_MUX1_DISABLED

DMAMUX2 Source DMA_IP_REQ_MUX2_DISABLED

DMAMUX3 Source DMA_IP_REQ_MUX3_DISABLED

Enable DMA Request ☐

- For hardware triggered groups configured with `ADC_ACCESS_MODE_SINGLE`, multiple channels and transfer type set to DMA, to support multiple occurrences of a trigger, the ScatterGather elements need to be configured the same as for `ADC_ACCESS_MODE_STREAMING` (described above, at point 1). For software triggered groups this is not mandatory. Multiple calls to `Adc_StartGroupConversion()` are supported in both cases, because the TCD is reconfigured in `Adc_StartGroupConversion()`.

3.6.17 Without Dma group

When this feature is disabled and DMA transfer is used, the ADC driver reconfigures the DMA channel at each call to `Adc_StartGroupConversion()` or `Adc_EnableHardwareTrigger()`. This feature allows complete bypass of the DMA configuration done by ADC driver for the group, so it will not affect other groups for which the ADC driver has already configured the DMA. This feature can be useful for example when starting a normal software group while an injected hardware trigger with `ADC_ACCESS_MODE_STREAMING` is already enabled - to allow the hardware triggered group to execute correctly, the DMA configuration for the software triggered group must be bypassed to not reconfigure the DMA channel of the active hardware triggered group. The results for the software group configured as Without Dma can be read with `Adc_ReadGroup()`, but the channels of the 2 groups must not be overlapped. Otherwise results will not be read, because the valid bit of the result register will be cleared by DMA transfers. The feature can only be enabled if following conditions are met:

- ADC Unit Transfer Type is `ADC_DMA`
- Only for SW, one shot, single access mode groups
- Group Without Interrupts is enabled

3.6.18 External DMA channel feature

The ADC driver shall support configuring the DMA channel externally by using a boolean parameter named "↔ External Dma Chan Config" at group configuration level. In this mode, ADC driver will only enable/disable the DMA request for the unit. The user must configure the DMA to transfer data according to his needs.

Configuration-time-prerequisites:

- To use the External DMA channel feature, `AdcExtDMACHanEnable` needs to be enabled in each group container first:

Adc External Dma Chan Config



Note

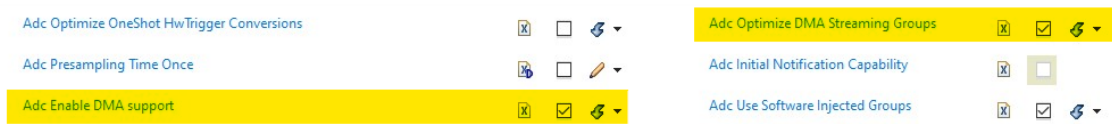
- When External Dma Chan Config is enabled, the driver will not configure any TCDs for DMA
- The user still needs to register `Adc_Ipw_Adc0DmaTransferCompleteNotification()` as a notification for DMA, but if External Dma Chan Config is enabled for that group, the function only updates the group status and calls the `GroupNotification`.
- If a group is in streaming access mode, the code for updating the destination in `Adc_Ipw_Adc0Dma↔ TransferCompleteNotification()` (ie: update destination address of TCD, enable DMA hardware request) will not be executed for that group so this will need to be handled by the user in the notification
- If this feature is enabled simultaneously with `AdcWithoutInterrupts` ([Without Interrupts group](#)), the user is not required to register `Adc_Ipw_Adc0DmaTransferCompleteNotification()` as a notification for DMA, because the group status is updated when calling `Adc_ReadGroup()`. It is required for the application to register a result buffer via `Adc_SetupResultBuffer()`, but the buffer and DMA transfers are completely managed by the application, so the actual buffer is used only by `Adc_ReadGroup()`. `Adc_ReadGroup()` assumes that the registered buffer is one dimensional, with number of elements equal to the number of channels in the group and that it stores the latest conversion result (latest sample). This assumption is required to determine if latest conversion has occurred before updating the group status. Additionally, for groups configured with Ext DMA and Without interrupt, the `Adc_SetupResultBuffer()` resets the buffer to an invalid value so `Adc_ReadGroup()` can determine if the result has been transferred and status should be updated. Note that if updating the group status is not required, the user can read directly from the DMA destination buffer without calling `Adc_ReadGroup()`.

3.6.19 Optimize DMA streaming groups

The ADC driver provides an optional configuration parameter for reducing the number of DMA interrupts required for processing the conversions of ADC groups that are configured with `ADC_ACCESS_MODE_STREAMING` option. Instead of having an interrupt after every end of chain group conversion, only one or two interrupts will be raised for a stream of N conversions: one after N/2 conversions, and one after all N conversions are completed. With this feature enabled, if a group has more than one channel, a second DMA channel is required, named streaming DMA channel. In this case, the driver will use 2 DMA channels configured: transferring DMA channel and streaming DMA channel. Transferring DMA channel is used for moving data from ADC data register (CDR) to an intermediate buffer. Streaming DMA acts as a hardware counter which has CITER/BITER value equal to group sample as configured by `AdcStreamingNumSamples` parameter. After every end of chain conversion, ADC hardware will trigger a request to transferring DMA channel to complete its major loop. After that, transferring DMA will trigger streaming DMA by a Major Elink request. Streaming DMA will transfer data from intermediate buffer to user buffer and also decrease DMA CITER count by 1. When all ADC samples are completed, the streaming DMA channel will finish its major loop and automatically disable the hardware request from DMA (this is done by disabling DMA in `Adc_Ipw_↔ Adc0DmaTransferCompleteNotification()`). Both interrupts occurring after half of conversions are done and after all of them are done will be raised by the streaming DMA channel.

Driver

- If this feature is enabled with a group having only a single channel, driver will only use transferring DMA channel.
- The driver implementation is similar to configuring groups without interrupt, streaming access mode with result reordered. (Referring to [Without Interrupts group](#)) Configuration-time-prerequisites:
- Enable both Adc Enable DMA support and Adc Optimize DMA Streaming Groups:



- Configure AdcTransferType as ADC_DMA type and select transferring DMA channel (and also streaming DMA channel if there is at least one ADC group enabled with Optimize DMA streaming groups with more than one ADC channel configured).

AdcHwUnit

Name AdcHwUnit_1

General | AdcChannel | AdcGroup | AdcThresholdControl | AdcHwUnitEcucPartitionRef

Adc Hardware Unit ADC1

Adc Logical Unit ID 0

Adc Transfer Type ADC_DMA

Select Dma Channel /Mcl/Mcl/MclConfig/CHANNEL_FOR_ADC_1

Select Adc Streaming Dma Channel /Mcl/Mcl/MclConfig/COUNTING_CHANNEL_FOR_ADC_1

- Transferring DMA needs to be enabled DMA MUX source (e.g: DMA_IP_REQ_MUX1_SARADC1) and Streaming DMA needs to have Interrupt Callback selected (e.g: Adc_Ipw_Adc1DmaTransferComplete←Notification()) equivalent to the ADC hardware unit is being used

Logic Channel

Name  CHANNEL_FOR_ADC_1

Logic Channel Configuration | Global | Transfer | ScatterGather | Crc

Logic Channel Name  DMA_LOGIC_CH_1

Hardware Instance  DMA_IP_HW_INST_0

Hardware Channel  DMA_IP_HW_CH_16

Interrupt Callback  NULL_PTR

Error Interrupt Callback  NULL_PTR

Ecuc Partition Ref 

Enable Global Config  ☒  Enable Transfer Config  ☐

Enable Scatter/Gather  ☒  Enable CRC Config  ☐

Logic Channel

Name  CHANNEL_FOR_ADC_1

Logic Channel Configuration | Global | Transfer | ScatterGather | Crc

Request

Name  dmaLogicChannelConfig_GlobalRequestType

Enable DMAMUX Trigger  ☐  Enable DMAMUX Source  ☒ 

DMAMUX0 Source  DMA_IP_REQ_MUX0_DISABLED

DMAMUX1 Source  DMA_IP_REQ_MUX1_SARADC1

DMAMUX2 Source  DMA_IP_REQ_MUX2_DISABLED

DMAMUX3 Source  DMA_IP_REQ_MUX3_DISABLED

Enable DMA Request  ☐ 

Logic Channel

Name  COUNTING_CHANNEL_FOR_ADC_1

Logic Channel Configuration | Global | Transfer | ScatterGather | Crc

Logic Channel Name  DMA_LOGIC_CH_7

Hardware Instance  DMA_IP_HW_INST_0

Hardware Channel  DMA_IP_HW_CH_17

Interrupt Callback  Adc_Ipw_Adc1DmaTransferComplete

Error Interrupt Callback  NULL_PTR

Ecuc Partition Ref 

Enable Global Config  ☒  Enable Transfer Config  ☐ 

Enable Scatter/Gather  ☐  Enable CRC Config  ☐

Logic Channel

Name  COUNTING_CHANNEL_FOR_ADC_1

Logic Channel Configuration | Global | Transfer | ScatterGather | Crc

Request

Name  dmaLogicChannelConfig_GlobalRequestType

Enable DMAMUX Trigger  ☐  Enable DMAMUX Source*  ☐ 

DMAMUX0 Source  DMA_IP_REQ_MUX0_DISABLED

DMAMUX1 Source*  DMA_IP_REQ_MUX1_DISABLED

DMAMUX2 Source  DMA_IP_REQ_MUX2_DISABLED

DMAMUX3 Source  DMA_IP_REQ_MUX3_DISABLED

Enable DMA Request  ☐ 

- If a group has multiple channels, transferring DMA channel will be configured in Scatter-Gather mode with the number of elements equal to the number of channels in that group. These elements need to be configured as a ring with no last element configured.

Logic Channel

Name  CHANNEL_FOR_ADC_1

Logic Channel Configuration | Global | Transfer | ScatterGather | Crc

ScatterGather

Index	Name	Element Name	Last Element of the Link
0	 dmaLogicChannelConfig_ScatterGatherArrayType_0	 DMA_LOGIC_CH_1_SGA_ELEMENT_0	 ☒
1	 dmaLogicChannelConfig_ScatterGatherArrayType_1	 DMA_LOGIC_CH_1_SGA_ELEMENT_1	 ☒

This introduces the limitation that on the same HW unit groups having without interrupt DISABLED cannot be used.

- Global channel linking also need to be enabled for this feature

Adc (Adc) Mcl (Mcl)

dmaLogicInstance_ConfigType

Name dmaLogicInstance_ConfigType_0

Instance

Logic Instance Name DMA_LOGIC_INST_0

Hardware Instance DMA_IP_HW_INST_0

Debug ☐ Round Robin Channel Arbitration ☐

Halt After Error ☐ Global Channel linking ☒

Global Master ID Replication ☐

☐ Ecuc Partition Ref

Dma Crc

Name dmaLogicInstance_DmaCrc

Enable Swap Bit ☐ Enable Swap Byte ☐

Enable Global ☐

- At group level, Adc Enable Optimize DMA Streaming Groups (and Adc Group Enable Half Interrupt if half interrupt is used) need to be enabled.

Adc Group Streaming Buffer Mode ADC_STREAM_BUFFER_LINEAR

Adc Enable Optimize DMA Streaming Groups ☒

Adc Group Enable Half Interrupt ☒

Adc Group Streaming Number Samples 10

- If a group has configured more than one channel, Adc Group Streaming Results Reorder is also required.

Adc Enable Optimize DMA Streaming Groups ☒

Adc Group Streaming Number Samples 10

Adc Group Streaming Results Reorder ☒

Note

- Adc_SetChannel(), Adc_EnableChannel(), Adc_DisableChannel() and Optimize one-shot hardware trigger cannot be use concurrently with this feature.

3.7 Runtime errors

The driver generates the following DET runtime errors at runtime.

Function	Error Code	Condition triggering the error
Adc_Init()	ADC_E_TIMEOUT	Driver initialization timed out. ADC hardware could not enter power down or idle state.
Adc_DeInit()	ADC_E_TIMEOUT	Timed out. ADC hardware could not enter power down or idle state.
Adc_StartGroupConversion()	ADC_E_TIMEOUT	Timed out. Ongoing conversion could not be aborted.
Adc_StopGroupConversion()	ADC_E_TIMEOUT	Timed out. Ongoing conversion could not be aborted.
Adc_SetHwUnitPowerMode()	ADC_E_TIMEOUT	Timed out. ADC hardware could not enter/exit power mode or idle state.
Adc_SetClockMode()	ADC_E_TIMEOUT	Timed out. ADC hardware could not enter power down or idle state.
Adc_SetClockMode()	ADC_E_UNINIT	Global pointer is NULL.
Adc_SetClockMode()	ADC_E_INVALID_CLOCK_MODE	Clock mode is not correctly set.
Adc_Calibrate()	ADC_E_TIMEOUT	Timed out. Calibration operation timed out.
Adc_SelfTest()	ADC_E_TIMEOUT	Timed out. Self test operation timed out.

3.8 Symbolic Names Disclaimer

All containers having symbolicNameValue set to TRUE in the AUTOSAR schema will generate defines like:

```
#define <Mip>Conf_<Container_ShortName>_<Container_ID>
```

For this reason it is forbidden to duplicate the names of such containers across the RTD configurations or to use names that may trigger other compile issues (e.g. match existing #ifdefs arguments).

Chapter 4

Tresos Configuration Plug-in

This chapter describes the Tresos configuration plug-in for the driver. All the parameters are described below.

- Module [Adc](#)
 - Container [AdcConfigSet](#)
 - * Container [AdcHwUnit](#)
 - Parameter [AdcHwUnitId](#)
 - Parameter [AdcLogicalUnitId](#)
 - Parameter [AdcTransferType](#)
 - Parameter [AdcClockSource](#)
 - Parameter [AdcPrescale](#)
 - Parameter [AdcAltPrescale](#)
 - Parameter [AdcCalibrationPrescale](#)
 - Parameter [AdcPowerDownDelay](#)
 - Parameter [AdcAltPowerDownDelay](#)
 - Parameter [AdcAutoClockOff](#)
 - Parameter [AdcBypassSampling](#)
 - Parameter [AdcHwUnitOverwriteEn](#)
 - Parameter [AdcPresamplingInternalSignal0](#)
 - Parameter [AdcPresamplingInternalSignal1](#)
 - Parameter [AdcPresamplingInternalSignal2](#)
 - Parameter [AdcHwUnitUsrOffset](#)
 - Parameter [AdcHwUnitUsrGain](#)
 - Parameter [AdcHwUnitDmaClearSource](#)
 - Reference [AdcDmaChannelId](#)
 - Reference [AdcCountingDmaChannelId](#)
 - Reference [AdcHwUnitEcucPartitionRef](#)
 - Container [AdcSelfTestThresholdConfiguration](#)
 - Parameter [AdcSTAW0RSelfTestHighThresholdValue](#)
 - Parameter [AdcSTAW0RSelfTestLowThresholdValue](#)
 - Parameter [AdcSTAW1ARSelfTestHighThresholdValue](#)
 - Parameter [AdcSTAW1ARSelfTestLowThresholdValue](#)
 - Parameter [AdcSTAW1BRSelfTestHighThresholdValue](#)

- Parameter [AdcSTAW1BRSelfTestLowThresholdValue](#)
- Parameter [AdcSTAW2RSelfTestLowThresholdValue](#)
- Parameter [AdcSTAW4RSelfTestHighThresholdValue](#)
- Parameter [AdcSTAW5RSelfTestHighThresholdValue](#)
- Container [AdcNormalConvTimings](#)
- Parameter [AdcSamplingDurationNormal0](#)
- Parameter [AdcSamplingDurationNormal1](#)
- Container [AdcAlternateConvTimings](#)
- Parameter [AdcSamplingDurationAlt0](#)
- Parameter [AdcSamplingDurationAlt1](#)
- Container [AdcChannel](#)
- Parameter [AdcLogicalChannelId](#)
- Parameter [AdcChannelName](#)
- Parameter [AdcChannelId](#)
- Parameter [AdcChannelConvTime](#)
- Parameter [AdcChannelLimitCheck](#)
- Parameter [AdcChannelHighLimit](#)
- Parameter [AdcChannelLowLimit](#)
- Parameter [AdcChannelRangeSelect](#)
- Parameter [AdcChannelRefVoltsrcHigh](#)
- Parameter [AdcChannelRefVoltsrcLow](#)
- Parameter [AdcChannelResolution](#)
- Parameter [AdcChannelSampTime](#)
- Parameter [AdcEnablePresampling](#)
- Parameter [AdcEnableThresholds](#)
- Parameter [AdcWdogNotification](#)
- Reference [AdcThresholdRegister](#)
- Container [AdcGroup](#)
- Parameter [AdcGroupAccessMode](#)
- Parameter [AdcGroupConversionMode](#)
- Parameter [AdcGroupConversionType](#)
- Parameter [AdcGroupId](#)
- Parameter [AdcGroupPriority](#)
- Parameter [AdcGroupReplacement](#)
- Parameter [AdcGroupTriggSrc](#)
- Parameter [AdcHwTrigSignal](#)
- Parameter [AdcHwTrigTimer](#)
- Parameter [AdcNotification](#)
- Parameter [AdcExtraNotification](#)
- Parameter [AdcStreamingBufferMode](#)
- Parameter [AdcEnableOptimizeDmaStreamingGroups](#)
- Parameter [AdcEnableHalfInterrupt](#)
- Parameter [AdcStreamingNumSamples](#)
- Parameter [AdcStreamResultGroup](#)
- Parameter [AdcEnableChDisableChGroup](#)
- Parameter [AdcWithoutInterrupts](#)
- Parameter [AdcWithoutDma](#)

- Parameter [AdcExtDMAChanEnable](#)
- Reference [AdcGroupHwTriggerSource](#)
- Reference [AdcGroupDefinition](#)
- Reference [AdcGroupEcucPartitionRef](#)
- Container [AdcGroupConversionConfiguration](#)
- Parameter [AdcSamplingDuration0](#)
- Parameter [AdcSamplingDuration1](#)
- Container [AdcAlternateGroupConvTimings](#)
- Parameter [AdcAltGroupSamplingDuration0](#)
- Parameter [AdcAltGroupSamplingDuration1](#)
- Container [AdcThresholdControl](#)
- Parameter [AdcThresholdControlRegister](#)
- Parameter [AdcHighThreshold](#)
- Parameter [AdcLowThreshold](#)
- * Container [AdcHwTrigger](#)
 - Parameter [AdcHwTrigSrc](#)
- * Container [CtuHwUnit](#)
 - Parameter [CtuHwUnitId](#)
 - Parameter [CtuLogicalUnitId](#)
 - Parameter [CtuTGSMODE](#)
 - Parameter [CtuInputClockPrescaler](#)
 - Parameter [CtuExtTrigMode](#)
 - Parameter [CtuTgsCounterCompareVal](#)
 - Parameter [CtuTgsCounterReloadVal](#)
 - Parameter [CtuAdcCmdListMode](#)
 - Parameter [CtuSeqModeMrsInput](#)
 - Parameter [CtuSeqModeMrsInputEdge](#)
 - Parameter [CtuDmaDoneGRE](#)
 - Parameter [CtuDmaReqMRS](#)
 - Parameter [CtuFifoDmaRawData](#)
 - Parameter [CtuDisableOutput](#)
 - Parameter [CtuErrorNotif](#)
 - Parameter [CtuMrsNotif](#)
 - Parameter [CtuAdcCmdIssueNotif](#)
 - Parameter [CtuDigitalFilter](#)
 - Parameter [CtuControlOnTime](#)
 - Container [ExpectedConvDurationConfig](#)
 - Parameter [CtuExpectedValuePortA](#)
 - Parameter [CtuExpectedNotifPortA](#)
 - Parameter [CtuExpectedValuePortB](#)
 - Parameter [CtuExpectedNotifPortB](#)
 - Parameter [CtuConvDurationCounterRange](#)
 - Container [CtuInputTrigConfigs](#)
 - Parameter [CtuInputTrigEdge](#)
 - Reference [CtuInputTrigSelect](#)
 - Container [CtuTriggerCfg](#)
 - Parameter [CtuTriggerIndex](#)

- Parameter [CtuCompareVal](#)
- Parameter [CtuCmdListStartAdr](#)
- Parameter [CtuOutputTrigEnMask](#)
- Parameter [CtuTriggerNotif](#)
- Container [CtuAdcCommandList](#)
- Parameter [CtuIntEn](#)
- Parameter [CtuFifoIdx](#)
- Parameter [CtuConvMode](#)
- Parameter [CtuLastCmd](#)
- Parameter [CtuAdcPort](#)
- Parameter [CtuAdcChanA](#)
- Parameter [CtuAdcChanB](#)
- Container [CtuResultFifos](#)
- Parameter [CtuFifoIndex](#)
- Parameter [CtuFifoThreshold](#)
- Parameter [CtuFifoDmaEn](#)
- Parameter [CtuFifoDmaBuffer](#)
- Parameter [CtuFifoThresholdNotif](#)
- Parameter [CtuFifoUnderrunNotif](#)
- Parameter [CtuFifoOverrunNotif](#)
- Parameter [CtuFifoFullNotif](#)
- Reference [CtuFifoDmaChannelId](#)
- Container [AdcGeneral](#)
 - * Parameter [AdcDeInitApi](#)
 - * Parameter [AdcDevErrorDetect](#)
 - * Parameter [AdcEnableLimitCheck](#)
 - * Parameter [AdcEnableQueuing](#)
 - * Parameter [AdcPriorityQueueMaxDepth](#)
 - * Parameter [AdcEnableStartStopGroupApi](#)
 - * Parameter [AdcGrpNotifCapability](#)
 - * Parameter [AdcHwTriggerApi](#)
 - * Parameter [AdcPriorityImplementation](#)
 - * Parameter [AdcReadGroupApi](#)
 - * Parameter [AdcResultAlignment](#)
 - * Parameter [AdcVersionInfoApi](#)
 - * Parameter [AdcLowPowerStatesSupport](#)
 - * Parameter [AdcPowerStateAsynchTransitionMode](#)
 - * Reference [AdcEcucPartitionRef](#)
 - * Reference [AdcKernelEcucPartitionRef](#)
 - * Container [AdcPowerStateConfig](#)
 - Parameter [AdcPowerState](#)
 - Parameter [AdcPowerStateReadyCbRef](#)
- Container [AdcInterrupt](#)

- * Parameter [AdcInterruptSource](#)
- * Parameter [AdcInterruptEnable](#)
- Container [AdcPublishedInformation](#)
 - * Parameter [AdcChannelValueSigned](#)
 - * Parameter [AdcGroupFirstChannelFixed](#)
 - * Parameter [AdcMaxChannelResolution](#)
- Container [CommonPublishedInformation](#)
 - * Parameter [ArReleaseMajorVersion](#)
 - * Parameter [ArReleaseMinorVersion](#)
 - * Parameter [ArReleaseRevisionVersion](#)
 - * Parameter [ModuleId](#)
 - * Parameter [SwMajorVersion](#)
 - * Parameter [SwMinorVersion](#)
 - * Parameter [SwPatchVersion](#)
 - * Parameter [VendorApiInfix](#)
 - * Parameter [VendorId](#)
- Container [AutosarExt](#)
 - * Parameter [AdcTimeoutMethod](#)
 - * Parameter [AdcTimeoutVal](#)
 - * Parameter [AdcSarIpDevErrorDetect](#)
 - * Parameter [CtuIpDevErrorDetect](#)
 - * Parameter [AdcMulticoreSupport](#)
 - * Parameter [AdcEnableGroupDependentChannelNames](#)
 - * Parameter [AdcBypassAbortChainCheck](#)
 - * Parameter [AdcConvTimeOnce](#)
 - * Parameter [AdcOptimizeOneShotHwTriggerConversions](#)
 - * Parameter [AdcOptimizeDmaStreamingGroups](#)
 - * Parameter [AdcPreSamplingOnce](#)
 - * Parameter [AdcEnableInitialNotification](#)
 - * Parameter [AdcEnableDmaTransferMode](#)
 - * Parameter [AdcUseSoftwareInjectedGroups](#)
 - * Parameter [AdcUseHardwareNormalGroups](#)
 - * Parameter [AdcEnableUserModeSupport](#)
 - * Parameter [AdcSetHwUnitPowerModeApi](#)
 - * Parameter [AdcEnableChDisableChApi](#)
 - * Parameter [AdcGetInjectedConvStatusApi](#)
 - * Parameter [AdcEnableThresholdConfigurationApi](#)
 - * Parameter [AdcEnableCtuTrigAutosarExtApi](#)
 - * Parameter [AdcCtuHardwareTriggerOptimization](#)
 - * Parameter [AdcEnableCtuControlModeApi](#)
 - * Parameter [CtuEnableDmaTransferMode](#)

- * Parameter [AdcCtuControlModeExtraApis](#)
- * Parameter [AdcEnableWatchdogApi](#)
- * Parameter [AdcEnableSetChannel](#)
- * Parameter [AdcEnableDualClockMode](#)
- * Parameter [AdcEnableCalibration](#)
- * Parameter [AdcEnableSelfTest](#)
- * Parameter [AdcEnableReadRawDataApi](#)
- * Parameter [AdcEnableGroupStreamingResultReorder](#)

4.1 Module Adc

Configuration of the Adc (Analog Digital Conversion) module.

Included containers:

- [AdcConfigSet](#)
- [AdcGeneral](#)
- [AdcInterrupt](#)
- [AdcPublishedInformation](#)
- [CommonPublishedInformation](#)
- [AutosarExt](#)

Property	Value
type	ECUC-MODULE-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantSupport	true
supportedConfigVariants	VARIANT-POST-BUILD, VARIANT-PRE-COMPILE

4.2 Container AdcConfigSet

This container contains the configuration parameters and sub containers of the AUTOSAR Adc module.

Included subcontainers:

- [AdcHwUnit](#)
- [AdcHwTrigger](#)
- [CtuHwUnit](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.3 Container AdcHwUnit

This container contains the Driver configuration (parameters) depending on grouping of channels. This container could contain HW specific parameters which are not defined in the Standardized Module Definition. They must be added in the Vendor Specific Module Definition.

Included subcontainers:

- [AdcSelfTestThresholdConfiguration](#)
- [AdcNormalConvTimings](#)
- [AdcAlternateConvTimings](#)
- [AdcChannel](#)
- [AdcGroup](#)
- [AdcThresholdControl](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.4 Parameter AdcHwUnitId

Numeric ID of the HW Unit. This symbolic name allows accessing Hw Unit data. Enumeration literals are defined vendor specific.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC

Property	Value
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	ADC0
literals	['ADC0', 'ADC1']

4.5 Parameter AdcLogicalUnitId

Specifies the Logical id of the Hardware Unit.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	1
min	0

4.6 Parameter AdcTransferType

Select the Interrupt or Dma transfer Type.

If DMA is required for AdcTransferType, it is recommended to keep AdcWithoutInterrupts as false, otherwise, DMA will not be configured and it will be user responsibility to read the results from registers directly by calling Adc_ReadGroup.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF

Property	Value
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	ADC_INTERRUPT
literals	['ADC_INTERRUPT', 'ADC_DMA']

4.7 Parameter AdcClockSource

The ADC module specific clock input for the conversion unit can statically be configured to select different clock sources if provided by hardware. Enumeration literals are defined vendor specific.

This parameter is not used by the current implementation.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	BUS_CLOCK
literals	['BUS_CLOCK']

4.8 Parameter AdcPrescale

Optional ADC module specific clock prescale factor, if supported by hardware.

ImplementationType: Adc_PrescaleType.

The Prescaler value for NORMAL mode. Only the following are allowed:

1: ADC module clock frequency.

2: ADC module clock frequency / 2.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	2
min	1

4.9 Parameter AdcAltPrescale

The Prescaler value for ALTERNATE mode. This feature can be configured if Adc Set Clock Mode API from General/AutosarExt is enabled.

1: ADC module clock frequency.

2: ADC module clock frequency / 2.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	2
max	2
min	1

4.10 Parameter AdcCalibrationPrescale

Specifies the used clock input for calibration. This feature can be configured if Adc Enable Calibration API from General/AutosarExt is enabled.

1: ADC module clock frequency.

2: ADC module clock frequency / 2.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	2
max	2
min	1

4.11 Parameter AdcPowerDownDelay

The delay between the power down bit reset and the starting of conversion.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	15
max	255
min	0

4.12 Parameter AdcAltPowerDownDelay

The delay between the power down bit reset and the starting of conversion when ADC runs on low power system frequency.

This node is EDITABLE only if "Adc Set Clock Mode API" is enabled

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	15
max	255
min	0

4.13 Parameter AdcAutoClockOff

Enables/disables the auto-clock-off features.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.14 Parameter AdcBypassSampling

Select whether the presampling is followed by the comparison.

Disabled: The presampling will be followed by sampling the selected input.

Enabled: The presampling will be followed by the successive approximation algorithm and the conversion data will be written to the conversion data register of the selected input.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.15 Parameter AdcHwUnitOverwriteEn

Specifies whether a conversion data register accepts new data before the current conversion data has been read. When enabled, the new conversion result overwrites the older data, regardless of the validity of the older conversion data. When $CDRn[VALID] = 1$, the conversion data has not been read.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	true

4.16 Parameter AdcPresamplingInternalSignal0

Select the Adc presampling internal voltage for channels 0-31, AdcPresamplingInternalSignal0 shall take the following values:

DVDD0p8/2 - Pre-sample voltage - 1.

AVDD1p8/4 - Pre-sample voltage - 2.

VREFL_1p8 - Pre-sample voltage - 3.

VREFH_1p8 - Pre-sample voltage - 4.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	DVDD
literals	['DVDD', 'AVDD', 'VREFL', 'VREFH']

4.17 Parameter AdcPresamplingInternalSignal1

Select the Adc presampling internal voltage for channels 32-63, AdcPresamplingInternalSignal0 shall take the following values:

DVDD0p8/2 - Pre-sample voltage - 1.

AVDD1p8/4 - Pre-sample voltage - 2.

VREFL_1p8 - Pre-sample voltage - 3.

VREFH_1p8 - Pre-sample voltage - 4.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	DVDD
literals	['DVDD', 'AVDD', 'VREFL', 'VREFH']

4.18 Parameter AdcPresamplingInternalSignal2

Select the Adc presampling internal voltage for channels 64-95, AdcPresamplingInternalSignal0 shall take the following values:

DVDD0p8/2 - Pre-sample voltage - 1.

AVDD1p8/4 - Pre-sample voltage - 2.

VREFL_1p8 - Pre-sample voltage - 3.

VREFH_1p8 - Pre-sample voltage - 4.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	DVDD
literals	['DVDD', 'AVDD', 'VREFL', 'VREFH']

4.19 Parameter AdcHwUnitUsrOffset

The ADC can compensate a maximum of +/-15LSB at 12b offset by programming OFFSET_USER field in USROFSGN register.

The field is in 2's complement format and stored with an 8x multiplication. Thus, if a value of +/-X is stored in this field, it leads to

a final result adjusted with +/- $(X/8)$ from its unprogrammed version. It is required to keep the 3-LSBs zero to avoid any fractional

subtraction-related errors.

$ADC_OUTPUT_NEW = ADC_OUTPUT_OLD - (OFFSET_USER/8).$

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP

Property	Value
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	255
min	0

4.20 Parameter AdcHwUnitUsrGain

The ADC can compensate a maximum of ± 63 LSB at 12b gain by programming GAIN_USER field in USROFSGN register.

The field is in 2's complement format and stored with an 8x multiplication. Thus, if a value of $\pm Y$ is stored in this field, it leads to a

final result adjusted with $\pm (Y/8)$ LSB from its unprogrammed version. It is required to keep the 3-LSBs zero to avoid any fractional

subtraction/multiplication related errors.

$$\text{ADC_OUTPUT_NEW} = \text{ADC_OUTPUT_OLD} * (1 + (\text{GAIN_USER} / (8 * 4096))).$$

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	65535
min	0

4.21 Parameter AdcHwUnitDmaClearSource

Select the DMA request clearing source.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	DMA_REQ_CLEAR_ON_ACK
literals	['DMA_REQ_CLEAR_ON_ACK', 'DMA_REQ_CLEAR_ON_READ']

4.22 Reference AdcDmaChannelId

ID of the DMA channel used to transfer the data.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/Mcl/MclConfig/dmaLogicChannel_Type

4.23 Reference AdcCountingDmaChannelId

Configurable only when Transfer Type is DMA and at least one group of hw unit has:

AdcEnableOptimizeDmaStreamingGroups enabled with more than one ADC channel

OR has selected Without Interrupts, ACCESS_MODE_STREAMING and Group Streaming Results Reorder and number of channels > 1

OR has selected Without Interrupts, ACCESS_MODE_STREAMING and a single channel - for this case, AutosarExt Adc Enable Group Streaming Results Reorder feature must be enabled, but can be disabled at group level.

Linked to DMA channel by ADC driver, to count the number of samples converted in streaming mode

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/Mcl/MclConfig/dmaLogicChannel_Type

4.24 Reference AdcHwUnitEcucPartitionRef

Maps a ADC hardware unit to zero or one ECUC partition to limit the access to this hardware unit. The ECUC partitions referenced are a subset of the ECUC partitions where the ADC driver is mapped to.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/EcuC/EcucPartitionCollection/EcucPartition

4.25 Container AdcSelfTestThresholdConfiguration

These values define the limits for results measured at each step of the self-test algorithms. If measured values are outside these ranges, the self test fails.

When selecting the values, please take into account ADC resolution and reference voltage used.

For more details please refer to reference manual.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.26 Parameter AdcSTAW0RSelfTestHighThresholdValue

SelfTest high threshold value for Algorithm S0.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	2912
max	65535
min	0

4.27 Parameter AdcSTAW0RSelfTestLowThresholdValue

SelfTest low threshold value for Algorithm S0.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	2367
max	65535
min	0

4.28 Parameter AdcSTAW1ARSelfTestHighThresholdValue

SelfTest high threshold value for Algorithm S1 - integer part.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	65535
min	0

4.29 Parameter AdcSTAW1ARSelfTestLowThresholdValue

SelfTest low threshold value for Algorithm S1 - integer part.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	65535
min	0

4.30 Parameter AdcSTAW1BRSelfTestHighThresholdValue

SelfTest high threshold value for Algorithm S1 - fractional part.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	2867
max	65535
min	0

4.31 Parameter AdcSTAW1BRSelfTestLowThresholdValue

SelfTest low threshold value for Algorithm S1 - fractional part.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP

Property	Value
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1414
max	65535
min	0

4.32 Parameter AdcSTAW2RSelfTestLowThresholdValue

SelfTest low threshold value for Algorithm S2.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	4085
max	65535
min	0

4.33 Parameter AdcSTAW4RSelfTestHighThresholdValue

SelfTest high threshold value for Algorithm C0. Low threshold value is two's-complement of high threshold value and automatically calculated.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	100
max	65535
min	0

4.34 Parameter AdcSTAW5RSelfTestHighThresholdValue

SelfTest high threshold value for Algorithm C1 to C11. Low threshold value is two's-complement of high threshold value and automatically calculated

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	100
max	65535
min	0

4.35 Container AdcNormalConvTimings

Selects Normal values used for programming CTR Conversion Timing Registers in Adc_SetClockMode API and also when AdcConvTimeOnce option is enabled.

This node is EDITABLE only if "Adc Conversion Time Once" is enabled

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.36 Parameter AdcSamplingDurationNormal0

Select the Normal Sampling Duration for channels 0-31 when calling `Adc_SetClockMode(ADC_NORMAL)` for CTR0 register.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	20
max	255
min	8

4.37 Parameter AdcSamplingDurationNormal1

Select the Normal Sampling Duration for channels 32-63 when calling `Adc_SetClockMode(ADC_NORMAL)` for CTR1 register.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Property	Value
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	20
max	255
min	8

4.38 Container AdcAlternateConvTimings

Selects Alternate values used in Adc_SetClockMode API for programming CTR Conversion Timing Registers.

This container is EDITABLE only if "Adc Conversion Time Once" and "Adc Set Clock Mode API" are enabled. Hardware averaging functionality is also available for configuration if supported.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.39 Parameter AdcSamplingDurationAlt0

Select the Alternate Sampling Duration when calling Adc_SetClockMode(ADC_NORMAL) for CTR0 register.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true

Property	Value
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	20
max	255
min	8

4.40 Parameter AdcSamplingDurationAlt1

Select the Alternate Sampling Duration when calling `Adc_SetClockMode(ADC_NORMAL)` for CTR1 register.

Property	Value
type	ECUC-INTEGGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	20
max	255
min	8

4.41 Container AdcChannel

This container contains the channel configuration (parameters) depending on the hardware capability.

The organization of this data structure could contain dependencies to the microcontroller so this is left up to the implementer and its location is left up to the configuration.

Note: Since a `AdcChannel` can be part of several `AdcGroups`, this container is not realized as a subcontainer of `AdcGroup` but instead as a subcontainer of `AdcHwUnit`.

Included subcontainers:

- None

Property	Value
----------	-------

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.42 Parameter AdcLogicalChannelId

This is the logical Id of the ADC channel.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	1024
min	0

4.43 Parameter AdcChannelName

This parameter defines the assignment of the channel to the physical ADC hardware channel. Note: - Range of the ADC Channels depends on the selected package.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	ADC_CH_00_ChanNum0
literals	['ADC_CH_00_ChanNum0', 'ADC_CH_01_ChanNum1', 'ADC_CH_02_ChanNum2', 'ADC_CH_03_ChanNum3', 'ADC_CH_08_ChanNum0', 'ADC_CH_09_ChanNum1', 'ADC_CH_10_ChanNum2', 'ADC_CH_11_ChanNum3', 'ADC_CH_04_ChanNum4', 'ADC_CH_05_ChanNum5', 'ADC_CH_06_ChanNum6', 'ADC_CH_07_ChanNum7', 'BANDGAP_ChanNum32', 'DVDD_DIV2_ChanNum35', 'AVDD_ChanNum36', 'VREFL_ChanNum37', 'VREFH_ChanNum38']

4.44 Parameter AdcChannelId

This parameter defines the assignment of the channel to the physical ADC hardware channel. Note: - Range of the ADC Channels depends on the selected package.

IMPORTANT NOTE: This node must be in sync with 'Adc Physical Channel Name': must be equal with number specified after 'ChanNum' in the selected 'Adc Physical Channel Name'. E.g. ADC_CH_0_ChanNum38 => Channel ID must be 38. The node is required by Autosar standard. 'Adc Physical Channel Name' node is added because refers to names from ReferenceManual and is more user friendly.

In Tresos configurator, after selecting a new Channel Name, the new Channel Id value can be filled in automatically by using 'Calculate Value' button.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	1024
min	0

4.45 Parameter AdcChannelConvTime

Configuration of conversion time, i.e. the time during which the analogue value is converted into digital representation, (in clock cycles) for each channel, if supported by hardware.

ImplementationType: Adc_ConversionTimeType.

This parameter is not used by the current implementation.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	9223372036854775807
min	0

4.46 Parameter AdcChannelLimitCheck

Enables or disables limit checking for an ADC channel.

This node is EDITABLE only if "Adc Enable Limit Check" is enabled

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.47 Parameter AdcChannelHighLimit

High limit - used for limit checking. This value depends on the Adc Hw Unit resolution.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	4095
max	9223372036854775807
min	0

4.48 Parameter AdcChannelLowLimit

Low limit - used for limit checking.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	9223372036854775807
min	0

4.49 Parameter AdcChannelRangeSelect

In case of active limit checking: defines which conversion values are taken into account related to the borders defined with AdcChannelLowLimit and AdcChannelHighLimit.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	ADC_RANGE_ALWAYS
literals	['ADC_RANGE_ALWAYS', 'ADC_RANGE_BETWEEN', 'ADC_RANGE↵_NOT_BETWEEN', 'ADC_RANGE_NOT_OVER_HIGH', 'ADC_RANG↵E_NOT_UNDER_LOW', 'ADC_RANGE_OVER_HIGH', 'ADC_RANGE↵_UNDER_LOW']

4.50 Parameter AdcChannelRefVoltsrcHigh

Upper reference voltage source for each channel. Enumeration literals are defined vendor specific.

This parameter is not used by the current implementation.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	UPPER_REF_VOLT_0
literals	['UPPER_REF_VOLT_0']

4.51 Parameter AdcChannelRefVoltsrcLow

Lower reference voltage source for each channel. Enumeration literals are defined vendor specific.

This parameter is not used by the current implementation.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	LOWER_REF_VOLT_0
literals	['LOWER_REF_VOLT_0']

4.52 Parameter AdcChannelResolution

Channel Resolution in bits of converted value.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	12
max	63
min	1

4.53 Parameter AdcChannelSampTime

Sampling time, i.e. the time during which the value is sampled, (in clock cycles) for each channel. This parameter is not used by the current implementation.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	8
max	9223372036854775807
min	0

4.54 Parameter AdcEnablePresampling

When true, this parameter enables the presampling phase for the selected channel.

The normal operation sequence on the channel: Presampling -> Sampling -> Conversion.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.55 Parameter AdcEnableThresholds

When true, this parameter enables the threshold detection feature for the selected channel.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.56 Parameter AdcWdogNotification

This function pointer is called whenever the conversion of the channel issued a watchdog interrupt.

The notification takes one uint8 parameter which represents the flags(low/high threshold) that triggered the interrupt.

In order to interpret them please use ADC_WDG_... defines.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	NULL_PTR

4.57 Reference AdcThresholdRegister

Select the threshold register which provides the values to be used for upper and lower thresholds.

ADCHwUnits support threshold registers from ADC_THRESHOLD_REG_0 to ADC_THRESHOLD_REG_7.

Note: This is an Implementation Specific Parameter.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D11M40I2R0/Adc/AdcConfigSet/AdcHwUnit/AdcThresholdControl

4.58 Container AdcGroup

This container contains the Group configuration (parameters).

Included subcontainers:

- [AdcGroupConversionConfiguration](#)
- [AdcAlternateGroupConvTimings](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.59 Parameter AdcGroupAccessMode

Type of access mode to group conversion results.

ImplementationType: Adc_GroupAccessModeType.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	ADC_ACCESS_MODE_SINGLE
literals	['ADC_ACCESS_MODE_SINGLE', 'ADC_ACCESS_MODE_STREAMING']

4.60 Parameter AdcGroupConversionMode

Type of conversion mode supported by the driver.

ImplementationType: Adc_GroupConvModeType.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	ADC_CONV_MODE_ONESHOT
literals	['ADC_CONV_MODE_ONESHOT', 'ADC_CONV_MODE_CONTINUOUS']

4.61 Parameter AdcGroupConversionType

Normal or Injected conversion type.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF

Property	Value
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	ADC_CONV_TYPE_NORMAL
literals	['ADC_CONV_TYPE_NORMAL', 'ADC_CONV_TYPE_INJECTED']

4.62 Parameter AdcGroupId

Numeric ID of the group. This parameter is the symbolic name to be used on the API. This symbolic name allows accessing Channel Group data. This value will be assigned to the symbolic name derived of the AdcGroup container shortName.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	true
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	1023
min	0

4.63 Parameter AdcGroupPriority

Priority level of the AdcGroup. This item is ignored if Adc/AdcGeneral/AdcPriorityImplementation is defined to ADC_PRIORITY_NONE.

Property	Value
type	ECUC-INTEGER-PARAM-DEF

Property	Value
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	255
min	0

4.64 Parameter AdcGroupReplacement

Replacement mechanism, which is used on ADC group level, if a group conversion is interrupted by a group which has a higher priority.

ImplementationType: Adc_GroupReplacementType

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	ADC_GROUP_REPL_ABORT_RESTART
literals	['ADC_GROUP_REPL_ABORT_RESTART', 'ADC_GROUP_REPL_SU← SPEND_RESUME']

4.65 Parameter AdcGroupTriggSrc

Type of source event that starts a group conversion. It's possible select Hw or Sw trigger.

In case of Hw trigger the trigger source can be from the CTU or External hardware pins of the controller.

In this controller only Ctu trigger source is supported which is selected by the "Adc Group Hardware Trigger Source" parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	ADC_TRIGG_SRC_SW
literals	['ADC_TRIGG_SRC_HW', 'ADC_TRIGG_SRC_SW']

4.66 Parameter AdcHwTrigSignal

Configures the edge of the hardware trigger signal, i.e. to start the conversion.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	ADC_HW_TRIG_RISING_EDGE
literals	['ADC_HW_TRIG_BOTH_EDGES', 'ADC_HW_TRIG_RISING_EDGE', 'ADC_HW_TRIG_FALLING_EDGE']

4.67 Parameter AdcHwTrigTimer

Reload value of the ADC module embedded timer.

Note: This value is used to write to CTU Counter Compare Value.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	10
max	65535
min	0

4.68 Parameter AdcNotification

Callback function for each group. This function pointer is called whenever the conversion of this group is completed.

This node is EDITABLE only if "Adc Notification Capability" is enabled

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	NULL_PTR

4.69 Parameter AdcExtraNotification

Extra callback function for each group. This function pointer will be called at the beginning of the interrupt routine, before updating any HW registers or Group status.

This feature is enabled if "Adc Notification Capability" from General/Adc General Configuration is enabled.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	NULL_PTR

4.70 Parameter AdcStreamingBufferMode

Select the streaming buffer as linear buffer (i.e. the ADC Driver stops the conversion as soon as the stream buffer is full) or as circular buffer (wraps around if the end of the stream buffer is reached).

This feature is enabled only if "Adc Group Access Mode" is set to ADC_ACCESS_MODE_STREAMING.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	ADC_STREAM_BUFFER_LINEAR
literals	['ADC_STREAM_BUFFER_LINEAR', 'ADC_STREAM_BUFFER_CIRCULAR']

4.71 Parameter AdcEnableOptimizeDmaStreamingGroups

Enable/Disable The Adc driver enable Optimize DMA streaming groups for reducing the number of interrupts required for processing the conversions of Adc Groups that consist of one or more channels (depending on HW capabilities) and which are configured as ADC_ACCESS_MODE_STREAMING.

When this feature is enabled, only one interrupt will be raised after the completion of all stream conversions (as configured by `AdcStreamingNumSamples` parameter). An additional interrupt to be raised after half of the stream is converted shall also be configurable.

This feature is enabled if "Adc Global Enable DMA Transfer" from General/AutosarExt is also enabled.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.72 Parameter `AdcEnableHalfInterrupt`

Enable/ Disable the interrupt when half sample complete for optimize DMA streaming groups feature.

The "Adc Optimize DMA Streaming Groups" must be enabled for configuring this feature.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.73 Parameter `AdcStreamingNumSamples`

Number of ADC values to be acquired per channel in streaming access mode.

Note: in single access mode this parameter assumes value 1, since only one sample per channel is processed.

ImplementationType: Adc_StreamNumSampleType.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	255
min	1

4.74 Parameter AdcStreamResultGroup

Arrange the ADC results as multiple sets of group result buffer.

E.g: for a group with channels {CH1 CH5 CH7} the resulting stream buffer shall be:

{ CH1, CH5, CH7, CH1, CH5, CH7, CH1, CH5, CH7}

instead of

{ CH1, CH1, CH1, CH5, CH5, CH5, CH7, CH7, CH7} like supported by AUTOSAR standard.

This Parameter can be configured only for groups configured with ADC_ACCESS_MODE_STREAMING Access Mode

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.75 Parameter AdcEnableChDisableChGroup

If this parameter is enabled, it allows the feature of enabling or disabling a particular channel in the group.

Max.no of Groups with this feature enabled, should be configured are 254 if the configuration parameter AdcEnableChDisableChApi is enabled in Autosar Extension container.

Note: This is an Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.76 Parameter AdcWithoutInterrupts

Enable/ Disable the occurring of ADC Interrupts and Reading of the group conversion results periodically without interrupts (ADC interrupt or DMA notification).

1. When this parameter is enabled, interrupts are disabled. The conversion will run without software intervention (no interrupt generated anymore) and the application can read the results by calling Adc_ReadGroup().
2. If Transfer Type is ADC_INTERRUPT (or ADC_DMA) and this one is enabled, the result buffer registered via Adc_SetupResultBuffer will no longer be used populated with results. Adc_ReadGroup will read the results directly from ADC HW registers (or Internal Dma Buffer).

When this parameter is Disabled, normal functionality shall be executed.

Note: This is an Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A

Property	Value
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.77 Parameter AdcWithoutDma

When true, disables completely DMA configuration done by ADC driver for the group, to not affect other groups for which the ADC driver has already configured the DMA. It is intended to be used when DMA Transfer Mode is selected for the ADC unit, for groups required to work without interrupt and without DMA.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.78 Parameter AdcExtDMAChanEnable

Enable/ Disable DMA functionality of individual group.

In this mode the adc will only enable the dma request for the last channel in the group and, so that the dma transfer will start after the group conversion has finished.

In this mode it is the users responsibility to configure the dma to transfer the data according to his needs.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.79 Reference AdcGroupHwTriggerSource

Select the HW trigger signal for triggering the conversion group. Configurable only when HW trigger source is selected for the group.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D11M40I2R0/Adc/AdcConfigSet/AdcHwTrigger

4.80 Reference AdcGroupDefinition

Assignment of channels to a AdcGroups. For each AdcChannel that should belong to the group, a reference needs to be defined.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	1
upperMultiplicity	Infinite
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD

Property	Value
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcuDefs/Adc/AdcConfigSet/AdcHwUnit/AdcChannel

4.81 Reference AdcGroupEcucPartitionRef

Maps an ADC channel group to zero or multiple ECUC partitions to limit the access to this channel group. The ECUC partitions referenced are a subset of the ECUC partitions where the ADC driver is mapped to.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcuDefs/EcuC/EcucPartitionCollection/EcucPartition

4.82 Container AdcGroupConversionConfiguration

Configure the Sampling and Conversion TimeGroup.

Note: This is an Implementation Specific Parameter

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.83 Parameter AdcSamplingDuration0

Select the Sampling Duration for channels 0-31 from CTR0 register

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	20
max	255
min	8

4.84 Parameter AdcSamplingDuration1

Select the Sampling Duration for channels 32-63 from CTR1 register

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	20
max	255
min	8

4.85 Container AdcAlternateGroupConvTimings

Selects Alternate values used in Adc_SetClockMode API for programming CTR Conversion Timing Registers.

This container is EDITABLE only if "Adc Conversion Time Once" is disabled and "Adc Set Clock Mode API" is enabled. Hardware averaging functionality is also available for configuration if supported.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.86 Parameter AdcAltGroupSamplingDuration0

Select the Alternate Sampling Duration for channels 0-31 from CTR0 register

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	20
max	255
min	8

4.87 Parameter AdcAltGroupSamplingDuration1

Select the Alternate Sampling Duration for channels 32-63 from CTR1 register

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP

Property	Value
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	20
max	255
min	8

4.88 Container AdcThresholdControl

Configure threshold detection feature for the selected channel.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	8
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.89 Parameter AdcThresholdControlRegister

Select the threshold register which provides the values to be used for upper and lower thresholds.

ADCHwUnits support threshold registers from ADC_THRESHOLD_REG_0 to ADC_THRESHOLD_REG_7.

Note: This is an Implementation Specific Parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	ADC_THRESHOLD_REG_0
literals	['ADC_THRESHOLD_REG_0', 'ADC_THRESHOLD_REG_1', 'ADC_THRESHOLD_REG_2', 'ADC_THRESHOLD_REG_3', 'ADC_THRESHOLD_REG_4', 'ADC_THRESHOLD_REG_5', 'ADC_THRESHOLD_REG_6', 'ADC_THRESHOLD_REG_7']

4.90 Parameter AdcHighThreshold

Set the value for High Threshold. This value depends on the Adc Hw Unit resolution.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	4095
max	4095
min	0

4.91 Parameter AdcLowThreshold

Set the value for Low Threshold.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP

Property	Value
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	4095
min	0

4.92 Container AdcHwTrigger

This container contains the Hardware trigger source configured for the group. Editable only if at least one group has HW trigger source selected.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.93 Parameter AdcHwTrigSrc

On this implementation the HW triggers available are from Ctu.

Note: This is an implementation specific parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	CTU_PWM_REL_FTM0_CH0
literals	['CTU_PWM_REL_FTM0_CH0', 'CTU_PWM_ODD_0_FTM0_CH1', 'CTU_PWM_ODD_1_FTM0_CH2', 'CTU_PWM_ODD_2_FTM1_CH0', 'CTU_PWM_ODD_3_FTM1_CH1', 'CTU_PWM_EVEN_0_FTM1_CH2', 'EXT_TRIG']

4.94 Container CtuHwUnit

This container contains configuration of the Ctu unit. This node is EDITABLE only if "Adc CTU Control Mode API" is enabled.

Included subcontainers:

- [ExpectedConvDurationConfig](#)
- [CtuInputTrigConfigs](#)
- [CtuTriggerCfg](#)
- [CtuAdcCommandList](#)
- [CtuResultFifos](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.95 Parameter CtuHwUnitId

Specifies the used CTU Hardware Unit.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	0
min	0

4.96 Parameter CtuLogicalUnitId

Specifies the Logical id of the Ctu Hardware Unit.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	0
min	0

4.97 Parameter CtuTGSMMode

Select mode for Trigger Generator Subunit - double-buffered (TGSCR.TGS_M).

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF

Property	Value
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	TGS_MODE_TRIGGERED
literals	['TGS_MODE_TRIGGERED', 'TGS_MODE_SEQUENTIAL']

4.98 Parameter CtuInputClockPrescaler

Input clock prescaler - double-buffered (TGSCR.PRES).

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	PRESCALER_1
literals	['PRESCALER_1', 'PRESCALER_2', 'PRESCALER_3', 'PRESCALER_4']

4.99 Parameter CtuExtTrigMode

Trigger mode for external triggers: eTimers and EXT_TRG - double-buffered (TGSCR.ET_TM).

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	EXT_TRIG_MODE_PULSE
literals	['EXT_TRIG_MODE_PULSE', 'EXT_TRIG_MODE_TOGGLE']

4.100 Parameter CtuTgsCounterCompareVal

TGS Counter Compare Value - double-buffered (TGSCCR).

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	256
max	65535
min	0

4.101 Parameter CtuTgsCounterReloadVal

TGS Counter Reload Value - double-buffered (TGSCRR).

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A

Property	Value
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	65535
min	0

4.102 Parameter CtuAdcCmdListMode

Select mode of execution for ADC Command List (LISTCSR.PAR_LIST).

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	ADC_CMD_LIST_MODE_STREAMING
literals	['ADC_CMD_LIST_MODE_STREAMING', 'ADC_CMD_LIST_MODE_↵ PARALLEL']

4.103 Parameter CtuSeqModeMrsInput

Select which input to be used for MRS when Sequential Mode is enabled (see CtuTGSMODE) - double-buffered (TGSCR.MRS_SM)

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A

Property	Value
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	PWM_REL_FTM0_CH0
literals	['PWM_REL_FTM0_CH0', 'PWM_ODD_0_FTM0_CH1', 'PWM_ODD_1_FTM0_CH2', 'PWM_ODD_2_FTM1_CH0', 'PWM_ODD_3_FTM1_CH1', 'PWM_EVEN_0_FTM1_CH2']

4.104 Parameter CtuSeqModeMrsInputEdge

Select which edge of the input signal (only rising or falling is allowed) to be used for MRS when SequenceMode is enabled - double-buffered (TGSCR.MRS_SM)

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	EDGE_BOTH
literals	['EDGE_RISING', 'EDGE_FALLING', 'EDGE_BOTH']

4.105 Parameter CtuDmaDoneGRE

Enable setting of General Reload Enable (GRE) when DMA-done occurs (IR.DMA_DE).

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A

Property	Value
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.106 Parameter CtuDmaReqMRS

Enable issuing of DMA request when MRS occurs, if GRE is enabled (IR.MRS_DMAE).

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.107 Parameter CtuFifoDmaRawData

Enable Dma Transfer Raw Data of Fifo Result Register (includes ADC Port, Channel Number and Data Conversion)

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.108 Parameter CtuDisableOutput

True/false - disable/enable the CTU outputs: pulses to other peripherals/pins, ADC commands or command streams.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.109 Parameter CtuErrorNotif

Error callback with triggered mask error(uint32) as parameter, can check with CTU_IP_ERROR_FLAG defines.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	NULL_PTR

4.110 Parameter CtuMrsNotif

Callback for MRS.

Property	Value
type	ECUC-FUNCTION-NAME-DEF

Property	Value
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	NULL_PTR

4.111 Parameter CtuAdcCmdIssueNotif

Notification called when issuing a command to the Adc unit.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	NULL_PTR

4.112 Parameter CtuDigitalFilter

Digital filter for external input signal (EXT_IN).

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A

Property	Value
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	255
min	0

4.113 Parameter CtuControlOnTime

Value of Control ON-Time and Guard Time for external trigger (COTR).

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	255
min	0

4.114 Container ExpectedConvDurationConfig

This container is used for configuring the expected conversion duration for each Adc Unit.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.115 Parameter CtuExpectedValuePortA

Expected number of system clock cycles (the slave bus clock) for an AD conversion to be completed by the ADC connected to Port A.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	65535
max	65535
min	0

4.116 Parameter CtuExpectedNotifPortA

This function pointer is called whenever a conversion takes longer than the expected duration to complete by the ADC connected to Port A.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	NULL_PTR

4.117 Parameter CtuExpectedValuePortB

Expected number of system clock cycles (the slave bus clock) for an AD conversion to be completed by the ADC connected to Port B.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	65535
max	65535
min	0

4.118 Parameter CtuExpectedNotifPortB

This function pointer is called whenever a conversion takes longer than the expected duration to complete by the ADC connected to Port B.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	NULL_PTR

4.119 Parameter CtuConvDurationCounterRange

This value is used to mask out conversion duration values. 1 bits in this value will mask out the corresponding bits in the expected conversion duration registers. These bits must be set from left to right.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP

Property	Value
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	65535
max	65535
min	0

4.120 Container CtuInputTrigConfigs

This container contains the CTU Input Trigger configuration parameters.

Select input triggers: even/odd bits correspond to rising/falling edge. Double-buffered (TGSISR)

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	8
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.121 Parameter CtuInputTrigEdge

Selects the edge of activation of the input signal.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	EDGE_BOTH
literals	['EDGE_RISING', 'EDGE_FALLING', 'EDGE_BOTH']

4.122 Reference CtuInputTrigSelect

This parameter selects the input triggers for Trigger Generator Subunit (TGS).

Input 0 - PWM Rel

Input 1 - PWM ODD 0

Input 2 - PWM ODD 1

Input 3 - PWM ODD 2

Input 4 - PWM ODD 3

Input 5 - PWM Even 0

Input 6 - PWM Even 1

Input 15 - External Pin

Other Inputs are reserved

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D11M40I2R0/Adc/AdcConfigSet/AdcHwTrigger

4.123 Container CtuTriggerCfg

This container contains the Ctu configuration that is used for configuring the properties of an internal trigger (TGS output trigger, input for SU).

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	8
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.124 Parameter CtuTriggerIndex

This parameter defines the assignment of the Ctu trigger to the physical CTU hardware triggers

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	7
min	0

4.125 Parameter CtuCompareVal

Compare Value - double-buffered (TnCR).

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	0
max	65535
min	0

4.126 Parameter CtuCmdListStartAdr

First address of the command list associated with the internal trigger - (CLCRx). Available when ADCE is enabled.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	0
max	23
min	0

4.127 Parameter CtuOutputTrigEnMask

Select the CTU output triggers enabled for each SU input trigger. To be used with CTU_IP_OUTPUT_TRIG_n defines. Double-buffered (THCR).

Trigger Handler Control Register:

Disable - Bit Mask b0

Tn_E - Bit Mask b1000000

Tn_ETE - Reserved for this platform

Tn_T4E - Reserved for this platform

Tn_T3E - Reserved for this platform

Tn_T2E - Reserved for this platform

Tn_T1E - Reserved for this platform

Tn_ADCE - Bit Mask b0000001

Tn_ALL - Bit Mask b1000001

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	65
min	0

4.128 Parameter CtuTriggerNotif

Notification called when triggered.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD

Property	Value
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	NULL_PTR

4.129 Container CtuAdcCommandList

This container contains the Ctu Adc Command List configuration parameters.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	24
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.130 Parameter CtuIntEn

Enable/Disable interrupt request for command execution (CIR).

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.131 Parameter CtuFifoIdx

Select result FIFO to store the conversion result.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	3
min	0

4.132 Parameter CtuConvMode

Select conversion mode.

Single conversion mode - conversion will run on a single ADC Unit

Dual conversion mode - conversion will run on two ADC units simultaneously

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	CONV_MODE_SINGLE
literals	['CONV_MODE_SINGLE', 'CONV_MODE_DUAL']

4.133 Parameter CtuLastCmd

Mark this as the last command in the list by setting the last command(LC) bit of the next command to 1.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	LAST
literals	['LAST', 'NOT_LAST']

4.134 Parameter CtuAdcPort

Select ADC Unit to run the conversion.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	ADC_0
literals	['ADC_0', 'ADC_1']

4.135 Parameter CtuAdcChanaA

Select target channel number for ADC Unit 0 (ADC port A).

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	ADC_CH_00_ChanNum0
literals	['ADC_CH_00_ChanNum0', 'ADC_CH_01_ChanNum1', 'ADC_CH_02_↵ ChanNum2', 'ADC_CH_03_ChanNum3', 'ADC_CH_04_ChanNum4', 'AD↵ C_CH_05_ChanNum5', 'ADC_CH_06_ChanNum6', 'ADC_CH_07_Chan↵ Num7', 'BANDGAP_ChanNum32', 'DVDD_DIV2_ChanNum35', 'AVDD_↵ ChanNum36', 'VREFL_ChanNum37', 'VREFH_ChanNum38']

4.136 Parameter CtuAdcChanB

Select target channel number for ADC Unit 1 (ADC port B).

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	ADC_CH_08_ChanNum0
literals	['ADC_CH_08_ChanNum0', 'ADC_CH_09_ChanNum1', 'ADC_CH_10_↵ ChanNum2', 'ADC_CH_11_ChanNum3', 'ADC_CH_04_ChanNum4', 'AD↵ C_CH_05_ChanNum5', 'ADC_CH_06_ChanNum6', 'ADC_CH_07_Chan↵ Num7', 'BANDGAP_ChanNum32', 'DVDD_DIV2_ChanNum35', 'AVDD_↵ ChanNum36', 'VREFL_ChanNum37', 'VREFH_ChanNum38']

4.137 Container CtuResultFifos

This container contains the Ctu Fifos configuration parameters.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	4
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.138 Parameter CtuFifoIndex

The FIFO index of CTU.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	3
min	0

4.139 Parameter CtuFifoThreshold

Threshold value of the fifo. FIFOs generate an interrupt when the number of elements in the FIFO exceeds the values present in the threshold register.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	8
max	8
min	0

4.140 Parameter CtuFifoDmaEn

Enable DMA requests to be issued when the fifoThreshold is reached - (FDCR.DEn).

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.141 Parameter CtuFifoDmaBuffer

Pointer to the Data Buffer (destination for conversion results) when CtuFifoDmaEn is enabled.

Property	Value
type	ECUC-LINKER-SYMBOL-DEF
origin	NXP
symbolicNameValue	false

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	CtuDmaFifo0

4.142 Parameter CtuFifoThresholdNotif

This function pointer is called when the number of elements in Fifo is greater than threshold value.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	NULL_PTR

4.143 Parameter CtuFifoUnderrunNotif

This function is called when the Fifo becomes empty.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true

Property	Value
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	NULL_PTR

4.144 Parameter CtuFifoOverrunNotif

This function is called when the Fifo is full and a write is attempted.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	NULL_PTR

4.145 Parameter CtuFifoFullNotif

This function is called when the Fifo becomes full.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	NULL_PTR

4.146 Reference CtuFifoDmaChannelId

Select Logical Dma Channel that will be used to transfer the Fifo Buffer from FIFO Result Data Register to the User Fifo Buffer.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/Mcl/MclConfig/dmaLogicChannel_Type

4.147 Container AdcGeneral

General configuration (parameters) of the ADC Driver software module.

Included subcontainers:

- [AdcPowerStateConfig](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.148 Parameter AdcDeInitApi

Adds/removes the service Adc_DeInit() from the code.

true: Adc_DeInit() can be used.

false: Adc_DeInit() can not be used.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	true

4.149 Parameter AdcDevErrorDetect

Switches the development error detection and notification on or off.

true: detection and notification is enabled.

false: detection and notification is disabled.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.150 Parameter AdcEnableLimitCheck

Enables or disables limit checking feature in the ADC driver.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC

Property	Value
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.151 Parameter AdcEnableQueuing

Determines, if the queuing mechanism is active in case of priority mechanism disabled.

Note: If priority mechanism is enabled, queuing mechanism is always active and the parameter ADC_ENABLE_QUEUING is not evaluated.

true: Enabled.

false: Disabled.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.152 Parameter AdcPriorityQueueMaxDepth

Maximum depth of queue used for queuing of incoming conversion requests when hardware unit is busy.

Property	Value
type	ECUC-INTEGER-PARAM-DEF

Property	Value
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	1024
min	1

4.153 Parameter AdcEnableStartStopGroupApi

Adds / removes the services `Adc_StartGroupConversion()` and `Adc_StopGroupConversion()` from the code.

true: `Adc_StartGroupConversion()` and `Adc_StopGroupConversion()` can be used.

false: `Adc_StartGroupConversion()` and `Adc_StopGroupConversion()` can not be used.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.154 Parameter AdcGrpNotifCapability

Determines, if the group notification mechanism (the functions to enable and disable the notifications) is available at runtime.

true: Enabled.

false: Disabled.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	true

4.155 Parameter AdcHwTriggerApi

Adds / removes the services `Adc_EnableHardwareTrigger()` and `Adc_DisableHardwareTrigger()` from the code.

true: `Adc_EnableHardwareTrigger()` and `Adc_DisableHardwareTrigger()` can be used.

false: `Adc_EnableHardwareTrigger()` and `Adc_DisableHardwareTrigger()` can not be used.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.156 Parameter AdcPriorityImplementation

Determines whether a priority mechanism is available for prioritization of the conversion requests and if available, the type of prioritization mechanism. The selection applies for groups with trigger source software and trigger source hardware. Two types of prioritization mechanism can be selected.

The hardware prioritization mechanism (`AdcPriorityHw`) uses the ADC hardware features for prioritization of the software conversion requests and hardware trigger signals for groups with trigger source hardware.

Tresos Configuration Plug-in

The mixed hardware and software prioritization mechanism (AdcPriorityHwSw) uses the ADC hardware features for prioritization of ADC hardware trigger for groups with trigger source hardware and a software implemented prioritization mechanism for groups with trigger source software.

The group priorities for software triggered groups are typically configured with lower priority levels than the group priorities for hardware triggered groups.

ImplementationType: Adc_PriorityImplementationType.

Note: In this version the ADC_PRIORITY_HW isn't used.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	ADC_PRIORITY_NONE
literals	['ADC_PRIORITY_HW', 'ADC_PRIORITY_HW_SW', 'ADC_PRIORITY_↵_NONE']

4.157 Parameter AdcReadGroupApi

Adds / removes the service Adc_ReadGroup() from the code.

true: Adc_ReadGroup() can be used.

false: Adc_ReadGroup() can not be used.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.158 Parameter AdcResultAlignment

Alignment of ADC raw results in ADC result buffer (left/right alignment).

Implementation Type: `Adc_ResultAlignmentType`.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	ADC_ALIGN_RIGHT
literals	['ADC_ALIGN_RIGHT', 'ADC_ALIGN_LEFT']

4.159 Parameter AdcVersionInfoApi

Adds / removes the service `Adc_GetVersionInfo()` from the code.

true: `Adc_GetVersionInfo()` can be used.

false: `Adc_GetVersionInfo()` can not be used.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.160 Parameter AdcLowPowerStatesSupport

Adds / removes all power state management related APIs (ADC_SetPowerState, ADC_GetCurrentPowerState, ADC_GetTargetPowerState, ADC_PreparePowerState, ADC_Main_PowerTransitionManager), indicating if the HW offers low power state management.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.161 Parameter AdcPowerStateAsynchTransitionMode

Enables / disables support of the ADC Driver to the asynchronous power state transition. This feature is not implemented on this platform.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.162 Reference AdcEcucPartitionRef

Maps the ADC driver to zero or multiple ECUC partitions to make the driver API available in the according partition.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcuDefs/EcuC/EcuPartitionCollection/EcuPartition

4.163 Reference AdcKernelEcucPartitionRef

Maps the ADC kernel to zero or one ECUC partition to assign the driver kernel to a certain core. The ECUC partition referenced is a subset of the ECUC partitions where the ADC driver is mapped to.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcuDefs/EcuC/EcuPartitionCollection/EcuPartition

4.164 Container AdcPowerStateConfig

Each instance of this parameter defines a power state and the callback to be called when this power state is reached.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.165 Parameter AdcPowerState

Each instance of this parameter describes a different power state

supported by the ADC HW. It should be defined by the HW supplier and

used by the ADCDriver to reference specific HW configurations which set

the ADC HW module in the referenced power state. At least the power mode

corresponding to full power state shall be always configured.

This parameter shall only be configured if the parameter AdcLowPowerStatesSupport is set to true.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	true
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	9223372036854775807
min	0

4.166 Parameter AdcPowerStateReadyCbkJRef

Each instance of this parameter contains a reference to a power mode callback defined in a CDD or IoHwAbs component.

This parameter shall only be configured if the parameter AdcLowPowerStatesSupport is set to true

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	NULL_PTR

4.167 Container AdcInterrupt

Selects whether the interrupt for each ADC Unit will be enabled. For each Adc HW unit, there are 2 interrupts that can be enabled: the End of Conversion and the Watchdog interrupts. These settings are used for optimizing the code size by removing the interrupt handling code for interrupts that are not needed.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	2
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE

4.168 Parameter AdcInterruptSource

The name of the interrupt. Note: Implementation Specific Parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	ADC0_EOC
literals	['ADC0_EOC', 'ADC0_WD', 'ADC1_EOC', 'ADC1_WD']

4.169 Parameter AdcInterruptEnable

Adds / removes the interrupt handling routine from the ADC driver code.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.170 Container AdcPublishedInformation

Additional published parameters not covered by CommonPublishedInformation container.

Note that these parameters do not have any configuration class setting, since they are published information.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.171 Parameter AdcChannelValueSigned

Information whether the result value of the ADC driver has sign information (true) or not (false). If the result shall be interpreted as signed value it shall apply to C-language rules.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	false

4.172 Parameter AdcGroupFirstChannelFixed

Information whether the first channel of an ADC Channel group can be configured (false) or is fixed (true) to a value determined by the ADC HW Unit.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	false

4.173 Parameter AdcMaxChannelResolution

Maximum Channel resolution in bits (does not specify accuracy).

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	12
max	63
min	1

4.174 Container CommonPublishedInformation

Common container, aggregated by all modules. It contains published information about vendor and versions.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.175 Parameter ArReleaseMajorVersion

Major version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	4
max	4
min	4

4.176 Parameter ArReleaseMinorVersion

Minor version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	4
max	4
min	4

4.177 Parameter ArReleaseRevisionVersion

Revision version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A

Property	Value
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.178 Parameter ModuleId

Module ID of this module from Module List.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	123
max	123
min	123

4.179 Parameter SwMajorVersion

Major version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false

Property	Value
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	4
max	4
min	4

4.180 Parameter SwMinorVersion

Minor version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.181 Parameter SwPatchVersion

Patch level version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION

Property	Value
defaultValue	2
max	2
min	2

4.182 Parameter VendorApiInfix

In driver modules which can be instantiated several times on a single ECU, BSW00347 requires that the name of APIs is extended by the VendorId and a vendor specific name.

This parameter is used to specify the vendor specific name. In total, the implementation specific name is generated as follows:

<ModuleName>_>VendorId>_<VendorApiInfix>.

E.g. assuming that the VendorId of the implementor is 123 and the implementer chose a VendorApiInfix of "v11r456" a api name Can_Write defined in the SWS will translate to Can_123_v11r456Write.

This parameter is mandatory for all modules with upper multiplicity > 1. It shall not be used for modules with upper multiplicity =1.

Property	Value
type	ECUC-STRING-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	

4.183 Parameter VendorId

Vendor ID of the dedicated implementation of this module according to the AUTOSAR vendor list.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	43
max	43
min	43

4.184 Container AutosarExt

Autosar Extension API settings.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.185 Parameter AdcTimeoutMethod

Configures the timeout method for Adc.

Based on this selection a certain timeout method from OsIf will be used in the driver.

Note: If OSIF_COUNTER_SYSTEM or OSIF_COUNTER_CUSTOM are selected make sure the corresponding timer is enabled in OsIf General configuration.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP

Property	Value
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	OSIF_COUNTER_DUMMY
literals	['OSIF_COUNTER_DUMMY', 'OSIF_COUNTER_SYSTEM', 'OSIF_COUNTER_CUSTOM']

4.186 Parameter AdcTimeoutVal

The timeout is used for preventing endless loops as resulted from driver FMEA analysis.

The hardware failure mode which is preventing, is potential cases with frozen peripheral status used for driver synchronization.

The timeout is used as escape for avoidance of endless loops. For more details please refer to driver FMEA.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	100000
max	4294967295
min	0

4.187 Parameter AdcSarIpDevErrorDetect

This parameter Enables / Disables development error detection for Adc Sar.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	false

4.188 Parameter CtuIpDevErrorDetect

Enables / Disables development error detection for Ctu.

This feature is enabled if Adc CTU Control Mode API from General/AutosarExt is enabled or if Adc Hw Trigger from General is enabled or when at least one Adc Group from at least one Hw Unit has Adc Group Trigger Source set as ADC_TRIGG_SRC_HW and Adc Group Hw Trigger Source different from EXT_TRIG.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	false

4.189 Parameter AdcMulticoreSupport

This parameter globally enables the possibility to support multicore. If this parameter is enabled, at least one EcucPartition needs to be defined (in all variants).

Note This is an Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	false

4.190 Parameter AdcEnableGroupDependentChannelNames

This is used to generate ADC symbolic names, that depend also on the ADC group

to which each ADC channel is mapped. The generated symbolic name will be something like:

```
#define "ADC_GroupName"_"ADC_ChannelName" "Channel index value",
```

where "Channel index value" is the channel index in the current group.

Channel indexes in each group are generated to allow result buffer access by symbolic names.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.191 Parameter AdcBypassAbortChainCheck

Bypass the delay introduced to check if an aborted conversion chain has stopped.

This increases ADC driver performance at the cost of HW-SW coherency no longer being guaranteed.

The user must make sure he does not call an ADC service before the hardware reaches the correct state.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.192 Parameter AdcConvTimeOnce

Implementation Specific Parameter.

Enable/Disable one time setting of the registers.

If Enabled, the setting of the conversion time registers will be done only once in `Adc_Init()` function

for the configured hardware unit.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.193 Parameter AdcOptimizeOneShotHwTriggerConversions

Implementation Specific Parameter.

Tresos Configuration Plug-in

Enable/Disable The Adc driver optimization for HW Triggered groups, OneShot, Single access.

If Enabled, other types of groups cannot be configured in ADC driver and the code for interrupt routine / Dma notification will be optimized for speed.

Also, all groups must have at most 8 channels configured.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.194 Parameter AdcOptimizeDmaStreamingGroups

Implementation Specific Parameter.

Enable/Disable The Adc driver enable Optimize DMA streaming groups for reducing the number of interrupts required for processing the conversions of Adc Groups that consist of one or more channels (depending on HW capabilities) and which are configured as ADC_ACCESS_MODE_STREAMING.

When this feature is enabled, only one interrupt will be raised after the completion of all stream conversions (as configured by AdcStreamingNumSamples parameter). An additional interrupt to be raised after half of the stream is converted shall also be configurable.

This feature is enabled if Adc Global Enable DMA Transfer from General/AutosarExt is also enabled.

Note:

- SetChannel(), Enable/DisableChannel() and Optimize one-shot hardware trigger cannot be use concurrently with this feature.

This node is EDITABLE only if "Adc Enable DMA support" is enabled

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.195 Parameter AdcPreSamplingOnce

Implementation Specific Parameter.

Enable/Disable one time setting of the registers.

If Enabled, the setting of the presampling time registers will be done only once in `Adc_Init()` function for the configured hardware unit.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.196 Parameter AdcEnableInitialNotification

Enable/disable an extra notification to be called for each Adc Group conversion.

This feature is intended to be used together with `Adc_SetChannel` service. The initial notification can be used by the user application to call `Adc_SetChannel` API before ADC driver updates the hardware configuration for the next conversion.

This node is EDITABLE only if "Adc Set Channel API" is enabled.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.197 Parameter AdcEnableDmaTransferMode

This parameter enables the possibility to configure DMA transfer for ADC converted data. If this parameter is disabled then DMA handling code will be removed at pre-compile time and DMA transfer cannot be configure for any Adc unit in any variant. If this parameter is enabled then the DMA configuration code will not be removed.

Note :This is an Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	false

4.198 Parameter AdcUseSoftwareInjectedGroups

This parameter defines if Software Injected Groups are used in any Hardware Unit, any variant.

It needs to be enabled if Software Injected Groups are needed. If Software Injected Groups are not needed, this parameter should be disabled for code optimizations.

Note This is an Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	false

4.199 Parameter AdcUseHardwareNormalGroups

This parameter defines if Hardware Normal Groups are used in any Hardware Unit, any variant. It needs to be enabled if Hardware Normal Groups are needed. If Hardware Normal Groups are not needed, this parameter should be disabled for code optimizations.

Note This is an Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	false

4.200 Parameter AdcEnableUserModeSupport

When this parameter is enabled, the Adc module will adapt to run from User Mode, by configuring REG_PROT for ADC IPs

Note: The Adc driver code can be executed at any time from both supervisor and user mode.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.201 Parameter AdcSetHwUnitPowerModeApi

Adds/removes the Autosar Extension implementation API `Adc_SetHwUnitPowerMode()` from the code.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.202 Parameter AdcEnableChDisableChApi

Enable/disable the Autosar Extension implementation api(s) `Adc_EnableChannel()` and `Adc_DisableChannel()` in ADC driver.

Note: This is an Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.203 Parameter AdcGetInjectedConvStatusApi

Enable/disable the Autosar Extension API `Adc_GetInjectedConversionStatus()` in ADC driver.

Note This is an Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.204 Parameter AdcEnableThresholdConfigurationApi

Enable/disable the Autosar Extension APIs `Adc_ConfigureThreshold()` in ADC driver.

Note: This is an Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.205 Parameter AdcEnableCtuTrigAutosarExtApi

This is used to enable the Autosar Extension API for the hardware triggered group.

If this parameter is enabled then `Adc_EnableCTUTrigger()`, `Adc_DisableCTUTrigger()` and `Adc_HwResultReadGroup()` will be available in the driver code.

This is an Implementation Specific Parameter.

When this parameter is enabled, the result buffer is no longer to be used to read the results as the result will be directly read from HW registers.

When this parameter is disabled, normal functionality shall be executed.

This node is EDITABLE only if "Adc Hw Trigger API" is enabled

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.206 Parameter AdcCtuHardwareTriggerOptimization

When this parameter is enabled, the Ctu channel lists and triggers are configured only once at initialization in order to reduce cpu load when calling the `Adc_EnableHardwareTrigger` and `Adc_EnableCtuTrigger` APIs.

The `Adc_SetChannel` service cannot be used, and the maximum size of groups cannot exceed the size of the entire command list divided by the number of adc hardware units triggered by the CTU/BCTU.

This node is EDITABLE only if "Adc Hw Trigger API" is enabled

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.207 Parameter `AdcEnableCtuControlModeApi`

This is used to enable the Autosar Extension API for the enabling and disabling CTU control mode for an ADC unit.

If this parameter is enabled than `Adc_EnableCtuControlMode()`, `Adc_DisableCtuControlMode()` will be available in the driver code.

When a unit works in CTU control mode, no other conversions shall run in parallel(`Adc`). The only conversions occurring shall be the ones defined in the CTU configuration.

If `AdcEnableCtuControlModeApi` is enabled, CTU must be configured.

Note This is an Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.208 Parameter CtuEnableDmaTransferMode

This parameter enables the possibility to configure DMA transfer for CTU control mode.

If this parameter is disabled then DMA handling code will be removed at pre-compile time and DMA transfer cannot be configured for any CTU unit in any variant.

If this parameter is enabled then the DMA configuration code will not be removed.

Note: This is an Implementation Specific Parameter.

This node is EDITABLE only if "Adc CTU Control Mode API" is enabled

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.209 Parameter AdcCtuControlModeExtraApis

The option provides some extra APIs for Ctu Control Mode: Adc_CtuWriteTriggerEnableMask, Adc_CtuSetTriggerEnable, Adc_CtuSetTriggerAdcCmdAddress and Adc_CtuSetTriggerCompare.

These APIs are used to configure Trigger Handler Control Register (THCR), Command List Control Register (CLCR) and Trigger Compare Register (TxCR).

Note: This is an Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Property	Value
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.210 Parameter AdcEnableWatchdogApi

This parameter globally enables the possibility to use the Adc Watchdog feature. If this parameter is disabled, the Watchdog handling code will be removed at pre-compile time and nothing related to this functionality can be configured in any unit, for any variant. If this parameter is enabled, Analog Watchdog functionality can be configured.

Note: This is an Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.211 Parameter AdcEnableSetChannel

If this parameter has been configured to "TRUE", the Autosar Extension function "Adc_SetChannel()" shall be accessible, otherwise this function shall be removed from the code.

Note: This is an Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A

Property	Value
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	false

4.212 Parameter AdcEnableDualClockMode

Adds/removes the Dual Clock mode service `Adc_SetClockMode` from the code.

Also it enables the Programming of Conversion Timing registers in `Adc_SetClockMode`.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.213 Parameter AdcEnableCalibration

If this parameter has been configured to "TRUE", the Autosar Extension function "`Adc_Calibrate()`" shall be accessible, otherwise this function shall be removed from the code.

Note This is an Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Property	Value
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.214 Parameter AdcEnableSelfTest

If this parameter has been configured to "TRUE", the Autosar Extension function "Adc_SelfTest()" shall be accessible, otherwise this function shall be removed from the code.

Note This is an Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.215 Parameter AdcEnableReadRawDataApi

When this parameter is enabled, the Api for reading the raw result data from an ADC unit is available to use at runtime.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE

Property	Value
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.216 Parameter AdcEnableGroupStreamingResultReorder

When this parameter is enabled, the adc results can be arranged as multiple sets of group result buffer if AdcStream-ResultGroup is enabled for that selected group.

E.g: for a group with channels {CH1 CH5 CH7} the resulting stream buffer shall be:

{ CH1, CH5, CH7, CH1, CH5, CH7, CH1, CH5, CH7}

instead of

{ CH1, CH1, CH1, CH5, CH5, CH5, CH7, CH7, CH7} like supported by AUTOSAR standard.

Apply only for ADC_ACCESS_MODE_STREAMING Access Mode.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false



Chapter 5

Module Index

5.1 Software Specification

Here is a list of all modules:

Adc driver	152
Adc Sar IPL	197
Ctu IPL	233

Chapter 6

Module Documentation

6.1 Adc driver

6.1.1 Detailed Description Adc HLD.

Adc Autosar High Level Driver.

Data Structures

- struct [Adc_ValidationResultType](#)
Structure for validation results. [More...](#)
- struct [Adc_GroupStatusType](#)
Structure for group status. [More...](#)
- struct [Adc_UnitStatusType](#)
Structure for hardware unit status. [More...](#)
- struct [Adc_ConfigType](#)
Structure for ADC configuration. [More...](#)

Macros

- `#define ADC_E_UNINIT`
API service used without Adc module initialization.
- `#define ADC_E_BUSY`
Adc module is busy with a running operation.
- `#define ADC_E_IDLE`
Adc module is in idle state.
- `#define ADC_E_ALREADY_INITIALIZED`
The ADC module is already initialized.
- `#define ADC_E_PARAM_CONFIG`
The ADC module is not properly configured.
- `#define ADC_E_PARAM_POINTER`
API service is called using an invalid pointer (e.g. the pointer should not be NULL).

- `#define ADC_E_PARAM_GROUP`
API service used with an invalid ADC group.
- `#define ADC_E_WRONG_CONV_MODE`
API service used with an invalid ADC Conversion Mode.
- `#define ADC_E_WRONG_TRIGG_SRC`
API service used with an invalid ADC Trigger Source.
- `#define ADC_E_NOTIF_CAPABILITY`
Check the notification capability of a group.
- `#define ADC_E_BUFFER_UNINIT`
API service used without initializing the buffer.
- `#define ADC_E_QUEUE_FULL`
The `Adc_StartGroupConversion` and `Adc_EnableHardwareTrigger` services can not queue another conversion (queue is full)
- `#define ADC_E_SET_HW_UNIT_POWER_MODE`
An error occurred when the `Adc_SetHwUnitPowerMode` services is used.
- `#define ADC_E_PARAM_TRIGGER`
Wrong trigger source to be used for the group.
- `#define ADC_E_WRONG_CONF_THRHLN_VALUE`
`Adc_ConfigureThreshold` service is called using wrong values.
- `#define ADC_E_PARAM_UNIT`
API service called using a wrong ADC unit.
- `#define ADC_E_WRONG_CTU_TRIGGER`
API service called using a wrong CTU trigger.
- `#define ADC_E_WRONG_CTU_CLCR_TRIGGER`
API service called using a wrong CTU CLCR trigger.
- `#define ADC_E_INVALID_CLOCK_MODE`
`Adc_SetClockMode` service called using an invalid clock mode.
- `#define ADC_E_TIMEOUT`
An error occurred if the timeout counter variable has expired when checking status flags.
- `#define ADC_E_CONTROL_MODE_DISABLED`
Error when a function which works only in CTU Control Mode is called when CTU control mode is disabled.
- `#define ADC_INIT_ID`
API service ID for `Adc_Init` function.
- `#define ADC_DEINIT_ID`
API service ID for `Adc_DeInit` function.
- `#define ADC_STARTGROUPCONVERSION_ID`
API service ID for `Adc_StartGroupConversion` function.
- `#define ADC_STOPGROUPCONVERSION_ID`
API service ID for `Adc_StopGroupConversion` function.
- `#define ADC_VALUEREADGROUP_ID`
API service ID for `Adc_ReadGroup` function.
- `#define ADC_ENABLEHARDWARETRIGGER_ID`
API service ID for `Adc_EnableHardwareTrigger` function.
- `#define ADC_DISABLEHARDWARETRIGGER_ID`
API service ID for `Adc_DisableHardwareTrigger` function.
- `#define ADC_ENABLEGROUPNOTIFICATION_ID`
API service ID for `Adc_EnableGroupNotification` function.

- #define [ADC_DISABLEGROUPNOTIFICATION_ID](#)
API service ID for *Adc_DisableGroupNotification* function.
- #define [ADC_GETGROUPSTATUS_ID](#)
API service ID for *Adc_GetGroupStatus* function.
- #define [ADC_GETVERSIONINFO_ID](#)
API service ID for *Adc_GetVersionInfo* function.
- #define [ADC_GETSTREAMLASTPOINTER_ID](#)
API service ID for *Adc_GetStreamLastPointer* function.
- #define [ADC_SETUPRESULTBUFFER_ID](#)
API service ID for *Adc_SetupResultBuffer* function.
- #define [ADC_HWRESULTREADGROUP_ID](#)
API service ID for *Adc_HwResultReadGroup* function.
- #define [ADC_ENABLECTUTRIGGER_ID](#)
API service ID for *Adc_EnableCTUTrigge* function.
- #define [ADC_DISABLECTUTRIGGER_ID](#)
API service ID for *Adc_DisableCTUTrigger* function.
- #define [ADC_SET_HW_UNIT_POWER_MODE_ID](#)
API service ID for *Adc_SetHwUnitPowerMode* function.
- #define [ADC_SETCLOCKMODE_ID](#)
API service ID for *Adc_SetClockMode* function.
- #define [ADC_GETINJECTEDCONVERSIONSTATUS_ID](#)
API service ID for *Adc_GetInjectedConversionStatus* function.
- #define [ADC_CALIBRATE_ID](#)
API service ID for *Adc_Calibrate* function.
- #define [ADC_CONFIGURE_THRESHOLD_ID](#)
API service ID for *Adc_ConfigureThreshold* function.
- #define [ADC_CTU_SET_WRITE_TRIG_EN_MASK_ID](#)
API service ID for *Adc_CtuWriteTriggerEnableMask* function.
- #define [ADC_CTU_SET_TRIGGER_ENABLE_ID](#)
API service ID for *Adc_CtuSetTriggerEnable* function.
- #define [ADC_CTU_SET_TRIGGER_ADC_CMD_ADDRESS_ID](#)
API service ID for *Adc_CtuSetTriggerAdcCmdAddress* function.
- #define [ADC_CTU_SET_TRIGGER_COMPARE_ID](#)
API service ID for *Adc_CtuSetTriggerCompare* function.
- #define [ADC_ENABLE_CTU_CONTROL_MODE_ID](#)
API service ID for *Adc_EnableCtuControlMode* function.
- #define [ADC_DISABLE_CTU_CONTROL_MODE_ID](#)
API service ID for *Adc_DisableCtuControlMode* function.
- #define [ADC_DATA_MASK_U16](#)
Mask used to verify the DATA registers content.

Types Reference

- typedef void(* [Adc__NotifyType](#)) (void)
Notification function pointer definition.
- typedef uint8 [Adc__ResolutionType](#)
channel resolution in number of bits
- typedef Adc_ChannelType [Adc__GroupDefType](#)
definition of channels in a group
- typedef uint8 [Adc__PrescaleType](#)
clock prescaler factor
- typedef uint8 [Adc__SamplingTimeType](#)
sampling time
- typedef uint16 [Adc__StreamNumSampleType](#)
Number of samples of a streaming conversion buffer.

Enum Reference

- enum [Adc__GlobalStateType](#)
ADC driver status.
- enum [Adc__GroupConversionStateType](#)
ADC group already converted type.
- enum [Adc__GroupAccessModeType](#)
Adc group access Mode.
- enum [Adc__GroupReplacementType](#)
Adc group replacement.
- enum [Adc__StreamBufferModeType](#)
Adc group streaming buffer mode.
- enum [Adc__StatusType](#)
ADC group status.
- enum [Adc__NotificationType](#)
ADC group notification.
- enum [Adc__HwTriggerSignalType](#)
Adc hardware trigger edge.
- enum [Adc__TriggerSourceType](#)
Adc hardware trigger source.
- enum [Adc__HwTriggeringType](#)
Adc Hardware trigger.

Function Reference

- void [Adc_Init](#) (const [Adc_ConfigType](#) *ConfigPtr)
Initializes the ADC hardware unit and the driver.
- Std_ReturnType [Adc_SetupResultBuffer](#) ([Adc_GroupType](#) Group, [Adc_ValueGroupType](#) *const DataBufferPtr)
Initializes the group specific ADC result buffer pointer as configured to point to the pDataBufferPtr address which is passed as parameter.
- void [Adc_DeInit](#) (void)
Returns all ADC HW Units to a state comparable to their power on reset state.
- void [Adc_StartGroupConversion](#) ([Adc_GroupType](#) Group)
Starts the conversion of all channels of the requested ADC Channel group.
- void [Adc_StopGroupConversion](#) ([Adc_GroupType](#) Group)
Stops the conversion of all channels of the requested ADC Channel group.
- Std_ReturnType [Adc_ReadGroup](#) ([Adc_GroupType](#) Group, [Adc_ValueGroupType](#) *DataBufferPtr)
Reads the group conversion results.
- void [Adc_ReadRawData](#) ([Adc_HwUnitType](#) Unit, const [Adc_ChannelType](#) *const ChansArray, uint8 NumItems, [Adc_ValueGroupType](#) *const DataBufferPtr)
Read the raw result data from an ADC unit.
- void [Adc_EnableHardwareTrigger](#) ([Adc_GroupType](#) Group)
Enables the hardware trigger for the requested ADC Channel group.
- void [Adc_DisableHardwareTrigger](#) ([Adc_GroupType](#) Group)
Disables the hardware trigger for the requested ADC Channel group.
- void [Adc_EnableGroupNotification](#) ([Adc_GroupType](#) Group)
Enables the notification mechanism for the requested ADC channel group.
- void [Adc_DisableGroupNotification](#) ([Adc_GroupType](#) Group)
Disables the notification mechanism for the requested ADC channel group.
- [Adc_StatusType](#) [Adc_GetGroupStatus](#) ([Adc_GroupType](#) Group)
Returns the conversion status of the requested ADC Channel group.
- [Adc_StreamNumSampleType](#) [Adc_GetStreamLastPointer](#) ([Adc_GroupType](#) Group, [Adc_ValueGroupType](#) **PtrToSamplePtr)
Returns the number of valid samples per channel.
- void [Adc_GetVersionInfo](#) ([Std_VersionInfoType](#) *versioninfo)
Returns the version information of this module.
- Std_ReturnType [Adc_SetHwUnitPowerMode](#) ([Adc_HwUnitType](#) Unit, [Adc_SetPowerModeType](#) SetPowerMode)
Set the ADC mode either to powerdown or normal.
- void [Adc_EnableCTUTrigger](#) ([Adc_GroupType](#) Group, [Adc_HwTriggerTimerType](#) TriggerSource)
Enable the TriggerSource for group selected by Group parameter.
- void [Adc_DisableCTUTrigger](#) ([Adc_GroupType](#) Group, [Adc_HwTriggerTimerType](#) TriggerSource)
Disable the TriggerSource for group selected by Group parameter.
- Std_ReturnType [Adc_HwResultReadGroup](#) ([Adc_GroupType](#) Group, [Adc_ValueGroupType](#) *DataPtr)
Read the result of the hardware triggered groups conversion result.
- [Adc_StatusType](#) [Adc_GetInjectedConversionStatus](#) ([Adc_HwUnitType](#) Unit)
Get the injected conversions status.
- void [Adc_Calibrate](#) ([Adc_HwUnitType](#) Unit, [Adc_CalibrationStatusType](#) *pStatus)
Executes high accuracy calibration of a ADC HW unit.

- Std_ReturnType [Adc_SelfTest](#) (Adc_HwUnitType Unit)
Executes hardware Self Test of a ADC HW unit.
- void [Adc_ConfigureThreshold](#) (Adc_ThresholdControlIndexType ThresholdControlIndex, Adc_WdgThresholdValueType LowValue, Adc_WdgThresholdValueType HighValue)
Function to reconfigure High and Low thresholds for a given threshold control index.
- void [Adc_EnableWdgNotification](#) (Adc_ChannelType ChannelId)
Enable notification of a channel that has watchdog functionality configured at initialization.
- void [Adc_DisableWdgNotification](#) (Adc_ChannelType ChannelId)
Disable notification of a channel that has watchdog functionality configured at initialization.
- Std_ReturnType [Adc_SetClockMode](#) (Adc_SelectPrescalerType Prescaler)
Set the ADC clock prescaler if available and modify the conversion timings.
- Std_ReturnType [Adc_CtuWriteTriggerEnableMask](#) (uint8 Trigger, uint8 ThcrValue)
Function to set the trigger handler control registers of the CTU IPL.
- Std_ReturnType [Adc_CtuSetTriggerEnable](#) (uint8 Trigger, boolean Enable)
Function to set or clear the bit Tx_E of the trigger handler control registers of the CTU IPL.
- Std_ReturnType [Adc_CtuSetTriggerAdcCmdAddress](#) (uint8 Trigger, uint8 CldrValue)
Function to set the cmd list handler control registers of the CTU IPL.
- Std_ReturnType [Adc_CtuSetTriggerCompare](#) (uint8 Trigger, uint16 CompareValue)
Function to set the trigger compare registers of the CTU IPL.
- void [Adc_EnableCtuControlMode](#) (Adc_HwUnitType Unit)
Function to enable CTU control mode for an ADC unit.
- void [Adc_DisableCtuControlMode](#) (Adc_HwUnitType Unit)
Function to disable CTU control mode for an ADC unit.
- void [Adc_CtuEnableHwTrigger](#) (Adc_CtuTrigSrcType TrigSource)
Function to enable CTU hardware trigger.
- void [Adc_CtuDisableHwTrigger](#) (Adc_CtuTrigSrcType TrigSource)
Function to disable CTU hardware trigger.
- void [Adc_CtuStartConversion](#) (Adc_CtuTrigSrcType TrigSource)
Function to start CTU conversion.
- void [Adc_CtuMasterReload](#) (Adc_HwUnitType CtuUnit)
Issue a software master reload signal for CTU.
- void [Adc_CtuReadFifoData](#) (Adc_CtuFifoIdxType FifoIdx, uint16 *Data, uint8 DataLength)
Function to read CTU conversion data from FIFO.
- void [Adc_CtuReadFifoResult](#) (Adc_CtuFifoIdxType FifoIdx, [Adc_CtuFifoResultType](#) *Result, uint8 ResultLength)
Function to read CTU conversion results from FIFO.
- void [Adc_CtuSetFifoWatermark](#) (Adc_CtuFifoIdxType FifoIdx, uint8 Watermark)
Function to set CTU FIFO watermark.
- void [Adc_CtuEnableNotification](#) (Adc_CtuNotificationType Notification)
Function to enable CTU notification.
- void [Adc_CtuDisableNotification](#) (Adc_CtuNotificationType Notification)
Function to disable CTU notification.
- void [Adc_CtuSetList](#) (const [Adc_CtuListItemType](#) *const ListItemsArray, const uint8 NumItems, const uint8 ListStartPosition)
Function to set CTU list.
- void [Adc_CtuSetListPointer](#) (Adc_CtuTrigSrcType TrigSource, Adc_CtuListPtrType ListPtr)
Reconfigure the list pointer of a triggered CTU source.

Variables

- const [Adc_ConfigType](#) * [Adc_apxCfgPtr](#) [(1U)]
Used to point the configuration structure.

6.1.2 Data Structure Documentation

6.1.2.1 struct Adc_ValidationResultType

Structure for validation results.

This structure contains the validation information

Definition at line 285 of file [Adc_Types.h](#).

Data Fields

Type	Name	Description
boolean	EndValidations	Signal if validation ended.
Std_ReturnType	ValidParams	Return status.

6.1.2.2 struct Adc_GroupStatusType

Structure for group status.

This structure contains the group status information.

Definition at line 296 of file [Adc_Types.h](#).

Data Fields

Type	Name	Description
volatile Adc_StatusType	Conversion	Group status.
volatile Adc_GroupConversionStateType	AlreadyConverted	Group was previously converted or not.
Adc_HwTriggeringType	HwTriggering	hw trigger enabled/disabled
Adc_NotificationType	Notification	notification enabled/disabled
volatile Adc_StreamNumSampleType	ResultIndex	index into streaming buffer that is currently being filled
Adc_ValueGroupType *	ResultsBufferPtr	Pointer to user result buffer array.
volatile boolean	LimitCheckFailed	check limit check fail

6.1.2.3 struct Adc_UnitStatusType

Structure for hardware unit status.

This structure contains the HW unit status information.

Definition at line 322 of file Adc_Types.h.

Data Fields

Type	Name	Description
volatile Adc_QueueIndexType	SwNormalQueueIndex	Filled slots in the queue.
volatile Adc_GroupType	SwNormalQueue[(1U)]	Queued groups indexes, always executing Queue[0].
volatile Adc_GroupType	OngoingHwGroup	Ongoing hardware group ID.
volatile Adc_GroupType	SwInjectedQueue[1U]	The depth of the software injected queue.
volatile Adc_QueueIndexType	SwInjectedQueueIndex	Filled slots in the Sw injected queue.
boolean	CtuControlOngoing	Indicates Ctu control mode is ongoing.

6.1.2.4 struct Adc_ConfigType

Structure for ADC configuration.

Data structure containing the set of configuration parameters required for initializing the ADC Driver.

SWS_Adc_00505

Definition at line 441 of file Adc_Types.h.

Data Fields

- const Adc_GroupConfigurationType * [GroupsPtr](#)
Group configurations.
- Adc_GroupType [GroupCount](#)
Total number of groups.
- const uint16 * [GroupIdToIndexMapPtr](#)
Miscellaneous configuration parameters.
- uint32 [CoreId](#)
Configuration CoreID.
- const uint8 [AssignedPartitionCount](#)
Number of Partition.

6.1.2.4.1 Field Documentation

6.1.2.4.1.1 **GroupsPtr** `const Adc_GroupConfigurationType* GroupsPtr`

Group configurations.

Definition at line 446 of file `Adc_Types.h`.

6.1.2.4.1.2 **GroupCount** `Adc_GroupType GroupCount`

Total number of groups.

Definition at line 448 of file `Adc_Types.h`.

6.1.2.4.1.3 **GroupIdToIndexMapPtr** `const uint16* GroupIdToIndexMapPtr`

Miscellaneous configuration parameters.

Definition at line 450 of file `Adc_Types.h`.

6.1.2.4.1.4 **CoreId** `uint32 CoreId`

Configuration CoreID.

Assigned Partition

Definition at line 452 of file `Adc_Types.h`.

6.1.2.4.1.5 **AssignedPartitionCount** `const uint8 AssignedPartitionCount`

Number of Partition.

<

Definition at line 456 of file `Adc_Types.h`.

6.1.3 Macro Definition Documentation

6.1.3.1 ADC_E_UNINIT

```
#define ADC_E_UNINIT
```

API service used without Adc module initialization.

Development errors. The following errors shall be detectable by the ADC module depending on its configuration (development / production mode).

All error codes

Definition at line 132 of file Adc.h.

6.1.3.2 ADC_E_BUSY

```
#define ADC_E_BUSY
```

Adc module is busy with a running operation.

Definition at line 137 of file Adc.h.

6.1.3.3 ADC_E_IDLE

```
#define ADC_E_IDLE
```

Adc module is in idle state.

Definition at line 142 of file Adc.h.

6.1.3.4 ADC_E_ALREADY_INITIALIZED

```
#define ADC_E_ALREADY_INITIALIZED
```

The ADC module is already initialized.

Definition at line 147 of file Adc.h.

6.1.3.5 ADC_E_PARAM_CONFIG

```
#define ADC_E_PARAM_CONFIG
```

The ADC module is not properly configured.

Definition at line 152 of file Adc.h.

6.1.3.6 ADC_E_PARAM_POINTER

```
#define ADC_E_PARAM_POINTER
```

API service is called using an invalid pointer (e.g. the pointer should not be NULL).

Definition at line 157 of file Adc.h.

6.1.3.7 ADC_E_PARAM_GROUP

```
#define ADC_E_PARAM_GROUP
```

API service used with an invalid ADC group.

Definition at line 162 of file Adc.h.

6.1.3.8 ADC_E_WRONG_CONV_MODE

```
#define ADC_E_WRONG_CONV_MODE
```

API service used with an invalid ADC Conversion Mode.

Definition at line 167 of file Adc.h.

6.1.3.9 ADC_E_WRONG_TRIGG_SRC

```
#define ADC_E_WRONG_TRIGG_SRC
```

API service used with an invalid ADC Trigger Source.

Definition at line 172 of file Adc.h.

6.1.3.10 ADC_E_NOTIF_CAPABILITY

```
#define ADC_E_NOTIF_CAPABILITY
```

Check the notification capability of a group.

Definition at line 177 of file Adc.h.

6.1.3.11 ADC_E_BUFFER_UNINIT

```
#define ADC_E_BUFFER_UNINIT
```

API service used without initializing the buffer.

Definition at line 182 of file Adc.h.

6.1.3.12 ADC_E_QUEUE_FULL

```
#define ADC_E_QUEUE_FULL
```

The `Adc_StartGroupConversion` and `Adc_EnableHardwareTrigger` services can not queue another conversion (queue is full)

Definition at line 210 of file Adc.h.

6.1.3.13 ADC_E_SET_HW_UNIT_POWER_MODE

```
#define ADC_E_SET_HW_UNIT_POWER_MODE
```

An error occurred when the `Adc_SetHwUnitPowerMode` services is used.

Definition at line 216 of file Adc.h.

6.1.3.14 ADC_E_PARAM_TRIGGER

```
#define ADC_E_PARAM_TRIGGER
```

Wrong trigger source to be used for the group.

Definition at line 223 of file Adc.h.

6.1.3.15 ADC_E_WRONG_CONF_THRHLD_VALUE

```
#define ADC_E_WRONG_CONF_THRHLD_VALUE
```

Adc_ConfigureThreshold service is called using wrong values.

Definition at line 241 of file Adc.h.

6.1.3.16 ADC_E_PARAM_UNIT

```
#define ADC_E_PARAM_UNIT
```

API service called using a wrong ADC unit.

Definition at line 247 of file Adc.h.

6.1.3.17 ADC_E_WRONG_CTU_TRIGGER

```
#define ADC_E_WRONG_CTU_TRIGGER
```

API service called using a wrong CTU trigger.

Definition at line 253 of file Adc.h.

6.1.3.18 ADC_E_WRONG_CTU_CLCR_TRIGGER

```
#define ADC_E_WRONG_CTU_CLCR_TRIGGER
```

API service called using a wrong CTU CLCR trigger.

Definition at line 257 of file Adc.h.

6.1.3.19 ADC_E_INVALID_CLOCK_MODE

```
#define ADC_E_INVALID_CLOCK_MODE
```

Adc_SetClockMode service called using an invalid clock mode.

Definition at line 264 of file Adc.h.

6.1.3.20 ADC_E_TIMEOUT

```
#define ADC_E_TIMEOUT
```

An error occurred if the timeout counter variable has expired when checking status flags.

Definition at line 277 of file Adc.h.

6.1.3.21 ADC_E_CONTROL_MODE_DISABLED

```
#define ADC_E_CONTROL_MODE_DISABLED
```

Error when a function which works only in CTU Control Mode is called when CTU control mode is disabled.

Definition at line 283 of file Adc.h.

6.1.3.22 ADC_INIT_ID

```
#define ADC_INIT_ID
```

API service ID for Adc_Init function.

All AUTOSAR API's service IDs

Definition at line 307 of file Adc.h.

6.1.3.23 ADC_DEINIT_ID

```
#define ADC_DEINIT_ID
```

API service ID for Adc_DeInit function.

Definition at line 312 of file Adc.h.

6.1.3.24 ADC_STARTGROUPCONVERSION_ID

```
#define ADC_STARTGROUPCONVERSION_ID
```

API service ID for Adc_StartGroupConversion function.

Definition at line 317 of file Adc.h.

6.1.3.25 ADC_STOPGROUPCONVERSION_ID

```
#define ADC_STOPGROUPCONVERSION_ID
```

API service ID for `Adc_StopGroupConversion` function.

Definition at line 322 of file `Adc.h`.

6.1.3.26 ADC_VALUEREADGROUP_ID

```
#define ADC_VALUEREADGROUP_ID
```

API service ID for `Adc_ReadGroup` function.

Definition at line 327 of file `Adc.h`.

6.1.3.27 ADC_ENABLEHARDWARETRIGGER_ID

```
#define ADC_ENABLEHARDWARETRIGGER_ID
```

API service ID for `Adc_EnableHardwareTrigger` function.

Definition at line 332 of file `Adc.h`.

6.1.3.28 ADC_DISABLEHARDWARETRIGGER_ID

```
#define ADC_DISABLEHARDWARETRIGGER_ID
```

API service ID for `Adc_DisableHardwareTrigger` function.

Definition at line 337 of file `Adc.h`.

6.1.3.29 ADC_ENABLEGROUPNOTIFICATION_ID

```
#define ADC_ENABLEGROUPNOTIFICATION_ID
```

API service ID for `Adc_EnableGroupNotification` function.

Definition at line 342 of file `Adc.h`.

6.1.3.30 ADC_DISABLEGROUPNOTIFICATION_ID

```
#define ADC_DISABLEGROUPNOTIFICATION_ID
```

API service ID for `Adc_DisableGroupNotification` function.

Definition at line 347 of file `Adc.h`.

6.1.3.31 ADC_GETGROUPSTATUS_ID

```
#define ADC_GETGROUPSTATUS_ID
```

API service ID for `Adc_GetGroupStatus` function.

Definition at line 352 of file `Adc.h`.

6.1.3.32 ADC_GETVERSIONINFO_ID

```
#define ADC_GETVERSIONINFO_ID
```

API service ID for `Adc_GetVersionInfo` function.

Definition at line 357 of file `Adc.h`.

6.1.3.33 ADC_GETSTREAMLASTPOINTER_ID

```
#define ADC_GETSTREAMLASTPOINTER_ID
```

API service ID for `Adc_GetStreamLastPointer` function.

Definition at line 362 of file `Adc.h`.

6.1.3.34 ADC_SETUPRESULTBUFFER_ID

```
#define ADC_SETUPRESULTBUFFER_ID
```

API service ID for `Adc_SetupResultBuffer` function.

Definition at line 367 of file `Adc.h`.

6.1.3.35 ADC_HWRESULTREADGROUP_ID

```
#define ADC_HWRESULTREADGROUP_ID
```

API service ID for Adc_HwResultReadGroup function.

All Autosar Extension API's service IDs NOTE: Parameters used when raising an error/exception

Definition at line 399 of file Adc.h.

6.1.3.36 ADC_ENABLECTUTRIGGER_ID

```
#define ADC_ENABLECTUTRIGGER_ID
```

API service ID for Adc_EnableCTUTrigge function.

Definition at line 403 of file Adc.h.

6.1.3.37 ADC_DISABLECTUTRIGGER_ID

```
#define ADC_DISABLECTUTRIGGER_ID
```

API service ID for Adc_DisableCTUTrigger function.

Definition at line 407 of file Adc.h.

6.1.3.38 ADC_SET_HW_UNIT_POWER_MODE_ID

```
#define ADC_SET_HW_UNIT_POWER_MODE_ID
```

API service ID for Adc_SetHwUnitPowerMode function.

Definition at line 414 of file Adc.h.

6.1.3.39 ADC_SETCLOCKMODE_ID

```
#define ADC_SETCLOCKMODE_ID
```

API service ID for Adc_SetClockMode function.

Definition at line 427 of file Adc.h.

6.1.3.40 ADC_GETINJECTEDCONVERSIONSTATUS_ID

```
#define ADC_GETINJECTEDCONVERSIONSTATUS_ID
```

API service ID for Adc_GetInjectedConversionStatus function.

Definition at line 445 of file Adc.h.

6.1.3.41 ADC_CALIBRATE_ID

```
#define ADC_CALIBRATE_ID
```

API service ID for Adc_Calibrate function.

Definition at line 452 of file Adc.h.

6.1.3.42 ADC_CONFIGURE_THRESHOLD_ID

```
#define ADC_CONFIGURE_THRESHOLD_ID
```

API service ID for Adc_ConfigureThreshold function.

Definition at line 463 of file Adc.h.

6.1.3.43 ADC_CTU_SET_WRITE_TRIG_EN_MASK_ID

```
#define ADC_CTU_SET_WRITE_TRIG_EN_MASK_ID
```

API service ID for Adc_CtuWriteTriggerEnableMask function.

Definition at line 470 of file Adc.h.

6.1.3.44 ADC_CTU_SET_TRIGGER_ENABLE_ID

```
#define ADC_CTU_SET_TRIGGER_ENABLE_ID
```

API service ID for Adc_CtuSetTriggerEnable function.

Definition at line 474 of file Adc.h.

6.1.3.45 ADC_CTU_SET_TRIGGER_ADC_CMD_ADDRESS_ID

```
#define ADC_CTU_SET_TRIGGER_ADC_CMD_ADDRESS_ID
```

API service ID for `Adc_CtuSetTriggerAdcCmdAddress` function.

Definition at line 478 of file `Adc.h`.

6.1.3.46 ADC_CTU_SET_TRIGGER_COMPARE_ID

```
#define ADC_CTU_SET_TRIGGER_COMPARE_ID
```

API service ID for `Adc_CtuSetTriggerCompare` function.

Definition at line 482 of file `Adc.h`.

6.1.3.47 ADC_ENABLE_CTU_CONTROL_MODE_ID

```
#define ADC_ENABLE_CTU_CONTROL_MODE_ID
```

API service ID for `Adc_EnableCtuControlMode` function.

Definition at line 496 of file `Adc.h`.

6.1.3.48 ADC_DISABLE_CTU_CONTROL_MODE_ID

```
#define ADC_DISABLE_CTU_CONTROL_MODE_ID
```

API service ID for `Adc_DisableCtuControlMode` function.

Definition at line 500 of file `Adc.h`.

6.1.3.49 ADC_DATA_MASK_U16

```
#define ADC_DATA_MASK_U16
```

Mask used to verify the DATA registers content.

Definition at line 536 of file `Adc.h`.

6.1.4 Types Reference

6.1.4.1 `Adc__NotifyType`

```
typedef void(* Adc_NotifyType) (void)
```

Notification function pointer definition.

Definition at line 252 of file `Adc_Types.h`.

6.1.4.2 `Adc__ResolutionType`

```
typedef uint8 Adc_ResolutionType
```

channel resolution in number of bits

Definition at line 256 of file `Adc_Types.h`.

6.1.4.3 `Adc__GroupDefType`

```
typedef Adc_ChannelType Adc_GroupDefType
```

definition of channels in a group

Definition at line 266 of file `Adc_Types.h`.

6.1.4.4 `Adc__PrescaleType`

```
typedef uint8 Adc_PrescaleType
```

clock prescaler factor

Definition at line 270 of file `Adc_Types.h`.

6.1.4.5 Adc_SamplingTimeType

```
typedef uint8 Adc_SamplingTimeType
```

sampling time

Definition at line 274 of file Adc_Types.h.

6.1.4.6 Adc_StreamNumSampleType

```
typedef uint16 Adc_StreamNumSampleType
```

Number of samples of a streaming conversion buffer.

Definition at line 278 of file Adc_Types.h.

6.1.5 Enum Reference

6.1.5.1 Adc_GlobalStateType

```
enum Adc_GlobalStateType
```

ADC driver status.

Used to differentiate if ADC driver is already uninit, during init or already initialized or not.

Enumerator

ADC_STATE_UNINIT	Adc driver uninitialized.
ADC_STATE_BUSY	Adc driver busy.
ADC_STATE_IDLE	Adc driver idle.

Definition at line 119 of file Adc_Types.h.

6.1.5.2 Adc_GroupConversionStateType

```
enum Adc_GroupConversionStateType
```

ADC group already converted type.

Used to differentiate if group is already converted or not.

Enumerator

ADC_NOT_YET_CONVERTED	Group not yet converted.
ADC_ALREADY_CONVERTED	Group is already converted.

Definition at line 132 of file Adc_Types.h.

6.1.5.3 Adc_GroupAccessModeType

enum [Adc_GroupAccessModeType](#)

Adc group access Mode.

Used for value received by Tressos interface configuration.

SWS_Adc_00528

Enumerator

ADC_ACCESS_MODE_SINGLE	Single access mode.
ADC_ACCESS_MODE_STREAMING	Streaming access mode.

Definition at line 145 of file Adc_Types.h.

6.1.5.4 Adc_GroupReplacementType

enum [Adc_GroupReplacementType](#)

Adc group replacement.

Used for value received by Tressos interface configuration.

SWS_Adc_00523

Enumerator

ADC_GROUP_REPL_ABORT_RESTART	Abort and restart of group.
ADC_GROUP_REPL_SUSPEND_RESUME	Suspend and resuming of group.

Definition at line 158 of file Adc_Types.h.

6.1.5.5 Adc_StreamBufferModeType

enum [Adc_StreamBufferModeType](#)

Adc group streaming buffer mode.

Used for value received by Tressos interface configuration.

SWS__Adc__00519

Enumerator

ADC_STREAM_BUFFER_LINEAR	Linear streaming.
ADC_STREAM_BUFFER_CIRCULAR	Circular streaming.

Definition at line 171 of file Adc_Types.h.

6.1.5.6 Adc_StatusType

enum [Adc_StatusType](#)

ADC group status.

ADC group enumeration type.

SWS__Adc__00513

Enumerator

ADC_IDLE	Group is in IDLE state.
ADC_BUSY	Group is in BUSY state.
ADC_COMPLETED	Group is in COMPLETED state.
ADC_STREAM_COMPLETED	Group is in STREAM_COMPLETED state.

Definition at line 184 of file Adc_Types.h.

6.1.5.7 Adc_NotificationType

enum [Adc_NotificationType](#)

ADC group notification.

Indicates if notification is enabled for the group.

Enumerator

ADC_NOTIFICATION_DISABLED	Notification is disabled.
ADC_NOTIFICATION_ENABLED	Notification is enabled.

Definition at line 197 of file Adc_Types.h.

6.1.5.8 Adc_HwTriggerSignalType

enum `Adc_HwTriggerSignalType`

Adc hardware trigger edge.

Used for value received by Tressos interface configuration.

SWS_Adc_00520

Enumerator

ADC_HW_TRIG_RISING_EDGE	Rising edge.
ADC_HW_TRIG_FALLING_EDGE	Falling edge.
ADC_HW_TRIG_BOTH_EDGES	falling and rising edge

Definition at line 211 of file Adc_Types.h.

6.1.5.9 Adc_TriggerSourceType

enum `Adc_TriggerSourceType`

Adc hardware trigger source.

Used for value received by Tressos interface configuration.

SWS_Adc_00514

Enumerator

ADC_TRIGG_SRC_SW	Software triggered.
ADC_TRIGG_SRC_HW	Hardware triggered.

Definition at line 225 of file Adc_Types.h.

6.1.5.10 Adc_HwTriggeringType

enum `Adc_HwTriggeringType`

Adc Hardware trigger.

Indicates if hardware trigger is enabled for group.

Enumerator

ADC_HWTRIGGER_DISABLED	Hardware trigger is disabled.
ADC_HWTRIGGER_ENABLED	Hardware trigger is enabled.

Definition at line 240 of file `Adc_Types.h`.

6.1.6 Function Reference

6.1.6.1 Adc_Init()

```
void Adc_Init (
    const Adc_ConfigType * ConfigPtr )
```

Initializes the ADC hardware unit and the driver.

This function will initialize both the ADC HW unit and the driver structures.

Parameters

in	<i>ConfigPtr</i>	Pointer to configuration set in Variant PB (Variant PC requires a NULL_PTR).
----	------------------	--

Returns

void

6.1.6.2 Adc_SetupResultBuffer()

```
Std_ReturnType Adc_SetupResultBuffer (
    Adc_GroupType Group,
    Adc_ValueGroupType *const DataBufferPtr )
```

Initializes the group specific ADC result buffer pointer as configured to point to the `pDataBufferPtr` address which is passed as parameter.

Initializes ADC driver with the group specific result buffer start address where the conversion results will be stored. The application has to ensure that the application buffer, where `pDataBufferPtr` points to, can hold all the conversion results of the specified group. The initialization with `Adc_SetupResultBuffer` is required after reset, before a group conversion can be started.

Parameters

in	<i>Group</i>	Numeric ID of requested ADC channel group.
in	<i>pDataBufferPtr</i>	Pointer to result data buffer

Returns

`Std_ReturnType` Standard return type. `E_OK`: Result buffer pointer initialized correctly. `E_NOT_OK`: Operation failed or development error occurred.

6.1.6.3 `Adc_DeInit()`

```
void Adc_DeInit (
    void )
```

Returns all ADC HW Units to a state comparable to their power on reset state.

Returns all ADC HW Units to a state comparable to their power on reset state, and de-initialize the ADC driver.

Returns

`void`

6.1.6.4 `Adc_StartGroupConversion()`

```
void Adc_StartGroupConversion (
    Adc_GroupType Group )
```

Starts the conversion of all channels of the requested ADC Channel group.

This function will start the SW conversion of all channels of the requested ADC channel group.

Parameters

in	<i>Group</i>	Numeric ID of requested ADC channel group.
----	--------------	--

Returns

void

6.1.6.5 **Adc_StopGroupConversion()**

```
void Adc_StopGroupConversion (
    Adc_GroupType Group )
```

Stops the conversion of all channels of the requested ADC Channel group.

This function will stop the SW conversion of all channels of the requested ADC channel group.

Parameters

in	<i>Group</i>	Numeric ID of requested ADC channel group.
----	--------------	--

Returns

void

6.1.6.6 **Adc_ReadGroup()**

```
Std_ReturnType Adc_ReadGroup (
    Adc_GroupType Group,
    Adc_ValueGroupType * DataBufferPtr )
```

Reads the group conversion results.

Reads the group conversion results of the last completed conversion round of the requested group and stores the channel values starting at the pDataBufferPtr address. The group channel values are stored in ascending channel number order (in contrast to the storage layout of the result buffer if streaming access is configured).

Parameters

in	<i>Group</i>	Numeric ID of requested ADC Channel group.
in	<i>pDataBufferPtr</i>	ADC results of all channels of the selected group are stored in the data buffer addressed with the pointer.

Returns

Std_ReturnType Standard return type. E_OK: results are available and written to the data buffer. E_NO←T_OK: no results are available or development error occurred.

6.1.6.7 Adc_ReadRawData()

```
void Adc_ReadRawData (
    Adc_HwUnitType Unit,
    const Adc_ChannelType *const ChansArray,
    uint8 NumItems,
    Adc_ValueGroupType *const DataBufferPtr )
```

Read the raw result data from an ADC unit.

Read the raw result data from an ADC unit. Intended for reading ADC results directly from ADC registers and can eliminate surplus interrupts if there are more triggered measurements than FIFO length. Measured values remain in ADC result registers(user must ensure that they are not overwritten).

Parameters

in	<i>Unit</i>	Adc unit used. Recommended to use generated define for Adc Logical Unit Id.
in	<i>ChansArray</i>	List of channels for which results to be read
in	<i>NumItems</i>	Number of results to read
out	<i>DataBufferPtr</i>	Destination pointer in which the results will be written

Returns

void

6.1.6.8 Adc_EnableHardwareTrigger()

```
void Adc_EnableHardwareTrigger (
    Adc_GroupType Group )
```

Enables the hardware trigger for the requested ADC Channel group.

This function will enable the HW trigger source for the requested ADC channel group. This function does set the CTU register for all platform that have the CTU Hw Unit.

Parameters

in	<i>Group</i>	Numeric ID of requested ADC channel group.
----	--------------	--

Returns

void

6.1.6.9 **Adc_DisableHardwareTrigger()**

```
void Adc_DisableHardwareTrigger (
    Adc_GroupType Group )
```

Disables the hardware trigger for the requested ADC Channel group.

This function will disable the HW trigger source for the requested ADC channel group.

Parameters

in	<i>Group</i>	Numeric ID of requested ADC channel group.
----	--------------	--

Returns

void

6.1.6.10 **Adc_EnableGroupNotification()**

```
void Adc_EnableGroupNotification (
    Adc_GroupType Group )
```

Enables the notification mechanism for the requested ADC channel group.

This function will enable the notification mechanism only for the requested ADC channel group.

Parameters

in	<i>Group</i>	Numeric ID of requested ADC channel group.
----	--------------	--

Returns

void

6.1.6.11 **Adc_DisableGroupNotification()**

```
void Adc_DisableGroupNotification (
    Adc_GroupType Group )
```

Disables the notification mechanism for the requested ADC channel group.

This function will disable the notification mechanism only for the requested ADC channel group.

Parameters

in	<i>Group</i>	Numeric ID of requested ADC channel group.
----	--------------	--

Returns

void

6.1.6.12 **Adc_GetGroupStatus()**

```
Adc_StatusType Adc_GetGroupStatus (
    Adc_GroupType Group )
```

Returns the conversion status of the requested ADC Channel group.

This function will return the conversion status of the requested ADC channel group.

Parameters

in	<i>Group</i>	Numeric ID of requested ADC channel group.
----	--------------	--

Returns

Adc_StatusType Conversion status for the requested group. ADC_IDLE in case of errors. conversion status in case of no errors.

6.1.6.13 **Adc_GetStreamLastPointer()**

```
Adc_StreamNumSampleType Adc_GetStreamLastPointer (
    Adc_GroupType Group,
    Adc_ValueGroupType ** PtrToSamplePtr )
```

Returns the number of valid samples per channel.

Returns the number of valid samples per channel, stored in the result buffer. Reads a pointer, pointing to a position in the group result buffer. With the pointer position, the results of all group channels of the last completed conversion round can be accessed. With the pointer and the return value, all valid group conversion results can be accessed (the user has to take the layout of the result buffer into account).

Parameters

in	<i>Group</i>	Numeric ID of requested ADC channel group.
out	<i>PtrToSamplePtr</i>	Pointer to result buffer pointer.

Module Documentation

Returns

Adc_StreamNumSampleType Number of valid samples per channel. 0 in case of errors. >0 Number of valid samples per channel.

6.1.6.14 Adc__GetVersionInfo()

```
void Adc_GetVersionInfo (
    Std_VersionInfoType * versioninfo )
```

Returns the version information of this module.

Returns the version information of this module.

Parameters

out	<i>pVersionInfo</i>	Pointer to where to store the version information of this module. structure in case of no errors.
-----	---------------------	---

6.1.6.15 Adc__SetHwUnitPowerMode()

```
Std_ReturnType Adc_SetHwUnitPowerMode (
    Adc_HwUnitType Unit,
    Adc_SetPowerModeType SetPowerMode )
```

Set the ADC mode either to powerdown or normal.

Set the ADC either to powerdown or normal mode.

Parameters

in	<i>Unit</i>	Adc unit used. Recommended to use generated define for Adc Logical Unit Id
in	<i>SetPowerMode</i>	Power mode to set: normal or powerdown.

Returns

Std_ReturnType Standard return type. E_OK: Transition successful. E_NOT_OK: Transition unsuccessful.

6.1.6.16 Adc_EnableCTUTrigger()

```
void Adc_EnableCTUTrigger (
    Adc_GroupType Group,
    Adc_HwTriggerTimerType TriggerSource )
```

Enable the TriggerSource for group selected by Group parameter.

This Autosar Extension API is used to enable any one of the configured TriggerSource of the Group. When this Autosar Extension API is used to enable the trigger source the CTU interrupt will be disabled by the driver. So user has to call the Autosar Extension API Adc_HwResultReadGroup to read the converted result from the ADC hardware register.

Parameters

in	<i>Group</i>	Index of group.
in	<i>TriggerSource</i>	Trigger source to be used for the group. (Configuration file should contain it for that group).

Returns

void

6.1.6.17 Adc_DisableCTUTrigger()

```
void Adc_DisableCTUTrigger (
    Adc_GroupType Group,
    Adc_HwTriggerTimerType TriggerSource )
```

Disable the TriggerSource for group selected by Group parameter.

This Autosar Extension API is used to disable the already enabled TriggerSource of the Group.

Parameters

in	<i>Group</i>	Index of group.
in	<i>TriggerSource</i>	Trigger source to be disabled for the group. (Configuration file should contain it for that group).

Returns

void

6.1.6.18 Adc_HwResultReadGroup()

```
Std_ReturnType Adc_HwResultReadGroup (
    Adc_GroupType Group,
    Adc_ValueGroupType * DataPtr )
```

Read the result of the hardware triggered groups conversion result.

This Autosar Extension API is used to read the result of the hardware triggered groups conversion result from the ADC hardware register in this case the CTU interrupt will be disabled for the group. The VALID bit C↔DR register will be cleared automatically when we read the results from the channel data register. If the user calls Autosar Extension function [Adc_HwResultReadGroup\(\)](#) once again before the next conversion takes place, the [Adc_HwResultReadGroup\(\)](#) returns E_NOT_OK.

Parameters

in	<i>Group</i>	Index of group.
in	<i>DataPtr</i>	Pointer to a buffer which will be filled by the conversion results.

Returns

Std_ReturnType Standard return type. E_OK: results are available and written to the data buffer. E_NO↔T_OK: no results are available or development error occurred.

6.1.6.19 Adc_GetInjectedConversionStatus()

```
Adc_StatusType Adc_GetInjectedConversionStatus (
    Adc_HwUnitType Unit )
```

Get the injected conversions status.

This function checks if an injected conversion (HW,SW) is ongoing

Parameters

in	<i>Unit</i>	Adc unit used. Recommended to use generated define for Adc Logical Unit Id
----	-------------	--

Returns

Adc_StatusType Status of the ADC HW unit. ADC_IDLE: SW,HW Injection or Hardware Trigger group are idle. ADC_BUSY: SW,HW Injection or Hardware Trigger is in progress.

6.1.6.20 Adc_Calibrate()

```
void Adc_Calibrate (
    Adc_HwUnitType Unit,
    Adc_CalibrationStatusType * pStatus )
```

Executes high accuracy calibration of a ADC HW unit.

This function calibrates the ADC HW unit and updates calibration related registers

Parameters

in	<i>Unit</i>	Adc unit used. Recommended to use generated define for Adc Logical Unit Id
in	<i>pStatus</i>	Status of the ADC HW unit calibration and list of failed and passed tests.

Returns

void

6.1.6.21 Adc_SelfTest()

```
Std_ReturnType Adc_SelfTest (
    Adc_HwUnitType Unit )
```

Executes hardware Self Test of a ADC HW unit.

This function checks if the ADC HW unit is functioning correctly

Parameters

in	<i>Unit</i>	Adc unit used. Recommended to use generated define for Adc Logical Unit Id
----	-------------	--

Returns

Std_ReturnType. Status of the ADC HW unit Self Test.

6.1.6.22 Adc_ConfigureThreshold()

```
void Adc_ConfigureThreshold (
    Adc_ThresholdControlIndexType ThresholdControlIndex,
```



```
Adc_WdgThresholdValueType LowValue,  
Adc_WdgThresholdValueType HighValue )
```

Function to reconfigure High and Low thresholds for a given threshold control index.

This function is used to reconfigure High and Low thresholds for a given threshold control index. Use generated symbolic name defines of threshold register (e.g. `AdcThresholdControl_0_0`), because function assumes `ThresholdControlIndex` to be in following format: Logical threshold id on bits until position defined by `ADC_THRESHOLD_↵D_SYMBOLIC_NAME_SHIFT_HW_UNIT_ID_U16`, and for the rest the Logical Unit Id

Parameters

in	<i>ThresholdControlIndex</i>	Symbolic name of threshold control
in	<i>LowValue</i>	Low threshold value of the threshold control
in	<i>HighValue</i>	High threshold value of the threshold control

Returns

void

6.1.6.23 **Adc__EnableWdgNotification()**

```
void Adc_EnableWdgNotification (  
    Adc_ChannelType ChannelId )
```

Enable notification of a channel that has watchdog functionality configured at initialization.

Enable notification of a channel that has watchdog functionality configured at initialization Use generated symbolic channel name defines (e.g. `AdcChannel_0_0`), because function assumes `ChannelId` to be in following format: Logical channel id on bits until position defined by `ADC_CHANNEL_SYMBOLIC_NAME_SHIFT_HW_UNIT_ID_↵U16`, and for the rest the Logical Unit Id

Parameters

in	<i>Adc_ChannelType</i>	Symbolic name of channel
----	------------------------	--------------------------

Returns

void.

6.1.6.24 **Adc__DisableWdgNotification()**

```
void Adc_DisableWdgNotification (  
    Adc_ChannelType ChannelId )
```

Disable notification of a channel that has watchdog functionality configured at initialization.

Disable notification of a channel that has watchdog functionality configured at initialization Use generated symbolic channel name defines (e.g. `AdcChannel_0_0`), because function assumes `ChannelId` to be in following format: Logical channel id on bits until position defined by `ADC_CHANNEL_SYMBOLIC_NAME_SHIFT_HW_UNIT_ID_` ← U16, and for the rest the Logical Unit Id

Parameters

in	<i>Adc_ChannelType</i>	Symbolic name of channel
----	------------------------	--------------------------

Returns

void.

6.1.6.25 `Adc_SetClockMode()`

```
Std_ReturnType Adc_SetClockMode (
    Adc_SelectPrescalerType Prescaler )
```

Set the ADC clock prescaler if available and modify the conversion timings.

This function sets the ADC clock prescaler (Analog clock frequency selector)

Parameters

in	<i>Prescaler</i>	Normal or Alternate mode.
----	------------------	---------------------------

Returns

`Std_ReturnType` Standard return type. `E_OK`: In case of successful settings. `E_NOT_OK`: In case of unsuccessful settings.

6.1.6.26 `Adc_CtuWriteTriggerEnableMask()`

```
Std_ReturnType Adc_CtuWriteTriggerEnableMask (
    uint8 Trigger,
    uint8 ThcrValue )
```

Function to set the trigger handler control registers of the CTU IPL.

Full configurability of CTU THCR1 & THCR2 registers of the CTU IPL. This function has in input the CTU trigger (0...7) and the mask (8 bit) to enable the External Trigger and Timer output for that Trigger, without considering the ADC command output enable. See from the RM the THCR1 and THCR2 registers for the right `Thcr_value` to use.

Module Documentation

Note

The function Service ID[hex]: 0x35.

Parameters

in	<i>Trigger</i>	Index of the trigger: 0...7.
in	<i>ThcrValue</i>	THCRx mask value (only with bits for Ext. Trigger and Timer) for the selected input trigger.

Returns

Std_ReturnType E_OK or E_NOT_OK. E_OK In case of successful settings. E_NOT_OK In case of unsuccessful settings.

6.1.6.27 Adc_CtuSetTriggerEnable()

```
Std_ReturnType Adc_CtuSetTriggerEnable (
    uint8 Trigger,
    boolean Enable )
```

Function to set or clear the bit Tx_E of the trigger handler control registers of the CTU IPL.

Full configurability of CTU THCR1 & THCR2 registers of the CTU IPL. This function has in input the CTU trigger (0...7) and a Boolean to set to clear the right trigger bit. See from the RM the THCR1 and THCR2 registers for the right API use.

Parameters

in	<i>Trigger</i>	Index of the trigger: 0...7.
in	<i>Enable</i>	if True, the bit Tx_E shall be set to 1, 0 otherwise

Note

The function Service ID[hex]: 0x36.

Returns

Std_ReturnType E_OK or E_NOT_OK. E_OK In case of successful settings. E_NOT_OK In case of unsuccessful settings.

6.1.6.28 `Adc_CtuSetTriggerAdcCmdAddress()`

```
Std_ReturnType Adc_CtuSetTriggerAdcCmdAddress (
    uint8 Trigger,
    uint8 ClcrValue )
```

Function to set the cmd list handler control registers of the CTU IPL.

Full configurability of CLCR1 & CLCR2 registers of the CTU IPL. This function has in input the CTU trigger (0...7) and the position of the first command in the command list. See from the RM the CLCR1 and CLCR2 registers for the right API use.

Parameters

in	<i>Trigger</i>	Index of the trigger: 0...7.
in	<i>ClcrValue</i>	Position of the first command

Note

The function Service ID[hex]: 0x37.

Returns

Std_ReturnType E_OK or E_NOT_OK. E_OK In case of successful settings. E_NOT_OK In case of unsuccessful settings.

6.1.6.29 `Adc_CtuSetTriggerCompare()`

```
Std_ReturnType Adc_CtuSetTriggerCompare (
    uint8 Trigger,
    uint16 CompareValue )
```

Function to set the trigger compare registers of the CTU IPL.

Full configurability of CTU TxCR registers of the CTU IPL. This function has in input the CTU trigger (0...7) and the comparator value See from the RM the TxCR trigger compare registers for the right compare_value to use.

Note

The function Service ID[hex]: 0x38.

Parameters

in	<i>Trigger</i>	Index of the trigger: 0...7.
in	<i>CompareValue</i>	value to set in the register TxCR.

Returns

Std_ReturnType E_OK or E_NOT_OK. E_OK In case of successful settings. E_NOT_OK In case of unsuccessful settings.

6.1.6.30 Adc_EnableCtuControlMode()

```
void Adc_EnableCtuControlMode (  
    Adc_HwUnitType Unit )
```

Function to enable CTU control mode for an ADC unit.

Enable CTU control mode for an ADC unit. This function to enable CTU control mode for Adc. When a unit works in CTU control mode, no other conversions shall run in parallel(Adc). The only conversions occurring shall be the ones defined in the CTU configuration.

Parameters

in	<i>Unit</i>	Adc unit used. Recommended to use generated define for Adc Logical Unit Id
----	-------------	--

Returns

void

6.1.6.31 Adc_DisableCtuControlMode()

```
void Adc_DisableCtuControlMode (  
    Adc_HwUnitType Unit )
```

Function to disable CTU control mode for an ADC unit.

Disable CTU control mode for an ADC unit. This function to disable CTU control mode for Adc. The other Adc conversions can run in software trigger normal mode, software trigger injected mode or hardware trigger mode.

Parameters

in	<i>Unit</i>	Adc unit used. Recommended to use generated define for Adc Logical Unit Id
----	-------------	--

Returns

void

6.1.6.32 Adc_CtuEnableHwTrigger()

```
void Adc_CtuEnableHwTrigger (
    Adc_CtuTrigSrcType TrigSource )
```

Function to enable CTU hardware trigger.

Enable CTU hardware trigger.

Parameters

in	<i>TrigSource</i>	Trigger source.
----	-------------------	-----------------

Returns

void

6.1.6.33 Adc_CtuDisableHwTrigger()

```
void Adc_CtuDisableHwTrigger (
    Adc_CtuTrigSrcType TrigSource )
```

Function to disable CTU hardware trigger.

Disable CTU hardware trigger.

Parameters

in	<i>TrigSource</i>	Trigger source.
----	-------------------	-----------------

Returns

void

6.1.6.34 Adc_CtuStartConversion()

```
void Adc_CtuStartConversion (
    Adc_CtuTrigSrcType TrigSource )
```

Function to start CTU conversion.

Start CTU conversion.

Parameters

in	<i>TrigSource</i>	Trigger source.
----	-------------------	-----------------

Returns

void

6.1.6.35 **Adc_CtuMasterReload()**

```
void Adc_CtuMasterReload (
    Adc_HwUnitType CtuUnit )
```

Issue a software master reload signal for CTU.

Issue a software master reload signal for CTU in CTU Control Mode.

Parameters

in	<i>CtuUnit</i>	CTU hardware unit. Recommended to use generated define for Ctu Logical Unit Id
----	----------------	--

Returns

void

6.1.6.36 **Adc_CtuReadFifoData()**

```
void Adc_CtuReadFifoData (
    Adc_CtuFifoIdxType FifoIdx,
    uint16 * Data,
    uint8 DataLength )
```

Function to read CTU conversion data from FIFO.

Read CTU conversion data from FIFO.

Parameters

in	<i>FifoIdx</i>	CTU FIFO index.
out	<i>Data</i>	Pointer to pre-allocated result array.
in	<i>DataLength</i>	Max amount of results to be read.

Returns

void

6.1.6.37 **Adc_CtuReadFifoResult()**

```
void Adc_CtuReadFifoResult (
    Adc_CtuFifoIdxType FifoIdx,
    Adc_CtuFifoResultType * Result,
    uint8 ResultLength )
```

Function to read CTU conversion results from FIFO.

Read CTU conversion results from FIFO.

Parameters

in	<i>FifoIdx</i>	CTU FIFO index.
out	<i>Result</i>	Pointer to pre-allocated result array.
in	<i>ResultLength</i>	Max amount of results to be read.

Returns

void

6.1.6.38 **Adc_CtuSetFifoWatermark()**

```
void Adc_CtuSetFifoWatermark (
    Adc_CtuFifoIdxType FifoIdx,
    uint8 Watermark )
```

Function to set CTU FIFO watermark.

Set CTU FIFO watermark.

Parameters

in	<i>FifoIdx</i>	FIFO index.
in	<i>Watermark</i>	Watermark value.

Returns

void

6.1.6.39 **Adc_CtuEnableNotification()**

```
void Adc_CtuEnableNotification (
    Adc_CtuNotificationType Notification )
```

Function to enable CTU notification.

Enable CTU notification.

Parameters

in	<i>Notification</i>	Notification to be enabled.
----	---------------------	-----------------------------

Returns

void

6.1.6.40 **Adc_CtuDisableNotification()**

```
void Adc_CtuDisableNotification (
    Adc_CtuNotificationType Notification )
```

Function to disable CTU notification.

Disable CTU notification.

Parameters

in	<i>Notification</i>	Notification to be disabled.
----	---------------------	------------------------------

Returns

void

6.1.6.41 Adc_CtuSetList()

```
void Adc_CtuSetList (
    const Adc_CtuListItemType *const ListItemsArray,
    const uint8 NumItems,
    const uint8 ListStartPosition )
```

Function to set CTU list.

Reconfigure the CTU list of conversions in CTU Control Mode.

Parameters

in	<i>ListItemsArray</i>	Pointer to list items array to be set.
in	<i>NumItems</i>	Number of items in the array.
in	<i>ListStartPosition</i>	Start position of the list.

Returns

void

6.1.6.42 Adc_CtuSetListPointer()

```
void Adc_CtuSetListPointer (
    Adc_CtuTrigSrcType TrigSource,
    Adc_CtuListPtrType ListPtr )
```

Reconfigure the list pointer of a triggered CTU source.

Reconfigure the list pointer of a trigger source in CTU Control Mode.

Parameters

in	<i>TrigSource</i>	Source trigger index.
in	<i>ListPtr</i>	Position of the first channel of the command list.

Returns

void

6.1.7 Variable Documentation

6.1.7.1 Adc_apxCfgPtr

```
const Adc\_ConfigType* Adc_apxCfgPtr[(1U)] [extern]
```

Used to point the configuration structure.

6.2 Adc Sar IPL

6.2.1 Detailed Description Adc Sar HW module.

Adc Sar IP layer hardware module.

Data Structures

- struct [Adc_Sar_Ip_ChanConfigType](#)
Defines the channel configuration. [More...](#)
- struct [Adc_Sar_Ip_ClockConfigType](#)
Defines the ADC clock configuration. [More...](#)
- struct [Adc_Sar_Ip_WdgThresholdType](#)
Defines the upper and lower thresholds for analog watchdog. [More...](#)
- struct [Adc_Sar_Ip_ChanResultType](#)
Defines the data regarding a conversion, beyond the conversion data. [More...](#)
- struct [Adc_Sar_Ip_ChansIdxMaskType](#)
Defines configuration of channels in a chain. [More...](#)
- struct [Adc_Sar_Ip_SelfTestThresholdType](#)
Defines configuration of self-test threshold values. [More...](#)
- struct [Adc_Sar_Ip_ConfigType](#)
Defines the module configuration. [More...](#)
- struct [Adc_Sar_Ip_StateStructType](#)
Structure used to store runtime info. [More...](#)

Macros

- `#define ADC\_SAR\_IP\_NOTIF\_FLAG\_NORMAL\_ENDCHAIN`
Macros for status and notification flags.
- `#define ADC\_SAR\_IP\_CHAN\_NOTIF\_EOC`
Macros for channel notifications.
- `#define ADC\_SAR\_IP\_WDG\_HIGH\_FLAG`
Macros for watchdog registers.
- `#define ADC\_SAR\_IP\_CONV\_CHAIN\_NORMAL`
Macros for conversion chain selection.
- `#define ADC\_SAR\_IP\_CONV\_CHAIN\_INJECTED`
- `#define ADC\_SAR\_IP\_CONV\_CHAIN\_CTU`

Types Reference

- typedef uint32 [Adc_Sar_Ip_ConvChainType](#)
Conversion chain selection.
- typedef void [Adc_Sar_Ip_WdgNotificationType](#)(const uint16 ChanIdx, const uint8 Flags)
Defines the watchdog notification header.

Enum Reference

- enum [Adc_Sar_Ip_StatusType](#)
ADC_SAR status return type.
- enum [Adc_Sar_Ip_ConvModeType](#)
Conversion mode selection (One-shot or Scan)
- enum [Adc_Sar_Ip_ClockSelType](#)
Converter input clock.
- enum [Adc_Sar_Ip_ExtTriggerEdgeType](#)
External Trigger selection.
- enum [Adc_Sar_Ip_ExtTriggerSourceType](#)
External Trigger Source Enable.
- enum [Adc_Sar_Ip_DataAlignedType](#)
Data alignment selection.
- enum [Adc_Sar_Ip_ClearSourceType](#)
Clear DMA source.
- enum [Adc_Sar_Ip_PresamplingSourceType](#)
Presampling Voltage selection.
- enum [Adc_Sar_Ip_ChanGroupType](#)
Channel group selection.

Function Reference

- uint32 [Adc_Sar_Ip_GetDataAddress](#) (uint32 u32Instance, uint32 u32ChannelIndex)
Return the address of the specified data register.
- [Adc_Sar_Ip_StatusType](#) [Adc_Sar_Ip_Init](#) (const uint32 u32Instance, const [Adc_Sar_Ip_ConfigType](#) *const pConfig)
Initialize ADC_SAR module.
- [Adc_Sar_Ip_StatusType](#) [Adc_Sar_Ip_Deinit](#) (const uint32 u32Instance)
Deinitialize ADC_SAR module.
- void [Adc_Sar_Ip_ChainConfig](#) (const uint32 u32Instance, const [Adc_Sar_Ip_ChansIdxMaskType](#) *const pChansIdxMask, const [Adc_Sar_Ip_ConvChainType](#) pChainType)
Configures the converter chains the given configuration structure.
- void [Adc_Sar_Ip_EnableChannel](#) (const uint32 u32Instance, const [Adc_Sar_Ip_ConvChainType](#) pChainType, const uint32 u32ChnIdx)
Enable a channel.
- void [Adc_Sar_Ip_DisableChannel](#) (const uint32 u32Instance, const [Adc_Sar_Ip_ConvChainType](#) pChainType, const uint32 u32ChnIdx)
Disable a channel.
- void [Adc_Sar_Ip_StartConversion](#) (const uint32 u32Instance, const [Adc_Sar_Ip_ConvChainType](#) pChainType)
Start conversion.
- uint32 [Adc_Sar_Ip_GetStatusFlags](#) (const uint32 u32Instance)
Get the status flags.
- void [Adc_Sar_Ip_ClearStatusFlags](#) (const uint32 u32Instance, const uint32 u32Mask)
Clear the status flags.

- [Adc_Sar_Ip_StatusType Adc_Sar_Ip_SelfTest](#) (const uint32 u32Instance)
Self test.
- uint32 [Adc_Sar_Ip_GetConvDataToArray](#) (const uint32 u32Instance, const [Adc_Sar_Ip_ConvChainType](#) pChainType, const uint32 u32Length, uint16 *const pResults)
Get conversion results for a conversion chain.
- uint32 [Adc_Sar_Ip_GetConvResultsToArray](#) (const uint32 u32Instance, const [Adc_Sar_Ip_ConvChainType](#) pChainType, const uint32 u32Length, [Adc_Sar_Ip_ChanResultType](#) *const pResults)
Get conversion results for a conversion chain with extended information.
- uint16 [Adc_Sar_Ip_GetConvData](#) (const uint32 u32Instance, const uint32 u32ChnIdx)
Return the result of the conversion.
- void [Adc_Sar_Ip_GetConvResult](#) (const uint32 u32Instance, const uint32 u32ChnIdx, const [Adc_Sar_Ip_ConvChainType](#) pChainType, [Adc_Sar_Ip_ChanResultType](#) *const pResult)
Return the result and the status of the conversion.
- [Adc_Sar_Ip_StatusType Adc_Sar_Ip_DoCalibration](#) (const uint32 u32Instance)
Perform Calibration of the ADC.
- [Adc_Sar_Ip_StatusType Adc_Sar_Ip_Powerup](#) (const uint32 u32Instance)
Power up the ADC.
- [Adc_Sar_Ip_StatusType Adc_Sar_Ip_Powerdown](#) (const uint32 u32Instance)
Power down the ADC.
- void [Adc_Sar_Ip_EnableNotifications](#) (const uint32 u32Instance, const uint32 u32NotificationMask)
Enable ADC interrupts.
- void [Adc_Sar_Ip_DisableNotifications](#) (const uint32 u32Instance, const uint32 u32NotificationMask)
Disable ADC interrupts.
- void [Adc_Sar_Ip_EnableChannelNotifications](#) (const uint32 u32Instance, const uint32 u32ChnIdx, const uint32 u32Mask)
Enable ADC interrupt for a channel.
- void [Adc_Sar_Ip_DisableChannelNotifications](#) (const uint32 u32Instance, const uint32 u32ChnIdx, const uint32 u32Mask)
Disable ADC interrupt for a channel.
- [Adc_Sar_Ip_StatusType Adc_Sar_Ip_SetClockMode](#) (const uint32 u32Instance, const [Adc_Sar_Ip_ClockConfigType](#) *const pConfig)
Set the ADC clocks.
- void [Adc_Sar_Ip_SetSampleTimes](#) (const uint32 u32Instance, const uint8 *const aSampleTimes)
Set the sample times.
- void [Adc_Sar_Ip_AbortConversion](#) (const uint32 u32Instance)
Abort ongoing conversion.
- [Adc_Sar_Ip_StatusType Adc_Sar_Ip_AbortChain](#) (const uint32 u32Instance, const boolean bBlocking, const boolean bAllowRestart)
Abort ongoing chain conversion.
- void [Adc_Sar_Ip_SetPresamplingSource](#) (const uint32 u32Instance, const [Adc_Sar_Ip_ChanGroupType](#) pChnGroup, const [Adc_Sar_Ip_PresamplingSourceType](#) pPresampleSource)
Set the Presampling Source for the channel group.
- void [Adc_Sar_Ip_EnableChannelPresampling](#) (const uint32 u32Instance, const uint32 u32ChnIdx)
Enable Presampling on one channel.
- void [Adc_Sar_Ip_DisableChannelPresampling](#) (const uint32 u32Instance, const uint32 u32ChnIdx)
Disable Presampling on one channel.
- void [Adc_Sar_Ip_EnablePresampleConversion](#) (const uint32 u32Instance)
Enable Conversion Presampled Data.

- void [Adc_Sar_Ip_DisablePresampleConversion](#) (const uint32 u32Instance)
Disable Conversion of Presampled Data.
- void [Adc_Sar_Ip_EnableDma](#) (const uint32 u32Instance)
Enable DMA Requests.
- void [Adc_Sar_Ip_DisableDma](#) (const uint32 u32Instance)
Disable DMA Requests.
- void [Adc_Sar_Ip_EnableChannelDma](#) (const uint32 u32Instance, const uint32 u32ChnIdx)
Enable DMA on one channel.
- void [Adc_Sar_Ip_DisableChannelDma](#) (const uint32 u32Instance, const uint32 u32ChnIdx)
Disable DMA on one channel.
- void [Adc_Sar_Ip_DisableChannelDmaAll](#) (const uint32 u32Instance)
Disable DMA on all channels.
- void [Adc_Sar_Ip_SetDmaClearSource](#) (const uint32 u32Instance, const [Adc_Sar_Ip_ClearSourceType](#) p←DmaClear)
Set DMA Request Clear Source.
- void [Adc_Sar_Ip_SetWdgThreshold](#) (const uint32 u32Instance, const uint8 u8RegisterIdx, const [Adc_Sar_Ip_WdgThresholdType](#) *const pThresholdValues)
Configure watchdog threshold register.
- void [Adc_Sar_Ip_SetConversionMode](#) (const uint32 u32Instance, const [Adc_Sar_Ip_ConvModeType](#) e←ConvMode)
Configure conversion mode.
- void [Adc_Sar_Ip_SetExternalTrigger](#) (const uint32 u32Instance, const [Adc_Sar_Ip_ExtTriggerEdgeType](#) eTriggerEdge, const [Adc_Sar_Ip_ExtTriggerSourceType](#) eTrggerSrc)
Configure external trigger.

6.2.2 Data Structure Documentation

6.2.2.1 struct [Adc_Sar_Ip_ChanConfigType](#)

Defines the channel configuration.

This structure is used to configure channels

Implements : [Adc_Sar_Ip_ChanConfigType_Class](#)

Definition at line 330 of file [Adc_Sar_Ip_Types.h](#).

6.2.2.2 struct [Adc_Sar_Ip_ClockConfigType](#)

Defines the ADC clock configuration.

This structure is used to the ADC_SAR clock

Implements : [Adc_Sar_Ip_ClockConfigType_Class](#)

Definition at line 351 of file [Adc_Sar_Ip_Types.h](#).

Data Fields

Type	Name	Description
Adc_Sar_Ip_ClockSelType	ClkSelect	Selected clock
uint8	SampleTimeArr[ADC_SAR_IP_NUM_CHANNELS]	Sample time for each channel group
uint8	PowerDownDelay	Delay before entering Power Down

6.2.2.3 struct Adc_Sar_Ip_WdgThresholdType

Defines the upper and lower thresholds for analog watchdog.

This structure is used to configure the analog watchdog threshold registers.

Implements : `Adc_Sar_Ip_WdgThresholdType_Class`

Definition at line 373 of file `Adc_Sar_Ip_Types.h`.

Data Fields

Type	Name	Description
uint8	WdgIndex	Watchdog threshold register index
uint16	LowThreshold	Lower threshold
uint16	HighThreshold	Upper threshold
boolean	LowThresholdIntEn	Enable interrupt when lower threshold exceeded
boolean	HighThresholdIntEn	Enable interrupt when upper threshold exceeded

6.2.2.4 struct Adc_Sar_Ip_ChanResultType

Defines the data regarding a conversion, beyond the conversion data.

This structure is used to return information about conversions beyond just conversion data

Implements : `Adc_Sar_Ip_ChanResultType_Class`

Definition at line 390 of file `Adc_Sar_Ip_Types.h`.

Data Fields

Type	Name	Description
uint8	AdcChnIdx	ADC Channel Index
boolean	ValidFlag	Data Valid Flag
boolean	OverWrittenFlag	Data Overwritten Flag
uint16	ConvData	Conversion Data

6.2.2.5 struct `Adc_Sar_Ip_ChansIdxMaskType`

Defines configuration of channels in a chain.

This structure is used to configure channels in chain.

Implements : `Adc_Sar_Ip_ChansIdxMaskType_Class`

Definition at line 404 of file `Adc_Sar_Ip_Types.h`.

Data Fields

Type	Name	Description
uint32	<code>ChanMaskArr[ADC_SAR_IP_NUM_GROUP_CHAN]</code>	Bit-mask used to configure channels in chain

6.2.2.6 struct `Adc_Sar_Ip_SelfTestThresholdType`

Defines configuration of self-test threshold values.

This structure is used to configure self-test threshold values.

Implements : `Adc_Sar_Ip_SelfTestThresholdType_Class`

Definition at line 417 of file `Adc_Sar_Ip_Types.h`.

6.2.2.7 struct `Adc_Sar_Ip_ConfigType`

Defines the module configuration.

This structure is used to configure the ADC_SAR module

Implements : `Adc_Sar_Ip_ConfigType_Class`

Definition at line 448 of file `Adc_Sar_Ip_Types.h`.

Data Fields

- `Adc_Sar_Ip_ConvModeType` `ConvMode`
- `Adc_Sar_Ip_ClockSelType` `ClkSelect`
- `Adc_Sar_Ip_ClockSelType` `CalibrationClkSelect`
- `Adc_Sar_Ip_ExtTriggerEdgeType` `InjectedEdge`
- `Adc_Sar_Ip_ExtTriggerEdgeType` `ExtTrigger`
- boolean `NormalExtTrgEn`
- uint8 `SampleTimeArr [ADC_SAR_IP_NUM_GROUP_CHAN]`
- `Adc_Sar_Ip_PresamplingSourceType` `PresamplingSourceArr [ADC_SAR_IP_NUM_GROUP_CHAN]`
- boolean `AutoClockOff`
- boolean `OverwriteEnable`
- `Adc_Sar_Ip_DataAlignedType` `DataAlign`
- uint8 `PowerDownDelay`
- const `Adc_Sar_Ip_SelfTestThresholdType * SelfTestThresholdConfig`
- uint32 `ChanMaskNormal [ADC_SAR_IP_NUM_GROUP_CHAN]`
- uint32 `ChanMaskInjected [ADC_SAR_IP_NUM_GROUP_CHAN]`

6.2.2.7.1 Field Documentation

6.2.2.7.1.1 ConvMode [Adc_Sar_Ip_ConvModeType](#) ConvMode

Conversion Mode (One-shot or Scan)

Definition at line 450 of file `Adc_Sar_Ip_Types.h`.

6.2.2.7.1.2 ClkSelect [Adc_Sar_Ip_ClockSelType](#) ClkSelect

Clock input

Definition at line 455 of file `Adc_Sar_Ip_Types.h`.

6.2.2.7.1.3 CalibrationClkSelect [Adc_Sar_Ip_ClockSelType](#) CalibrationClkSelect

Clock input for calibration

Definition at line 456 of file `Adc_Sar_Ip_Types.h`.

6.2.2.7.1.4 InjectedEdge [Adc_Sar_Ip_ExtTriggerEdgeType](#) InjectedEdge

Injected Trigger selection

Definition at line 464 of file `Adc_Sar_Ip_Types.h`.

6.2.2.7.1.5 ExtTrigger [Adc_Sar_Ip_ExtTriggerEdgeType](#) ExtTrigger

External Trigger selection

Definition at line 467 of file `Adc_Sar_Ip_Types.h`.

6.2.2.7.1.6 NormalExtTrgEn [boolean](#) NormalExtTrgEn

Enables normal trigger source

Definition at line 468 of file `Adc_Sar_Ip_Types.h`.

6.2.2.7.1.7 **SampleTimeArr** `uint8 SampleTimeArr[ADC_SAR_IP_NUM_GROUP_CHAN]`

Sample time for each channel group

Definition at line 473 of file `Adc_Sar_Ip_Types.h`.

6.2.2.7.1.8 **PresamplingSourceArr** `Adc_Sar_Ip_PresamplingSourceType PresamplingSourceArr[ADC_SAR_IP_NUM_GROUP_CHAN]`

Presampling sources for each channel group

Definition at line 475 of file `Adc_Sar_Ip_Types.h`.

6.2.2.7.1.9 **AutoClockOff** `boolean AutoClockOff`

Enable Auto Clock Off

Definition at line 476 of file `Adc_Sar_Ip_Types.h`.

6.2.2.7.1.10 **OverwriteEnable** `boolean OverwriteEnable`

Overwrite new conversion data over old data

Definition at line 477 of file `Adc_Sar_Ip_Types.h`.

6.2.2.7.1.11 **DataAlign** `Adc_Sar_Ip_DataAlignedType DataAlign`

Data alignment in conversion result register

Definition at line 478 of file `Adc_Sar_Ip_Types.h`.

6.2.2.7.1.12 **PowerDownDelay** `uint8 PowerDownDelay`

Delay before entering Power Down

Definition at line 482 of file `Adc_Sar_Ip_Types.h`.

6.2.2.7.1.13 SelfTestThresholdConfig `const Adc\_Sar\_Ip\_SelfTestThresholdType* SelfTestThresholdConfig`

Self test threshold configuration

Definition at line 484 of file `Adc_Sar_Ip_Types.h`.

6.2.2.7.1.14 ChanMaskNormal `uint32 ChanMaskNormal[ADC_SAR_IP_NUM_GROUP_CHAN]`

Bit-mask used to configure Normal Chain

Definition at line 495 of file `Adc_Sar_Ip_Types.h`.

6.2.2.7.1.15 ChanMaskInjected `uint32 ChanMaskInjected[ADC_SAR_IP_NUM_GROUP_CHAN]`

Bit-mask used to configure Injected Chain

Definition at line 496 of file `Adc_Sar_Ip_Types.h`.

6.2.2.8 struct Adc_Sar_Ip_StateStructType

Structure used to store runtime info.

This structure is used to store ADC SAR runtime info

Implements : `Adc_Sar_Ip_StateStructType_Class`

Definition at line 534 of file `Adc_Sar_Ip_Types.h`.

Data Fields

- boolean [InitStatus](#)
- [Adc_Sar_Ip_DataAlignedType](#) [DataAlign](#)
- [Adc_Sar_Ip_ClockSelType](#) [CalibrationClkSelect](#)
- uint32 [ChanWdgThresholdIndex](#) [ADC_SAR_IP_CDR_COUNT]

6.2.2.8.1 Field Documentation**6.2.2.8.1.1 InitStatus** `boolean InitStatus`

Check if the driver was initialized.

Definition at line 536 of file `Adc_Sar_Ip_Types.h`.

6.2.2.8.1.2 DataAlign `Adc_Sar_Ip_DataAlignedType` DataAlign

Data alignment in conversion result register

Definition at line 537 of file `Adc_Sar_Ip_Types.h`.

6.2.2.8.1.3 CalibrationClkSelect `Adc_Sar_Ip_ClockSelType` CalibrationClkSelect

Clock input for calibration

Definition at line 538 of file `Adc_Sar_Ip_Types.h`.

6.2.2.8.1.4 ChanWdgThresholdIndex `uint32 ChanWdgThresholdIndex[ADC_SAR_IP_CDR_COUNT]`

Channel index to threshold index mapping

Definition at line 543 of file `Adc_Sar_Ip_Types.h`.

6.2.3 Macro Definition Documentation

6.2.3.1 ADC_SAR_IP_NOTIF_FLAG_NORMAL_ENDCHAIN

```
#define ADC_SAR_IP_NOTIF_FLAG_NORMAL_ENDCHAIN
```

Macros for status and notification flags.

These flags map to internal hardware flags in different registers, but are grouped together for convenience.

Definition at line 113 of file `Adc_Sar_Ip.h`.

6.2.3.2 ADC_SAR_IP_CHAN_NOTIF_EOC

```
#define ADC_SAR_IP_CHAN_NOTIF_EOC
```

Macros for channel notifications.

These notification flags map to internal hardware flags in different registers, but are grouped together for convenience.

Definition at line 157 of file `Adc_Sar_Ip.h`.

6.2.3.3 ADC_SAR_IP_WDG_HIGH_FLAG

```
#define ADC_SAR_IP_WDG_HIGH_FLAG
```

Macros for watchdog registers.

These macros help decode and compose bit mask for watchdog functionality

Definition at line 171 of file `Adc_Sar_Ip.h`.

6.2.3.4 ADC_SAR_IP_CONV_CHAIN_NORMAL

```
#define ADC_SAR_IP_CONV_CHAIN_NORMAL
```

Macros for conversion chain selection.

These macros are used to configure type of the conversion (`Adc_Sar_Ip_ConvChainType`) Selects the "Normal" Conversion Chain

Definition at line 99 of file `Adc_Sar_Ip_Types.h`.

6.2.3.5 ADC_SAR_IP_CONV_CHAIN_INJECTED

```
#define ADC_SAR_IP_CONV_CHAIN_INJECTED
```

Selects the "Injected" Conversion Chain

Definition at line 100 of file `Adc_Sar_Ip_Types.h`.

6.2.3.6 ADC_SAR_IP_CONV_CHAIN_CTU

```
#define ADC_SAR_IP_CONV_CHAIN_CTU
```

Selects the "CTU" Conversion Chain

Definition at line 101 of file `Adc_Sar_Ip_Types.h`.

6.2.4 Types Reference

6.2.4.1 Adc_Sar_Ip_ConvChainType

```
typedef uint32 Adc_Sar_Ip_ConvChainType
```

Conversion chain selection.

This type definition is used to configure type of the conversion

Implements : Adc_Sar_Ip_ConvChainType_Class

Definition at line 209 of file Adc_Sar_Ip_Types.h.

6.2.4.2 Adc_Sar_Ip_WdgNotificationType

```
typedef void Adc_Sar_Ip_WdgNotificationType(const uint16 ChanIdx, const uint8 Flags)
```

Defines the watchdog notification header.

This header is used for out of range watchdog notification callbacks u8Flags will contain which WDG flag was triggered(low and/or high). These are defined by ADC_SAR_IP_WDG_LOW_FLAG and ADC_SAR_IP_WDG_HIGH_FLAG

Implements : Adc_Sar_Ip_WdgChanNotificationType_Class

Definition at line 320 of file Adc_Sar_Ip_Types.h.

6.2.5 Enum Reference

6.2.5.1 Adc_Sar_Ip_StatusType

```
enum Adc_Sar_Ip_StatusType
```

ADC_SAR status return type.

This enum is used as return type

Implements : Adc_Sar_Ip_StatusType_Class

Enumerator

ADC_SAR_IP_STATUS_SUCCESS	Function completed successfully
ADC_SAR_IP_STATUS_ERROR	Function didn't complete successfully
ADC_SAR_IP_STATUS_TIMEOUT	Function timed out

Definition at line 114 of file Adc_Sar_Ip_Types.h.

6.2.5.2 Adc_Sar_Ip_ConvModeType

enum `Adc_Sar_Ip_ConvModeType`

Conversion mode selection (One-shot or Scan)

This enum is used to configure the conversion mode

Implements : `Adc_Sar_Ip_ConvModeType_Class`

Enumerator

<code>ADC_SAR_IP_CONV_MODE_ONESHOT</code>	One-shot conversion mode
<code>ADC_SAR_IP_CONV_MODE_SCAN</code>	Scan conversion mode

Definition at line 128 of file `Adc_Sar_Ip_Types.h`.

6.2.5.3 Adc_Sar_Ip_ClockSelType

enum `Adc_Sar_Ip_ClockSelType`

Converter input clock.

This enum is used to configure the converter input clock

Implements : `Adc_Sar_Ip_ClockSelType_Class`

Enumerator

<code>ADC_SAR_IP_CLK_FULL_BUS</code>	Adc module clock
<code>ADC_SAR_IP_CLK_HALF_BUS</code>	Adc module clock/2

Definition at line 141 of file `Adc_Sar_Ip_Types.h`.

6.2.5.4 Adc_Sar_Ip_ExtTriggerEdgeType

enum `Adc_Sar_Ip_ExtTriggerEdgeType`

Module Documentation

External Trigger selection.

This enum is used to configure the external trigger

Implements : `Adc_Sar_Ip_ExtTriggerEdgeType_Class`

Enumerator

ADC_SAR_IP_EXT_TRIG_EDGE_DISABLED	Injected trigger disabled
ADC_SAR_IP_EXT_TRIG_EDGE_FALLING	Injected trigger on Falling Edge
ADC_SAR_IP_EXT_TRIG_EDGE_RISING	Injected trigger on Rising Edge

Definition at line 174 of file Adc_Sar_Ip_Types.h.

6.2.5.5 Adc_Sar_Ip_ExtTriggerSourceType

```
enum Adc_Sar_Ip_ExtTriggerSourceType
```

External Trigger Source Enable.

This enum is used to enable the external trigger source to start the conversion.

Implements : Adc_Sar_Ip_ExtTriggerSourceType_Class

Enumerator

ADC_SAR_IP_NORMAL_EXT_TRIG	Enables normal trigger input
ADC_SAR_IP_INJECTED_EXT_TRIG	Enables injection trigger input

Definition at line 188 of file Adc_Sar_Ip_Types.h.

6.2.5.6 Adc_Sar_Ip_DataAlignedType

```
enum Adc_Sar_Ip_DataAlignedType
```

Data alignment selection.

This enum is used to configure data alignment

Implements : Adc_Sar_Ip_DataAlignedType_Class

Enumerator

ADC_SAR_IP_DATA_ALIGNED_RIGHT	Measured data is right-aligned
ADC_SAR_IP_DATA_ALIGNED_LEFT	Measured data is left-aligned

Definition at line 218 of file Adc_Sar_Ip_Types.h.

6.2.5.7 Adc_Sar_Ip_ClearSourceType

enum `Adc_Sar_Ip_ClearSourceType`

Clear DMA source.

This enum is used to configure source used to clear a DMA request

Implements : `Adc_Sar_Ip_ClearSourceType_Class`

Enumerator

<code>ADC_SAR_IP_DMA_REQ_CLEAR_ON_ACK</code>	Clear DMA Request on Ack from DMA Controller
<code>ADC_SAR_IP_DMA_REQ_CLEAR_ON_READ</code>	Clear DMA Request on read of Data Registers

Definition at line 231 of file `Adc_Sar_Ip_Types.h`.

6.2.5.8 Adc_Sar_Ip_PresamplingSourceType

enum `Adc_Sar_Ip_PresamplingSourceType`

Presampling Voltage selection.

This enum is used to configure the presampling voltage

Implements : `Adc_Sar_Ip_PresamplingSourceType_Class`

Enumerator

<code>ADC_SAR_IP_PRESAMPLE_VREFL</code>	Presampling from VREFL
<code>ADC_SAR_IP_PRESAMPLE_VREFH</code>	Presampling from VREFH

Definition at line 243 of file `Adc_Sar_Ip_Types.h`.

6.2.5.9 Adc_Sar_Ip_ChGroupType

enum `Adc_Sar_Ip_ChGroupType`

Channel group selection.

This enum is used to select the group of ADC channels

Implements : `Adc_Sar_Ip_ChGroupType_Class`

Enumerator

ADC_SAR_IP_CHAN_GROUP↵ _0	Channels Group (0-31)
ADC_SAR_IP_CHAN_GROUP↵ _1	Channels Group (32-63)

Definition at line 259 of file `Adc_Sar_Ip_Types.h`.

6.2.6 Function Reference

6.2.6.1 Adc_Sar_Ip_GetDataAddress()

```
uint32 Adc_Sar_Ip_GetDataAddress (
    uint32 u32Instance,
    uint32 u32ChannelIndex )
```

Return the address of the specified data register.

This function returns the address of the specified data register

Parameters

in	<i>u32Instance</i>	- ADC instance number
in	<i>u32ChannelIndex</i>	- adc channel of the Hw unit

Returns

status:

- value of the address of the data for the specified channel

6.2.6.2 Adc_Sar_Ip_Init()

```
Adc_Sar_Ip_StatusType Adc_Sar_Ip_Init (
    const uint32 u32Instance,
    const Adc_Sar_Ip_ConfigType *const pConfig )
```

Initialize ADC_SAR module.

This function initializes the ADC_SAR module by configuring all available features.

Parameters

in	<i>u32Instance</i>	- ADC instance number
in	<i>pConfig</i>	- configuration struct pointer

Returns

status:

- ADC_SAR_IP_STATUS_SUCCESS: init successful
- ADC_SAR_IP_STATUS_TIMEOUT: init step timed out

6.2.6.3 Adc_Sar_Ip_Deinit()

```
Adc_Sar_Ip_StatusType Adc_Sar_Ip_Deinit (  
    const uint32 u32Instance )
```

Deinitialize ADC_SAR module.

This function resets the ADC internal registers to default values.

Parameters

in	<i>u32Instance</i>	- ADC instance number
----	--------------------	-----------------------

Returns

status:

- ADC_SAR_IP_STATUS_SUCCESS: deinit successful
- ADC_SAR_IP_STATUS_TIMEOUT: deinit step timed out

6.2.6.4 Adc_Sar_Ip_ChainConfig()

```
void Adc_Sar_Ip_ChainConfig (  
    const uint32 u32Instance,  
    const Adc_Sar_Ip_ChansIdxMaskType *const pChansIdxMask,  
    const Adc_Sar_Ip_ConvChainType pChainType )
```

Configures the converter chains the given configuration structure.

This function configures the ADC Normal and Injected Chains with the options provided in the structure.

Parameters

in	<i>u32Instance</i>	- ADC instance number
in	<i>pChansIdxMask</i>	- channel configuration structure
in	<i>pChainType</i>	- conversion chain (Normal or Injected)

Returns

void

6.2.6.5 Adc_Sar_Ip_EnableChannel()

```
void Adc_Sar_Ip_EnableChannel (
    const uint32 u32Instance,
    const Adc_Sar_Ip_ConvChainType pChainType,
    const uint32 u32ChnIdx )
```

Enable a channel.

This function enables a channel in a specified conversion chain

Parameters

in	<i>u32Instance</i>	- ADC instance number
in	<i>pChainType</i>	- conversion chain (Normal or Injected)
in	<i>u32ChnIdx</i>	- channel 0

Returns

void

6.2.6.6 Adc_Sar_Ip_DisableChannel()

```
void Adc_Sar_Ip_DisableChannel (
    const uint32 u32Instance,
    const Adc_Sar_Ip_ConvChainType pChainType,
    const uint32 u32ChnIdx )
```

Disable a channel.

This function disables a channel in a specified conversion chain

Parameters

in	<i>u32Instance</i>	- ADC instance number
in	<i>pChainType</i>	- conversion chain (Normal or Injected)
in	<i>u32ChnIdx</i>	- channel 0

Returns

void

6.2.6.7 Adc_Sar_Ip_StartConversion()

```
void Adc_Sar_Ip_StartConversion (
    const uint32 u32Instance,
    const Adc_Sar_Ip_ConvChainType pChainType )
```

Start conversion.

This function starts a conversion channel (Normal or Injected)

Parameters

in	<i>u32Instance</i>	- ADC instance number
in	<i>pChainType</i>	- conversion chain (Normal or Injected)

Returns

void

6.2.6.8 Adc_Sar_Ip_GetStatusFlags()

```
uint32 Adc_Sar_Ip_GetStatusFlags (
    const uint32 u32Instance )
```

Get the status flags.

This function returns the status flags of the ADC.

Parameters

in	<i>u32Instance</i>	- ADC instance number
----	--------------------	-----------------------

Returns

the status flag bit-mask

6.2.6.9 `Adc_Sar_Ip_ClearStatusFlags()`

```
void Adc_Sar_Ip_ClearStatusFlags (
    const uint32 u32Instance,
    const uint32 u32Mask )
```

Clear the status flags.

This function clears the status flags of the ADC.

Parameters

in	<i>u32Instance</i>	- ADC instance number
in	<i>u32Mask</i>	- bit-mask of flags to clear

Returns

void

6.2.6.10 `Adc_Sar_Ip_SelfTest()`

```
Adc_Sar_Ip_StatusType Adc_Sar_Ip_SelfTest (
    const uint32 u32Instance )
```

Self test.

This function executes a self test on the ADC instance.

Parameters

in	<i>u32Instance</i>	- ADC instance number
----	--------------------	-----------------------

Returns

status:

- `ADC_SAR_IP_STATUS_SUCCESS`: self testing successful
- `ADC_SAR_IP_STATUS_TIMEOUT`: self testing step timed out

6.2.6.11 Adc_Sar_Ip_GetConvDataToArray()

```
uint32 Adc_Sar_Ip_GetConvDataToArray (
    const uint32 u32Instance,
    const Adc_Sar_Ip_ConvChainType pChainType,
    const uint32 u32Length,
    uint16 *const pResults )
```

Get conversion results for a conversion chain.

This function gets the conversion results for the selected Conversion Chain.

Parameters

in	<i>u32Instance</i>	- ADC instance number
in	<i>pChainType</i>	- conversion chain (Normal, Injected or CTU)
in	<i>u32Length</i>	- the length of the buffer
out	<i>pResults</i>	- the output buffer

Returns

the number of values written in the buffer (max length)

6.2.6.12 Adc_Sar_Ip_GetConvResultsToArray()

```
uint32 Adc_Sar_Ip_GetConvResultsToArray (
    const uint32 u32Instance,
    const Adc_Sar_Ip_ConvChainType pChainType,
    const uint32 u32Length,
    Adc_Sar_Ip_ChanResultType *const pResults )
```

Get conversion results for a conversion chain with extended information.

This function gets the conversion results for the selected Conversion Chain, with extended information about each conversion result (channel index, valid an overwritten properties and conversion data). This function should be used in case of configurations with overlapping channel lists in different chains, resulting in overwrite of conversion data when a higher priority chain is executed before all data was read.

Parameters

in	<i>u32Instance</i>	- ADC instance number
in	<i>pChainType</i>	- conversion chain (Normal, Injected or CTU)
in	<i>u32Length</i>	- the length of the buffer
out	<i>pResults</i>	- the output buffer

Returns

the number of values written in the buffer (max length)

6.2.6.13 `Adc_Sar_Ip_GetConvData()`

```
uint16 Adc_Sar_Ip_GetConvData (
    const uint32 u32Instance,
    const uint32 u32ChnIdx )
```

Return the result of the conversion.

This function returns the result of the conversion for a single channel

Parameters

in	<i>u32Instance</i>	- ADC instance number
in	<i>u32ChnIdx</i>	- channel 0

Returns

conversion data

6.2.6.14 `Adc_Sar_Ip_GetConvResult()`

```
void Adc_Sar_Ip_GetConvResult (
    const uint32 u32Instance,
    const uint32 u32ChnIdx,
    const Adc_Sar_Ip_ConvChainType pChainType,
    Adc_Sar_Ip_ChanResultType *const pResult )
```

Return the result and the status of the conversion.

This function returns the result and the status of the conversion for a single channel

Parameters

in	<i>u32Instance</i>	- ADC instance number
in	<i>u32ChnIdx</i>	- channel 0
in	<i>pChainType</i>	- conversion chain (Normal, Injected or CTU)
out	<i>pResult</i>	- pointer to the buffer where the result will be written

Returns

void

6.2.6.15 `Adc_Sar_Ip_DoCalibration()`

```
Adc_Sar_Ip_StatusType Adc_Sar_Ip_DoCalibration (
    const uint32 u32Instance )
```

Perform Calibration of the ADC.

This function performs a calibration of the ADC. The maximum input clock frequency for the ADC is 80 MHz, checked with assertions if `DEV_ASSERT` is enabled. After calibration, the ADC is left in Powerup state (PWDN bit is clear).

Parameters

in	<i>u32Instance</i>	- ADC instance number
----	--------------------	-----------------------

Returns

the calibration result

- `ADC_SAR_IP_STATUS_SUCCESS`: calibration successful
- `ADC_SAR_IP_STATUS_TIMEOUT`: calibration step timed out
- `ADC_SAR_IP_STATUS_ERROR`: calibration failed

6.2.6.16 `Adc_Sar_Ip_Powerup()`

```
Adc_Sar_Ip_StatusType Adc_Sar_Ip_Powerup (
    const uint32 u32Instance )
```

Power up the ADC.

This function enables the ADC (disables the Power Down feature).

Parameters

in	<i>u32Instance</i>	- ADC instance number
----	--------------------	-----------------------

Returns

status:

- ADC_SAR_IP_STATUS_SUCCESS: power up successful
- ADC_SAR_IP_STATUS_TIMEOUT: power up failed

6.2.6.17 Adc_Sar_Ip_Powerdown()

```
Adc_Sar_Ip_StatusType Adc_Sar_Ip_Powerdown (
    const uint32 u32Instance )
```

Power down the ADC.

This function disables the ADC (enables the Power Down feature).

Parameters

in	<i>u32Instance</i>	- ADC instance number
----	--------------------	-----------------------

Returns

status:

- ADC_SAR_IP_STATUS_SUCCESS: power down successful
- ADC_SAR_IP_STATUS_TIMEOUT: power down failed

6.2.6.18 Adc_Sar_Ip_EnableNotifications()

```
void Adc_Sar_Ip_EnableNotifications (
    const uint32 u32Instance,
    const uint32 u32NotificationMask )
```

Enable ADC interrupts.

This function enables ADC interrupts.

Parameters

in	<i>u32Instance</i>	- ADC instance number
in	<i>u32NotificationMask</i>	- mask of interrupts to enable (of status flags)

Returns

void

6.2.6.19 Adc_Sar_Ip_DisableNotifications()

```
void Adc_Sar_Ip_DisableNotifications (
    const uint32 u32Instance,
    const uint32 u32NotificationMask )
```

Disable ADC interrupts.

This function disables ADC interrupts.

Parameters

in	<i>u32Instance</i>	- ADC instance number
in	<i>u32NotificationMask</i>	- mask of interrupts to disable (of status flags)

Returns

void

6.2.6.20 Adc_Sar_Ip_EnableChannelNotifications()

```
void Adc_Sar_Ip_EnableChannelNotifications (
    const uint32 u32Instance,
    const uint32 u32ChnIdx,
    const uint32 u32Mask )
```

Enable ADC interrupt for a channel.

This function enables interrupt generation on End of Conversion and watchdog events for a single channel. The mask parameter can be set using the defines provided in the driver header file that have the pattern ADC_SAR_IP_C↵HAN_NOTIF_... e.g. ADC_SAR_IP_CHAN_NOTIF_EOC.

Parameters

in	<i>u32Instance</i>	- ADC instance number
in	<i>u32ChnIdx</i>	- channel 0
in	<i>u32Mask</i>	- mask selecting targeted events

Returns

void

6.2.6.21 `Adc_Sar_Ip_DisableChannelNotifications()`

```
void Adc_Sar_Ip_DisableChannelNotifications (
    const uint32 u32Instance,
    const uint32 u32ChnIdx,
    const uint32 u32Mask )
```

Disable ADC interrupt for a channel.

This function disables interrupt generation on End of Conversion and watchdog events for a single channel. The mask parameter can be set using the defines provided in the driver header file that have the pattern `ADC_SAR_IP_CHAN_NOTIF_...` e.g. `ADC_SAR_IP_CHAN_NOTIF_EOC`.

Parameters

in	<i>u32Instance</i>	- ADC instance number
in	<i>u32ChnIdx</i>	- channel 0
in	<i>u32Mask</i>	- mask selecting targeted events

Returns

void

6.2.6.22 `Adc_Sar_Ip_SetClockMode()`

```
Adc_Sar_Ip_StatusType Adc_Sar_Ip_SetClockMode (
    const uint32 u32Instance,
    const Adc_Sar_Ip_ClockConfigType *const pConfig )
```

Set the ADC clocks.

This function initializes the ADC clock configuration.

Parameters

in	<i>u32Instance</i>	- ADC instance number
in	<i>pConfig</i>	- the clock configuration

Returns

- status:
- ADC_SAR_IP_STATUS_SUCCESS: set successful
 - ADC_SAR_IP_STATUS_TIMEOUT: power up or down sequence timed out

6.2.6.23 Adc_Sar_Ip_SetSampleTimes()

```
void Adc_Sar_Ip_SetSampleTimes (
    const uint32 u32Instance,
    const uint8 *const aSampleTimes )
```

Set the sample times.

This function sets the sample times for each channel group. Note: aSampleTimes must contain the sample times in order of the channel groups, e.g. sample time for the channel group 1 must be on the first position.

Parameters

in	<i>u32Instance</i>	- ADC instance number
in	<i>aSampleTimes</i>	- array with size ADC_SAR_IP_NUM_GROUP_CHAN containing sample times for each channel group

Returns

void

6.2.6.24 Adc_Sar_Ip_AbortConversion()

```
void Adc_Sar_Ip_AbortConversion (
    const uint32 u32Instance )
```

Abort ongoing conversion.

This function aborts an ongoing conversion.

Parameters

in	<i>u32Instance</i>	- ADC instance number
----	--------------------	-----------------------

Returns

void

6.2.6.25 Adc_Sar_Ip_AbortChain()

```
Adc_Sar_Ip_StatusType Adc_Sar_Ip_AbortChain (
    const uint32 u32Instance,
    const boolean bBlocking,
    const boolean bAllowRestart )
```

Abort ongoing chain conversion.

This function aborts an ongoing chain of conversions. If scan(continuous) mode is selected, then bAllowRestart selects whether the abort command restarts the chain or stops it. For one shot conversion, this should be FALSE.

Parameters

in	<i>u32Instance</i>	- ADC instance number
in	<i>bBlocking</i>	- wait and check that the ongoing conversion has stopped
in	<i>bAllowRestart</i>	- restart continuous conversion instead of aborting

Returns

status:

- ADC_SAR_IP_STATUS_SUCCESS: abort successful
- ADC_SAR_IP_STATUS_TIMEOUT: ongoing conversion could not be stopped

void

6.2.6.26 Adc_Sar_Ip_SetPresamplingSource()

```
void Adc_Sar_Ip_SetPresamplingSource (
    const uint32 u32Instance,
    const Adc_Sar_Ip_ChanGroupType pChanGroup,
    const Adc_Sar_Ip_PresamplingSourceType pPresampleSource )
```

Set the Presampling Source for the channel group.

This function configures the Presampling Source for a channel group.

Parameters

in	<i>u32Instance</i>	- ADC instance number
in	<i>pChanGroup</i>	- the channel group
in	<i>pPresampleSource</i>	- the presampling source

Returns

void

6.2.6.27 Adc_Sar_Ip_EnableChannelPresampling()

```
void Adc_Sar_Ip_EnableChannelPresampling (
    const uint32 u32Instance,
    const uint32 u32ChnIdx )
```

Enable Presampling on one channel.

This function enables the Presampling on one channel of the ADC.

Parameters

in	<i>u32Instance</i>	- ADC instance number
in	<i>u32ChnIdx</i>	- channel 0

Returns

void

6.2.6.28 Adc_Sar_Ip_DisableChannelPresampling()

```
void Adc_Sar_Ip_DisableChannelPresampling (
    const uint32 u32Instance,
    const uint32 u32ChnIdx )
```

Disable Presampling on one channel.

This function disables the Presampling on one channel of the ADC.

Parameters

in	<i>u32Instance</i>	- ADC instance number
in	<i>u32ChnIdx</i>	- channel 0

Returns

void

6.2.6.29 **Adc_Sar_Ip_EnablePresampleConversion()**

```
void Adc_Sar_Ip_EnablePresampleConversion (
    const uint32 u32Instance )
```

Enable Conversion Presampled Data.

This function enables bypass of the Sampling Phase, resulting in a conversion of the presampled data. This is available only for channels that have presampling enabled.

Parameters

in	<i>u32Instance</i>	- ADC instance number
----	--------------------	-----------------------

Returns

void

6.2.6.30 **Adc_Sar_Ip_DisablePresampleConversion()**

```
void Adc_Sar_Ip_DisablePresampleConversion (
    const uint32 u32Instance )
```

Disable Conversion of Presampled Data.

This function disables Sampling Phase bypass.

Parameters

in	<i>u32Instance</i>	- ADC instance number
----	--------------------	-----------------------

Returns

void

6.2.6.31 Adc_Sar_Ip_EnableDma()

```
void Adc_Sar_Ip_EnableDma (
    const uint32 u32Instance )
```

Enable DMA Requests.

This function enables requests to DMA from ADC

Parameters

in	<i>u32Instance</i>	- ADC instance number
----	--------------------	-----------------------

Returns

void

6.2.6.32 Adc_Sar_Ip_DisableDma()

```
void Adc_Sar_Ip_DisableDma (
    const uint32 u32Instance )
```

Disable DMA Requests.

This function disables requests to DMA from ADC

Parameters

in	<i>u32Instance</i>	- ADC instance number
----	--------------------	-----------------------

Returns

void

6.2.6.33 Adc_Sar_Ip_EnableChannelDma()

```
void Adc_Sar_Ip_EnableChannelDma (
    const uint32 u32Instance,
    const uint32 u32ChnIdx )
```

Enable DMA on one channel.

This function enables DMA requests triggered by End of Conversion event from a selected channel.

Parameters

in	<i>u32Instance</i>	- ADC instance number
in	<i>u32ChnIdx</i>	- channel 0

Returns

void

6.2.6.34 **Adc_Sar_Ip_DisableChannelDma()**

```
void Adc_Sar_Ip_DisableChannelDma (
    const uint32 u32Instance,
    const uint32 u32ChnIdx )
```

Disable DMA on one channel.

This function disables DMA requests triggered by End of Conversion event from a selected channel.

Parameters

in	<i>u32Instance</i>	- ADC instance number
in	<i>u32ChnIdx</i>	- channel 0

Returns

void

6.2.6.35 **Adc_Sar_Ip_DisableChannelDmaAll()**

```
void Adc_Sar_Ip_DisableChannelDmaAll (
    const uint32 u32Instance )
```

Disable DMA on all channels.

This function disables DMA requests triggered by End of Conversion event from all channels.

Parameters

in	<i>u32Instance</i>	- ADC instance number
----	--------------------	-----------------------

Returns

void

6.2.6.36 Adc_Sar_Ip_SetDmaClearSource()

```
void Adc_Sar_Ip_SetDmaClearSource (
    const uint32 u32Instance,
    const Adc_Sar_Ip_ClearSourceType pDmaClear )
```

Set DMA Request Clear Source.

This function selects the DMA Request Flag Clear Source.

Parameters

in	<i>u32Instance</i>	- ADC instance number
in	<i>pDmaClear</i>	- the clear source for DMA Requests (Ack from DMA Controller or read of data registers)

Returns

void

6.2.6.37 Adc_Sar_Ip_SetWdgThreshold()

```
void Adc_Sar_Ip_SetWdgThreshold (
    const uint32 u32Instance,
    const uint8 u8RegisterIdx,
    const Adc_Sar_Ip_WdgThresholdType *const pThresholdValues )
```

Configure watchdog threshold register.

This function configures the high/low thresholds for a certain register.

Parameters

in	<i>u32Instance</i>	- ADC instance number
in	<i>u8RegisterIdx</i>	- the index of the register
in	<i>pThresholdValues</i>	- the threshold values

Returns

void

6.2.6.38 Adc_Sar_Ip_SetConversionMode()

```
void Adc_Sar_Ip_SetConversionMode (
    const uint32 u32Instance,
    const Adc_Sar_Ip_ConvModeType eConvMode )
```

Configure conversion mode.

This function configures the used conversion mode. Note: The selected ADC instance must be in the IDLE state.

Parameters

in	<i>u32Instance</i>	- ADC instance number
in	<i>eConvMode</i>	- selected conversion mode

Returns

void

6.2.6.39 Adc_Sar_Ip_SetExternalTrigger()

```
void Adc_Sar_Ip_SetExternalTrigger (
    const uint32 u32Instance,
    const Adc_Sar_Ip_ExtTriggerEdgeType eTriggerEdge,
    const Adc_Sar_Ip_ExtTriggerSourceType eTrggerSrc )
```

Configure external trigger.

This function configures the external trigger.

Parameters

in	<i>u32Instance</i>	- ADC instance number
in	<i>eTriggerEdge</i>	- selected external trigger type
in	<i>eTrggerSrc</i>	- selects normal, auxiliary normal or injected trigger



Module Documentation

Returns

void

6.3 Ctu IPL

6.3.1 Detailed Description Ctu HW module.

Ctu IP layer module.

Data Structures

- struct [Ctu_Ip_TrigGenSubType](#)
Structure for configuring trigger generator subunit control. [More...](#)
- struct [Ctu_Ip_ListStatusType](#)
Structure for retrieving ADC command list status information. [More...](#)
- struct [Ctu_Ip_ConvResultType](#)
Structure for retrieving full result information for a conversion. [More...](#)
- struct [Ctu_Ip_AdcCmdConfigType](#)
Structure for configuring a conversion command. [More...](#)
- struct [Ctu_Ip_AdcCmdType](#)
Structure for configuring a conversion command at runtime. [More...](#)
- struct [Ctu_Ip_TrigConfigType](#)
Structure for configuring an internal trigger (TGS output trigger, input for SU). [More...](#)
- struct [Ctu_Ip_FifoConfigType](#)
Structure for configuring a result FIFO. [More...](#)
- struct [Ctu_Ip_FifoNotificationsType](#)
Store result FIFO notifications. [More...](#)
- struct [Ctu_Ip_ConfigType](#)
Structure for configuring global CTU functionalities. [More...](#)
- struct [Ctu_Ip_StateStructType](#)
Structure for storing runtime info. [More...](#)

Macros

- `#define CTU_IP_INPUT_TRIG_BITMASK_RISING(INPUT_IDX)`
Macros for selecting input triggers signal edges - mapped to TRGSISR register format. Implements : CTU_INP↔UT_TRIG_BITMASK_Class.
- `#define CTU_IP_INPUT_TRIG_BITMASK_FALLING(INPUT_IDX)`
- `#define CTU_IP_INPUT_TRIG_BITMASK_BOTH(INPUT_IDX)`
- `#define CTU_IP_INPUT_TRIG_BITMASK_ALL`
- `#define CTU_IP_OUTPUT_TRIG_ADC_CMD_EN`
Macros for selecting CTU outputs to be enabled - mapped to THCR register. Implements : CTU_IP_OUTPUT_↔TRIG_Class.
- `#define CTU_IP_OUTPUT_TRIG_EN`
- `#define CTU_IP_FIFO_FULL`
Macros for selecting CTU result FIFO status and interrupt flags - mapped to FST and FCR registers. Implements : CTU_FIFO_Class.
- `#define CTU_IP_ERROR_FLAG_MRS_RE`
Macros for selecting CTU error flags - mapped to EFR register. Implements : CTU_IP_ERROR_FLAG_Class.

- `#define CTU_IP_NOTIFICATION_FLAG_MRS`
Macros for selecting CTU NOTIFICATION status flags - mapped to IFR register. Implements : CTU_IP_NOTIFICATION_FLAG_Class.
- `#define CTU_IP_NOTIFICATION_FLAG_TRIG_0`
- `#define CTU_IP_NOTIFICATION_FLAG_TRIG_1`
- `#define CTU_IP_NOTIFICATION_FLAG_TRIG_2`
- `#define CTU_IP_NOTIFICATION_FLAG_TRIG_3`
- `#define CTU_IP_NOTIFICATION_FLAG_TRIG_4`
- `#define CTU_IP_NOTIFICATION_FLAG_TRIG_5`
- `#define CTU_IP_NOTIFICATION_FLAG_TRIG_6`
- `#define CTU_IP_NOTIFICATION_FLAG_TRIG_7`
- `#define CTU_IP_NOTIFICATION_FLAG_ADC_CMD`
- `#define CTU_IP_NOTIFICATION_FLAG_SERR_A`
- `#define CTU_IP_NOTIFICATION_FLAG_SERR_B`
- `#define CTU_IP_NOTIFICATION_MRS`
Macros for enabling CTU notifications - mapped to IR register. Implements : CTU_IP_NOTIFICATION_Class.
- `#define CTU_IP_NOTIFICATION_TRIG_0`
- `#define CTU_IP_NOTIFICATION_TRIG_1`
- `#define CTU_IP_NOTIFICATION_TRIG_2`
- `#define CTU_IP_NOTIFICATION_TRIG_3`
- `#define CTU_IP_NOTIFICATION_TRIG_4`
- `#define CTU_IP_NOTIFICATION_TRIG_5`
- `#define CTU_IP_NOTIFICATION_TRIG_6`
- `#define CTU_IP_NOTIFICATION_TRIG_7`

Types Reference

- `typedef void(* Ctu_Ip_TriggerNotificationType) (void)`
Defines the trigger notification header.

Enum Reference

- enum `Ctu_Ip_StatusType`
Defines the CTU status values.
- enum `Ctu_Ip_PrescalerType`
Defines the CTU prescaler values.
- enum `Ctu_Ip_TGSModeType`
Defines the CTU Trigger Generator Subunit (TGS) operating modes.
- enum `Ctu_Ip_AdcCmdListModeType`
Defines the CTU ADC command list operating mode.
- enum `Ctu_Ip_InputEdgeType`
Defines the CTU signal edges for input selection.
- enum `Ctu_Ip_InputTrigType`
Defines the CTU input triggers.
- enum `Ctu_Ip_ConvModeType`
Defines the CTU conversion mode.

- enum [Ctu_Ip_AdcPortType](#)
Defines the available ADC ports connected to CTU.
- enum [Ctu_Ip_DataAlignType](#)
Defines the possible data alignment for ADC conversion results available from CTU result FIFOs.
- enum [Ctu_Ip_IRQType](#)
Defines the available interrupt request types.

Function Reference

- [Ctu_Ip_StatusType Ctu_Ip_Init](#) (const uint32 u32Instance, const [Ctu_Ip_ConfigType](#) *const pConfig)
Configure the global functionalities of a CTU instance.
- [Ctu_Ip_StatusType Ctu_Ip_Deinit](#) (const uint32 u32Instance)
Reset all registers of a CTU instance to default values.
- void [Ctu_Ip_SetInputSelect](#) (const uint32 u32Instance, const [Ctu_Ip_InputTrigType](#) pInputTrig, const [Ctu_Ip_InputEdgeType](#) pInputEdge)
Enable the selected signal edge(s) for a CTU input trigger.
- void [Ctu_Ip_ClearInputSelect](#) (const uint32 u32Instance, const [Ctu_Ip_InputTrigType](#) pInputTrig, const [Ctu_Ip_InputEdgeType](#) pInputEdge)
Disable the selected signal edge(s) for a CTU input trigger.
- [Ctu_Ip_StatusType Ctu_Ip_SyncInputSelect](#) (const uint32 u32Instance)
Load the new value of input trigger select - TGSISR (double buffered register).
- void [Ctu_Ip_SetTriggerCompare](#) (const uint32 u32Instance, const uint32 u32TrigIdx, const uint16 u16Cmp←Val)
Configure the compare value of a single internal trigger.
- void [Ctu_Ip_SetTriggerAdcCmdAddress](#) (const uint32 u32Instance, const uint32 u32TrigIdx, const uint8 u8←AdcCmdAddress)
Configure the Adc command address of a single internal trigger.
- void [Ctu_Ip_WriteTriggerEnableMask](#) (const uint32 u32Instance, const uint32 u32TrigIdx, const uint8 u8←TriggerEnableMask)
Configure which CTU output triggers are enabled for a single internal trigger.
- void [Ctu_Ip_SetTriggerEnable](#) (const uint32 u32Instance, const uint32 u32TrigIdx, const boolean bEnable)
Set the enable bit for a single internal trigger.
- void [Ctu_Ip_ConfigAdcCmdList](#) (const uint32 u32Instance, const uint32 u32StartCmdListAdr, const [Ctu_Ip_AdcCmdType](#) *const pAdcCmdArray, const uint8 u8NumCmds)
Configure an array of ADC commands and marks accordingly the last command.
- void [Ctu_Ip_SetFifoThreshold](#) (const uint32 u32Instance, const uint32 u32FifoIdx, const uint8 u8Threshold)
Configure the threshold of a selected result FIFO.
- void [Ctu_Ip_GetFifoData](#) (const uint32 u32Instance, const uint32 u32FifoIdx, [Ctu_Ip_DataAlignType](#) p←Alignment, const uint8 u8ResultLength, uint16 *const pData)
Return the conversion data result aligned according to the alignment parameter.
- void [Ctu_Ip_GetFifoResult](#) (const uint32 u32Instance, const uint32 u32FifoIdx, const [Ctu_Ip_DataAlignType](#) pAlignment, const uint8 u8ResultLength, [Ctu_Ip_ConvResultType](#) *const pResults)
Get the full conversion result information, with the conversion data aligned according to the alignment parameter.
- void [Ctu_Ip_EnableFifoNotifications](#) (const uint32 u32Instance, const uint32 u32FifoIdx, const uint32 u32←NotificationMask)
Enable the notifications for the selected result FIFO, corresponding to the bits set in mask parameter.

- void [Ctu_Ip_DisableFifoNotifications](#) (const uint32 u32Instance, const uint32 u32FifoIdx, const uint32 u32← NotificationMask)
Disable the notifications for the selected result FIFO, corresponding to the bits set in mask parameter.
- void [Ctu_Ip_EnableNotifications](#) (const uint32 u32Instance, const uint16 u16NotificationMask)
Enable the notifications corresponding to the bits set in mask parameter.
- void [Ctu_Ip_DisableNotifications](#) (const uint32 u32Instance, const uint16 u16NotificationMask)
Disable the notifications corresponding to the bits set in mask parameter.
- void [Ctu_Ip_EnableTrigger](#) (const uint32 u32Instance, const uint8 u8TrigIdxMask)
Enable the triggers corresponding to the bits set in mask parameter.
- void [Ctu_Ip_DisableTrigger](#) (const uint32 u32Instance, const uint8 u8TrigIdxMask)
Disable the triggers corresponding to the bits set in mask parameter.
- void [Ctu_Ip_ClearErrorFlags](#) (const uint32 u32Instance, const uint32 u32FlagsMask)
Clear the error flags corresponding to the bits set in mask parameter.
- uint16 [Ctu_Ip_GetErrorFlags](#) (const uint32 u32Instance)
Return bit-mask with values for all the CTU error flags.
- [Ctu_Ip_StatusType](#) [Ctu_Ip_GetListStatus](#) (const uint32 u32Instance, const uint32 u32ListIdx, [Ctu_Ip_ListStatusType](#) *const pListStatus)
Get the list status information.
- void [Ctu_Ip_SwInternalTrigger](#) (const uint32 u32Instance, const uint32 u32TrigIdx)
Generates from software an event on the selected internal trigger.
- void [Ctu_Ip_SwMasterReload](#) (const uint32 u32Instance)
Software trigger the Master Reload Signal.
- void [Ctu_Ip_EnableOutput](#) (const uint32 u32Instance)
Enable CTU output.
- void [Ctu_Ip_DisableOutput](#) (const uint32 u32Instance)
Disable CTU output.
- [Ctu_Ip_StatusType](#) [Ctu_Ip_SetGeneralReloadEnable](#) (const uint32 u32Instance, const boolean bEnable)
Set the Enable General Reload flag (CR.GRE)
- void [Ctu_Ip_SetUserAccessAllowed](#) (const CTU_Type *const CtuBasePtr)
Sets the UAA bit in REG_PROT to make the instance accessible in user mode.
- void [Ctu_Ip_ClrUserAccessAllowed](#) (const CTU_Type *const CtuBasePtr)
Clears the UAA bit in REG_PROT to make the instance inaccessible in user mode.

6.3.2 Data Structure Documentation

6.3.2.1 struct Ctu_Ip_TrigGenSubType

Structure for configuring trigger generator subunit control.

This structure for configuring trigger generator subunit control.

Implements : Ctu_Ip_TrigGenSubType_Class

Definition at line 284 of file Ctu_Ip_Types.h.

Data Fields

Type	Name	Description
Ctu_Ip_TGSModeType	TgsMode	Trigger Generator Subunit (TGS) Mode - double-buffered (TGSCR.TGS_M)
Ctu_Ip_PrescalerType	Prescaler	Input clock prescaler - double-buffered (TGSCR.PRES)
Ctu_Ip_InputTrigType	SeqModeMrsInput	Select which input to be used for MRS when Sequential Mode is enabled (see tgsMode) - double-buffered (TGSCR.MRS_SM)
Ctu_Ip_InputEdgeType	SeqModeMrsInputEdge	Select which edge of the input signal (only rising or falling is allowed) to be used for MRS when SequenceMode is enabled - double-buffered (TGSCR.MRS_SM)

6.3.2.2 struct Ctu_Ip_ListStatusType

Structure for retrieving ADC command list status information.

This structure is used for retrieving ADC command list status information.

Implements : Ctu_Ip_ListStatusType_Class

Definition at line 301 of file Ctu_Ip_Types.h.

Data Fields

Type	Name	Description
uint8	ListAddr	Command address being executed when EFR.LIST_BE flag was set
boolean	ListBlocked	When set indicates that the address in listAddr had failed to issue an ADC cmd

6.3.2.3 struct Ctu_Ip_ConvResultType

Structure for retrieving full result information for a conversion.

This structure is used for retrieving full result information for a conversion.

Implements : Ctu_Ip_ConvResultType_Class

Definition at line 313 of file Ctu_Ip_Types.h.

Data Fields

Type	Name	Description
Ctu_Ip_AdcPortType	AdcPort	ADC instance on which the conversion has executed
uint8	AdcChanNum	ADC channel number on which the conversion has executed
uint16	ConvData	ADC conversion data result

6.3.2.4 struct Ctu_Ip_AdcCmdConfigType

Structure for configuring a conversion command.

This structure is used for configuring a conversion command. The structure fields are a reunion of all fields available for the 3 command formats: A, B and C.

Implements : Ctu_Ip_AdcCmdConfigType_Class

Definition at line 327 of file Ctu_Ip_Types.h.

Data Fields

Type	Name	Description
boolean	IntEn	Enable interrupt request for command execution
uint8	FifoIdx	Select result FIFO to store the conversion result
Ctu_Ip_ConvModeType	ConvMode	Select conversion mode - if selfTestEn==false, selects between format A and B
Ctu_Ip_AdcPortType	AdcPort	Select ADC instance to run the conversion - used in format A and C
uint8	AdcChanA	Select target channel number for ADC instance 0 (ADC port A)
uint8	AdcChanB	Select target channel number for ADC instance 1 (ADC port B) - used only in format B
boolean	bLastCmd	

6.3.2.5 struct Ctu_Ip_AdcCmdType

Structure for configuring a conversion command at runtime.

This structure is used for configuring a conversion command at runtime. The structure fields are a reunion of all fields available for the 3 command formats: A, B and C.

Implements : Ctu_Ip_AdcCmdType_Class

Definition at line 347 of file Ctu_Ip_Types.h.

Data Fields

Type	Name	Description
boolean	IntEn	Enable interrupt request for command execution
uint8	FifoIdx	Select result FIFO to store the conversion result
Ctu_Ip_ConvModeType	ConvMode	Select conversion mode - if selfTestEn==false, selects between format A and B
Ctu_Ip_AdcPortType	AdcPort	Select ADC instance to run the conversion - used in format A and C
uint8	AdcChanA	Select target channel number for ADC instance 0 (ADC port A)
uint8	AdcChanB	Select target channel number for ADC instance 1 (ADC port B) - used only in format B

6.3.2.6 struct Ctu_Ip_TrigConfigType

Structure for configuring an internal trigger (TGS output trigger, input for SU).

This structure is used for configuring the properties of an internal trigger (TGS output trigger, input for SU).

Implements : Ctu_Ip_TrigConfigType_Class

Definition at line 365 of file Ctu_Ip_Types.h.

Data Fields

Type	Name	Description
uint8	TrigIndex	
uint16	CompareVal	Compare Value - double-buffered (TnCR)
uint8	CmdListStartAdr	First address of the command list associated with the internal trigger - (CLCRx)
uint8	OutputTrigEnMask	Select the CTU output triggers enabled for each SU input trigger. To be used with CTU_IP_OUTPUT_TRIG_ defines. - double-buffered (THCR)
Ctu_Ip_TrgNotificationType	TrigNotification	

6.3.2.7 struct Ctu_Ip_FifoConfigType

Structure for configuring a result FIFO.

This structure is used for configuring a result FIFO.

Implements : Ctu_Ip_FifoConfigType_Class

Definition at line 380 of file Ctu_Ip_Types.h.

6.3.2.8 struct Ctu_Ip_FifoNotificationsType

Store result FIFO notifications.

This structure is used to store notifications for a result FIFO.

Implements : Ctu_Ip_FifoNotificationsType_Class

Definition at line 403 of file Ctu_Ip_Types.h.

6.3.2.9 struct Ctu_Ip_ConfigType

Structure for configuring global CTU functionalities.

This structure is used for configuring global CTU functionalities

Implements : Ctu_Ip_ConfigType_Class

Definition at line 417 of file Ctu_Ip_Types.h.

Data Fields

- Ctu_Ip_TGSMoDeType TgsMode
- Ctu_Ip_PrescalerType Prescaler
- uint32 InputTrigSelectMask
- uint16 TgsCounterCompareVal
- uint16 TgsCounterReloadVal
- Ctu_Ip_AdcCmdListMoDeType AdcCmdListMode
- Ctu_Ip_InputTrigType SeqModeMrsInput
- Ctu_Ip_InputEdgeType SeqModeMrsInputEdge
- boolean DmaDoneGRE
- boolean DmaReqMRS
- boolean DisableOutput
- void(* ErrNotification)(uint32 Mask)

6.3.2.9.1 Field Documentation

6.3.2.9.1.1 TgsMode Ctu_Ip_TGSMoDeType TgsMode

Trigger Generator Subunit (TGS) Mode - double-buffered (TGSCR.TGS_M)

Definition at line 418 of file Ctu_Ip_Types.h.

6.3.2.9.1.2 Prescaler Ctu_Ip_PrescalerType Prescaler

Input clock prescaler - double-buffered (TGSCR.PRES)

Definition at line 419 of file Ctu_Ip_Types.h.

6.3.2.9.1.3 InputTrigSelectMask uint32 InputTrigSelectMask

Select input triggers: even/odd bits correspond to rising/falling edge. May be set using CTU_INPUT_TRIG_B↔ITMASK_ macros. - double-buffered (TGSISR)

Definition at line 423 of file Ctu_Ip_Types.h.

6.3.2.9.1.4 TgsCounterCompareVal uint16 TgsCounterCompareVal

TGS Counter Compare Value - double-buffered (TGSCCR)

Definition at line 424 of file Ctu_Ip_Types.h.

6.3.2.9.1.5 TgsCounterReloadVal uint16 TgsCounterReloadVal

TGS Counter Reload Value - double-buffered (TGSCRR)

Definition at line 425 of file Ctu_Ip_Types.h.

6.3.2.9.1.6 AdcCmdListMode Ctu_Ip_AdcCmdListModeType AdcCmdListMode

Select mode of execution for ADC Command List (LISTCSR.PAR_LIST)

Definition at line 426 of file Ctu_Ip_Types.h.

6.3.2.9.1.7 SeqModeMrsInput Ctu_Ip_InputTrigType SeqModeMrsInput

Select which input to be used for MRS when Sequential Mode is enabled (see tgsMode) - double-buffered (TGSCR.MRS_SM)

Definition at line 427 of file Ctu_Ip_Types.h.

6.3.2.9.1.8 SeqModeMrsInputEdge Ctu_Ip_InputEdgeType SeqModeMrsInputEdge

Select which edge of the input signal (only rising or falling is allowed) to be used for MRS when SequenceMode is enabled - double-buffered (TGSCR.MRS_SM)

Definition at line 428 of file Ctu_Ip_Types.h.

6.3.2.9.1.9 DmaDoneGRE boolean DmaDoneGRE

Enable setting of General Reload Enable (GRE) when DMA-done occurs. (IR.DMA_DE)

Definition at line 429 of file Ctu_Ip_Types.h.

6.3.2.9.1.10 DmaReqMRS `boolean DmaReqMRS`

Enable issuing of DMA request when MRS occurs, if GRE is enabled. (IR.MRS_DMAE)

Definition at line 430 of file Ctu_Ip_Types.h.

6.3.2.9.1.11 DisableOutput `boolean DisableOutput`

True/false - disable/enable the CTU outputs: pulses to other peripherals/pins, ADC commands or command streams.

Definition at line 431 of file Ctu_Ip_Types.h.

6.3.2.9.1.12 ErrNotification `void(* ErrNotification) (uint32 Mask)`

Error callback with triggered mask error as parameter, can check with CTU_IP_ERROR_FLAG defines. Interrupt error enabled (IR.IEE) 9 error types.

Definition at line 436 of file Ctu_Ip_Types.h.

6.3.2.10 struct Ctu_Ip_StateStructType

Structure for storing runtime info.

This structure is used to store CTU runtime info

Implements : Ctu_Ip_StateStructType_Class

Definition at line 458 of file Ctu_Ip_Types.h.

Data Fields

- boolean [InitStatus](#)

6.3.2.10.1 Field Documentation

6.3.2.10.1.1 InitStatus `boolean InitStatus`

Check if the driver was initialized.

Definition at line 459 of file Ctu_Ip_Types.h.

6.3.3 Macro Definition Documentation

6.3.3.1 CTU_IP_INPUT_TRIG_BITMASK_RISING

```
#define CTU_IP_INPUT_TRIG_BITMASK_RISING(  
    INPUT_IDX )
```

Macros for selecting input triggers signal edges - mapped to TRGSISR register format. Implements : CTU_INP↔UT_TRIG_BITMASK_Class.

Select input triggers: even bits correspond to rising edge of the input idx

Definition at line 108 of file Ctu_Ip.h.

6.3.3.2 CTU_IP_INPUT_TRIG_BITMASK_FALLING

```
#define CTU_IP_INPUT_TRIG_BITMASK_FALLING(  
    INPUT_IDX )
```

Select input triggers: odd bits correspond to falling edge of the input idx

Definition at line 109 of file Ctu_Ip.h.

6.3.3.3 CTU_IP_INPUT_TRIG_BITMASK_BOTH

```
#define CTU_IP_INPUT_TRIG_BITMASK_BOTH(  
    INPUT_IDX )
```

Select input triggers: both rising and falling edges of the input idx

Definition at line 110 of file Ctu_Ip.h.

6.3.3.4 CTU_IP_INPUT_TRIG_BITMASK_ALL

```
#define CTU_IP_INPUT_TRIG_BITMASK_ALL
```

Select input triggers: both rising and falling edges for all inputs

Definition at line 111 of file Ctu_Ip.h.

6.3.3.5 CTU_IP_OUTPUT_TRIG_ADC_CMD_EN

```
#define CTU_IP_OUTPUT_TRIG_ADC_CMD_EN
```

Macros for selecting CTU outputs to be enabled - mapped to THCR register. Implements : CTU_IP_OUTPUT↵_TRIG_Class.

Definition at line 118 of file Ctu_Ip.h.

6.3.3.6 CTU_IP_OUTPUT_TRIG_EN

```
#define CTU_IP_OUTPUT_TRIG_EN
```

Enable the output for selected CTU trigger. If not set, all outputs are disabled.

Definition at line 141 of file Ctu_Ip.h.

6.3.3.7 CTU_IP_FIFO_FULL

```
#define CTU_IP_FIFO_FULL
```

Macros for selecting CTU result FIFO status and interrupt flags - mapped to FST and FCR registers. Implements : CTU_FIFO_Class.

Definition at line 161 of file Ctu_Ip.h.

6.3.3.8 CTU_IP_ERROR_FLAG_MRS_RE

```
#define CTU_IP_ERROR_FLAG_MRS_RE
```

Macros for selecting CTU error flags - mapped to EFR register. Implements : CTU_IP_ERROR_FLAG_Class.

Definition at line 171 of file Ctu_Ip.h.

6.3.3.9 CTU_IP_NOTIFICATION_FLAG_MRS

```
#define CTU_IP_NOTIFICATION_FLAG_MRS
```

Macros for selecting CTU NOTIFICATION status flags - mapped to IFR register. Implements : CTU_IP_NOTIFICATION_FLAG_Class.

Set when Master Reload Signal is issued.

Definition at line 210 of file Ctu_Ip.h.

6.3.3.10 CTU_IP_NOTIFICATION_FLAG_TRIG_0

```
#define CTU_IP_NOTIFICATION_FLAG_TRIG_0
```

Set when the corresponding internal trigger is issued.

Definition at line 211 of file Ctu_Ip.h.

6.3.3.11 CTU_IP_NOTIFICATION_FLAG_TRIG_1

```
#define CTU_IP_NOTIFICATION_FLAG_TRIG_1
```

Set when the corresponding internal trigger is issued.

Definition at line 212 of file Ctu_Ip.h.

6.3.3.12 CTU_IP_NOTIFICATION_FLAG_TRIG_2

```
#define CTU_IP_NOTIFICATION_FLAG_TRIG_2
```

Set when the corresponding internal trigger is issued.

Definition at line 213 of file Ctu_Ip.h.

6.3.3.13 CTU_IP_NOTIFICATION_FLAG_TRIG_3

```
#define CTU_IP_NOTIFICATION_FLAG_TRIG_3
```

Set when the corresponding internal trigger is issued.

Definition at line 214 of file Ctu_Ip.h.

6.3.3.14 CTU_IP_NOTIFICATION_FLAG_TRIG_4

```
#define CTU_IP_NOTIFICATION_FLAG_TRIG_4
```

Set when the corresponding internal trigger is issued.

Definition at line 215 of file Ctu_Ip.h.

6.3.3.15 CTU_IP_NOTIFICATION_FLAG_TRIG_5

```
#define CTU_IP_NOTIFICATION_FLAG_TRIG_5
```

Set when the corresponding internal trigger is issued.

Definition at line 216 of file Ctu_Ip.h.

6.3.3.16 CTU_IP_NOTIFICATION_FLAG_TRIG_6

```
#define CTU_IP_NOTIFICATION_FLAG_TRIG_6
```

Set when the corresponding internal trigger is issued.

Definition at line 217 of file Ctu_Ip.h.

6.3.3.17 CTU_IP_NOTIFICATION_FLAG_TRIG_7

```
#define CTU_IP_NOTIFICATION_FLAG_TRIG_7
```

Set when the corresponding internal trigger is issued.

Definition at line 218 of file Ctu_Ip.h.

6.3.3.18 CTU_IP_NOTIFICATION_FLAG_ADC_CMD

```
#define CTU_IP_NOTIFICATION_FLAG_ADC_CMD
```

Set when a new ADC command is issued.

Definition at line 219 of file Ctu_Ip.h.

6.3.3.19 CTU_IP_NOTIFICATION_FLAG_SERR_A

```
#define CTU_IP_NOTIFICATION_FLAG_SERR_A
```

Duration of ADC 0 conversion is greater than the maximum configured.

Definition at line 220 of file Ctu_Ip.h.

6.3.3.20 CTU_IP_NOTIFICATION_FLAG_SERR_B

```
#define CTU_IP_NOTIFICATION_FLAG_SERR_B
```

Duration of ADC 1 conversion is greater than the maximum configured.

Definition at line 221 of file Ctu_Ip.h.

6.3.3.21 CTU_IP_NOTIFICATION_MRS

```
#define CTU_IP_NOTIFICATION_MRS
```

Macros for enabling CTU notifications - mapped to IR register. Implements : CTU_IP_NOTIFICATION_Class.

Set when Master Reload Signal is issued.

Definition at line 233 of file Ctu_Ip.h.

6.3.3.22 CTU_IP_NOTIFICATION_TRIG_0

```
#define CTU_IP_NOTIFICATION_TRIG_0
```

Set when the corresponding internal trigger is issued.

Definition at line 234 of file Ctu_Ip.h.

6.3.3.23 CTU_IP_NOTIFICATION_TRIG_1

```
#define CTU_IP_NOTIFICATION_TRIG_1
```

Set when the corresponding internal trigger is issued.

Definition at line 235 of file Ctu_Ip.h.

6.3.3.24 CTU_IP_NOTIFICATION_TRIG_2

```
#define CTU_IP_NOTIFICATION_TRIG_2
```

Set when the corresponding internal trigger is issued.

Definition at line 236 of file Ctu_Ip.h.

6.3.3.25 CTU_IP_NOTIFICATION_TRIG_3

```
#define CTU_IP_NOTIFICATION_TRIG_3
```

Set when the corresponding internal trigger is issued.

Definition at line 237 of file Ctu_Ip.h.

6.3.3.26 CTU_IP_NOTIFICATION_TRIG_4

```
#define CTU_IP_NOTIFICATION_TRIG_4
```

Set when the corresponding internal trigger is issued.

Definition at line 238 of file Ctu_Ip.h.

6.3.3.27 CTU_IP_NOTIFICATION_TRIG_5

```
#define CTU_IP_NOTIFICATION_TRIG_5
```

Set when the corresponding internal trigger is issued.

Definition at line 239 of file Ctu_Ip.h.

6.3.3.28 CTU_IP_NOTIFICATION_TRIG_6

```
#define CTU_IP_NOTIFICATION_TRIG_6
```

Set when the corresponding internal trigger is issued.

Definition at line 240 of file Ctu_Ip.h.

6.3.3.29 CTU_IP_NOTIFICATION_TRIG_7

```
#define CTU_IP_NOTIFICATION_TRIG_7
```

Set when the corresponding internal trigger is issued.

Definition at line 241 of file Ctu_Ip.h.

6.3.4 Types Reference

6.3.4.1 Ctu_Ip_TriggerNotificationType

```
typedef void(* Ctu_Ip_TriggerNotificationType) (void)
```

Defines the trigger notification header.

This header is used in individual trigger notification callbacks

Implements : Ctu_Ip_TriggerNotificationType_Class

Definition at line 275 of file Ctu_Ip_Types.h.

6.3.5 Enum Reference

6.3.5.1 Ctu_Ip_StatusType

```
enum Ctu_Ip_StatusType
```

Defines the CTU status values.

Implements : Ctu_Ip_StatusTypeType_Class

Enumerator

CTU_IP_STATUS_SUCCESS	Function completed successfully
CTU_IP_STATUS_ERROR	Function didn't complete successfully
CTU_IP_STATUS_TIMEOUT	Function timed out

Definition at line 99 of file Ctu_Ip_Types.h.

6.3.5.2 Ctu_Ip_PrescalerType

enum `Ctu_Ip_PrescalerType`

Defines the CTU prescaler values.

Implements : `Ctu_Ip_PrescalerType_Class`

Enumerator

CTU_IP_PRESCALER↵ _1	CTU clock prescaler value of 1.
CTU_IP_PRESCALER↵ _2	CTU clock prescaler value of 2.
CTU_IP_PRESCALER↵ _3	CTU clock prescaler value of 3.
CTU_IP_PRESCALER↵ _4	CTU clock prescaler value of 4.

Definition at line 110 of file Ctu_Ip_Types.h.

6.3.5.3 Ctu_Ip_TGSModeType

enum `Ctu_Ip_TGSModeType`

Defines the CTU Trigger Generator Subunit (TGS) operating modes.

Implements : `Ctu_Ip_TGSModeType_Class`

Enumerator

CTU_IP_TGS_MODE_TRIGGERED	Trigger Generator Subunit (TGS) Triggered Mode.
CTU_IP_TGS_MODE_SEQUENTIAL	Trigger Generator Subunit (TGS) Sequential Mode.

Definition at line 122 of file Ctu_Ip_Types.h.

6.3.5.4 Ctu_Ip_AdcCmdListModeType

enum `Ctu_Ip_AdcCmdListModeType`

Defines the CTU ADC command list operating mode.

Implements : `Ctu_Ip_AdcCmdListModeType_Class`

Enumerator

<code>CTU_IP_ADC_CMD_LIST_MODE_STREAMING</code>	ADC Command List in streaming mode: no more than a list executing at a time.
<code>CTU_IP_ADC_CMD_LIST_MODE_PARALLEL</code>	ADC Command List in parallel mode: lists may execute in parallel, if targeting different ADCs.

Definition at line 144 of file `Ctu_Ip_Types.h`.

6.3.5.5 Ctu_Ip_InputEdgeType

```
enum Ctu_Ip_InputEdgeType
```

Defines the CTU signal edges for input selection.

Implements : `Ctu_Ip_InputEdgeType_Class`

Enumerator

<code>CTU_IP_INPUT_EDGE_RISING</code>	Input Selection Rising Edge
<code>CTU_IP_INPUT_EDGE_FALLING</code>	Input Selection Falling Edge
<code>CTU_IP_INPUT_EDGE_BOTH</code>	Input Selection both rising and falling edges

Definition at line 154 of file `Ctu_Ip_Types.h`.

6.3.5.6 Ctu_Ip_InputTrigType

```
enum Ctu_Ip_InputTrigType
```

Defines the CTU input triggers.

This enumeration defines the CTU input triggers. Inputs may be assigned to different trigger sources, depending on each CTU instance. Please refer to device reference manual for actual assignments - chapter regarding TGSISR input assignments.

Implements : `Ctu_Ip_InputTrigType_Class`

Enumerator

CTU_IP_INPUT_TRIG_PWM_REL	Input trigger PWM Reload
CTU_IP_INPUT_TRIG_PWM_ODD_0	Input trigger PWM ODD 0
CTU_IP_INPUT_TRIG_PWM_ODD_1	Input trigger PWM ODD 1
CTU_IP_INPUT_TRIG_PWM_ODD_2	Input trigger PWM ODD 2
CTU_IP_INPUT_TRIG_PWM_ODD_3	Input trigger PWM ODD 3
CTU_IP_INPUT_TRIG_PWM_EVEN↔ _0	Input trigger PWM EVEN 0

Definition at line 169 of file Ctu_Ip_Types.h.

6.3.5.7 Ctu_Ip_ConvModeType

enum `Ctu_Ip_ConvModeType`

Defines the CTU conversion mode.

Implements : `Ctu_Ip_ConvModeType_Class`

Enumerator

CTU_IP_CONV_MODE_SINGLE	Single conversion mode - conversion will run on a single ADC instance
CTU_IP_CONV_MODE_DUAL	Dual conversion mode - conversion will run on two ADC instances simultaneously

Definition at line 225 of file Ctu_Ip_Types.h.

6.3.5.8 Ctu_Ip_AdcPortType

enum `Ctu_Ip_AdcPortType`

Defines the available ADC ports connected to CTU.

Implements : `Ctu_Ip_AdcPortType_Class`

Enumerator

CTU_IP_ADC_PORT_A	ADC instance 0
CTU_IP_ADC_PORT_B	ADC instance 1

Definition at line 237 of file Ctu_Ip_Types.h.

6.3.5.9 Ctu_Ip_DataAlignType

enum `Ctu_Ip_DataAlignType`

Defines the possible data alignment for ADC conversion results available from CTU result FIFOs.

Implements : `Ctu_Ip_DataAlignType_Class`

Enumerator

CTU_IP_RESULT_ALIGN_RIGHT_UNSIGNED	FRn: data is unsigned, right aligned
CTU_IP_RESULT_ALIGN_LEFT_SIGNED	FLn: data is signed, left aligned - leftmost bit is sign bit.

Definition at line 250 of file Ctu_Ip_Types.h.

6.3.5.10 Ctu_Ip_IRQType

enum `Ctu_Ip_IRQType`

Defines the available interrupt request types.

Implements : `Ctu_Ip_InterruptType_Class`

Enumerator

CTU_IP_IRQ_ERROR	CTU error interrupt request.
CTU_IP_IRQ_TRIG	CTU trigger interrupt request.
CTU_IP_IRQ_FIFO	CTU FIFO interrupt request.

Definition at line 261 of file Ctu_Ip_Types.h.

6.3.6 Function Reference

6.3.6.1 Ctu_Ip_Init()

```
Ctu_Ip_StatusType Ctu_Ip_Init (
    const uint32 u32Instance,
    const Ctu_Ip_ConfigType *const pConfig )
```

Configure the global functionalities of a CTU instance.

This function configures the global functionalities of a CTU instance.

Parameters

in	<i>u32Instance</i>	- the CTU instance number
in	<i>pConfig</i>	- pointer to the input configuration structure

Returns

status:

- CTU_IP_STATUS_SUCCESS: Init successful
- CTU_IP_STATUS_TIMEOUT: Init step timed out

6.3.6.2 Ctu_Ip_Deinit()

```
Ctu_Ip_StatusType Ctu_Ip_Deinit (
    const uint32 u32Instance )
```

Reset all registers of a CTU instance to default values.

This function resets all registers of a CTU instance to default values. It stops and resets any running ADC command list. Note: the function also enables the CTU output (default state is enabled).

Parameters

in	<i>u32Instance</i>	- the CTU instance number
----	--------------------	---------------------------

Returns

status:

- CTU_IP_STATUS_SUCCESS: Deinit successful
- CTU_IP_STATUS_TIMEOUT: Deinit step timed out

6.3.6.3 Ctu_Ip_SetInputSelect()

```
void Ctu_Ip_SetInputSelect (
    const uint32 u32Instance,
    const Ctu_Ip_InputTrigType pInputTrig,
    const Ctu_Ip_InputEdgeType pInputEdge )
```

Enable the selected signal edge(s) for a CTU input trigger.

This function enables the selected signal edge(s) for a CTU input trigger. The affected register is double-buffered, so the new value will take effect only after MRS occurs or [Ctu_Ip_SyncInputSelect\(\)](#) is called. Please note that the set operation is cumulative, e.g. set falling edge, then set rising edge for same input trigger, results in both edges being enabled for that trigger. The same applies also when setting both edges, followed by set falling/rising.

Parameters

in	<i>u32Instance</i>	- the CTU instance number
in	<i>pInputTrig</i>	- the selected input trigger
in	<i>pInputEdge</i>	- the selected signal edge(s) of the input trigger

Returns

void

6.3.6.4 Ctu_Ip_ClearInputSelect()

```
void Ctu_Ip_ClearInputSelect (
    const uint32 u32Instance,
    const Ctu_Ip_InputTrigType pInputTrig,
    const Ctu_Ip_InputEdgeType pInputEdge )
```

Disable the selected signal edge(s) for a CTU input trigger.

This function disables the selected signal edge(s) for a CTU input trigger. The affected register is double-buffered, so the new value will take effect only after MRS occurs or [Ctu_Ip_SyncInputSelect\(\)](#) is called.

Parameters

in	<i>u32Instance</i>	- the CTU instance number
in	<i>pInputTrig</i>	- the selected input trigger
in	<i>pInputEdge</i>	- the selected signal edge(s) of the input trigger

Returns

void

6.3.6.5 Ctu_Ip_SyncInputSelect()

```
Ctu_Ip_StatusType Ctu_Ip_SyncInputSelect (
    const uint32 u32Instance )
```

Load the new value of input trigger select - TGSISR (double buffered register).

This function loads the new value of input trigger select - TGSISR (double buffered register). The function waits until the TGSISR register is updated from the corresponding buffer register.

Parameters

in	<i>u32Instance</i>	- the CTU instance number
----	--------------------	---------------------------

Returns

status:

- CTU_IP_STATUS_SUCCESS: Sync successful
- CTU_IP_STATUS_TIMEOUT: Sync timed out

6.3.6.6 Ctu_Ip_SetTriggerCompare()

```
void Ctu_Ip_SetTriggerCompare (
    const uint32 u32Instance,
    const uint32 u32TrigIdx,
    const uint16 u16CmpVal )
```

Configure the compare value of a single internal trigger.

This function configures the compare value of a single internal trigger. By internal trigger, the CTU driver refers to the output triggers from TGS (Trigger Generator Subunit), fed as input triggers to the SU (Scheduler Subunit).

Note: *u8CmpVal* corresponds to a double buffered register, so the configured value is loaded and will take effect only after MRS occurs.

Parameters

in	<i>u32Instance</i>	- the CTU instance number
in	<i>u32TrigIdx</i>	- index of the trigger
in	<i>u16CmpVal</i>	- compare value

Returns

void

6.3.6.7 Ctu_Ip_SetTriggerAdcCmdAddress()

```
void Ctu_Ip_SetTriggerAdcCmdAddress (
    const uint32 u32Instance,
    const uint32 u32TrigIdx,
    const uint8 u8AdcCmdAddress )
```

Configure the Adc command address of a single internal trigger.

This function configures the first address of the command list associated with an internal trigger. By internal trigger, the CTU driver refers to the output triggers from TGS (Trigger Generator Subunit), fed as input triggers to the SU (Scheduler Subunit).

Parameters

in	<i>u32Instance</i>	- the CTU instance number
in	<i>u32TrigIdx</i>	- index of the trigger
in	<i>u8AdcCmdAddress</i>	- address of the first command

Returns

void

6.3.6.8 Ctu_Ip_WriteTriggerEnableMask()

```
void Ctu_Ip_WriteTriggerEnableMask (
    const uint32 u32Instance,
    const uint32 u32TrigIdx,
    const uint8 u8TriggerEnableMask )
```

Configure which CTU output triggers are enabled for a single internal trigger.

This function configures which CTU output triggers are enabled for a single internal trigger. By internal trigger, the CTU driver refers to the output triggers from TGS (Trigger Generator Subunit), fed as input triggers to the SU (Scheduler Subunit).

Note: u8TriggerEnableMask corresponds to a double buffered register, so the configured value is loaded and will take effect only after MRS occurs.

Parameters

in	<i>u32Instance</i>	- the CTU instance number
in	<i>u32TrigIdx</i>	- index of the trigger
in	<i>u8TriggerEnableMask</i>	- mask that selects the CTU output triggers to be enabled

Returns

void

6.3.6.9 Ctu_Ip_SetTriggerEnable()

```
void Ctu_Ip_SetTriggerEnable (
    const uint32 u32Instance,
    const uint32 u32TrigIdx,
    const boolean bEnable )
```

Set the enable bit for a single internal trigger.

This function sets the enable bit Tx_E of THCRx register for a single internal trigger.

By internal trigger, the CTU driver refers to the output triggers from TGS (Trigger Generator Subunit), fed as input triggers to the SU (Scheduler Subunit).

Parameters

in	<i>u32Instance</i>	- the CTU instance number
in	<i>u32TrigIdx</i>	- index of the trigger
in	<i>bEnable</i>	- value of the enable bit

Returns

void

6.3.6.10 Ctu_Ip_ConfigAdcCmdList()

```
void Ctu_Ip_ConfigAdcCmdList (
    const uint32 u32Instance,
    const uint32 u32StartCmdListAdr,
    const Ctu_Ip_AdcCmdType *const pAdcCmdArray,
    const uint8 u8NumCmds )
```

Configure an array of ADC commands and marks accordingly the last command.

This function configures an array of ADC commands and marks accordingly the last command.

Note: the ADC command registers are double buffered, so configured values are loaded and take effect only after MRS occurs.

The function doesn't take into account the reload mechanism (refer to device reference manual for details), so it needs to be handled externally - may use [Ctu_Ip_SetGeneralReloadEnable\(\)](#). Implicitly wrap-around of the array onto the list, if $(listSize - startElemIdx) < numArrayElems$.

Parameters

in	<i>u32Instance</i>	- the CTU instance number
in	<i>u32StartCmdListAdr</i>	- index of the starting element in the conversion list
in	<i>pAdcCmdArray</i>	- pointer to the array of the ADC command list
in	<i>u8NumCmds</i>	- number of commands in the array of ADC command list

Returns

void

6.3.6.11 Ctu_Ip_SetFifoThreshold()

```
void Ctu_Ip_SetFifoThreshold (
    const uint32 u32Instance,
    const uint32 u32FifoIdx,
    const uint8 u8Threshold )
```

Configure the threshold of a selected result FIFO.

This function configures the threshold of a selected result FIFO.

Parameters

in	<i>u32Instance</i>	- the CTU instance number
in	<i>u32FifoIdx</i>	- index of the result FIFO
in	<i>u8Threshold</i>	- threshold value

Returns

void

6.3.6.12 Ctu_Ip_GetFifoData()

```
void Ctu_Ip_GetFifoData (
    const uint32 u32Instance,
    const uint32 u32FifoIdx,
    Ctu_Ip_DataAlignType pAlignment,
    const uint8 u8ResultLength,
    uint16 *const pData )
```

Return the conversion data result aligned according to the alignment parameter.

This function returns the conversion data result aligned according to the alignment parameter.

Parameters

in	<i>u32Instance</i>	- the CTU instance number
in	<i>u32FifoIdx</i>	- index of the result FIFO
in	<i>pAlignment</i>	- left or right alignment of the result data
in	<i>u8ResultLength</i>	- number of results to be read
out	<i>pData</i>	- pointer to the results array

Returns

void

6.3.6.13 Ctu_Ip_GetFifoResult()

```
void Ctu_Ip_GetFifoResult (
    const uint32 u32Instance,
    const uint32 u32FifoIdx,
    const Ctu_Ip_DataAlignType pAlignment,
    const uint8 u8ResultLength,
    Ctu_Ip_ConvResultType *const pResults )
```

Get the full conversion result information, with the conversion data aligned according to the alignment parameter.

This function gets the full conversion result information, with the conversion data aligned according to the alignment parameter.

Parameters

in	<i>u32Instance</i>	- the CTU instance number
in	<i>u32FifoIdx</i>	- index of the result FIFO
in	<i>pAlignment</i>	- left or right alignment of the result data
in	<i>u8ResultLength</i>	- number of results to be read
out	<i>pResults</i>	- pointer to the output structure containing full conversion result information

Returns

void

6.3.6.14 Ctu_Ip_EnableFifoNotifications()

```
void Ctu_Ip_EnableFifoNotifications (
    const uint32 u32Instance,
    const uint32 u32FifoIdx,
    const uint32 u32NotificationMask )
```

Enable the notifications for the selected result FIFO, corresponding to the bits set in mask parameter.

This function enables the notifications for the selected result FIFO, corresponding to the bits set in mask parameter. Note: FIFO notifications(except overrun) can't be cleared by flag. Clearing overflow and full notifications is done by reading from the FIFO.

Parameters

in	<i>u32Instance</i>	- the CTU instance number
in	<i>u32FifoIdx</i>	- index of the result FIFO
in	<i>u32NotificationMask</i>	- bit-mask selecting which FIFO notifications to be enabled

Returns

void

6.3.6.15 Ctu_Ip_DisableFifoNotifications()

```
void Ctu_Ip_DisableFifoNotifications (
    const uint32 u32Instance,
    const uint32 u32FifoIdx,
    const uint32 u32NotificationMask )
```

Disable the notifications for the selected result FIFO, corresponding to the bits set in mask parameter.

This function disables the notifications for the selected result FIFO, corresponding to the bits set in mask parameter.

Parameters

in	<i>u32Instance</i>	- the CTU instance number
in	<i>u32FifoIdx</i>	- index of the result FIFO
in	<i>u32NotificationMask</i>	- bit-mask selecting which FIFO notifications to be disabled

Module Documentation

Returns

void

6.3.6.16 Ctu_Ip_EnableNotifications()

```
void Ctu_Ip_EnableNotifications (
    const uint32 u32Instance,
    const uint16 u16NotificationMask )
```

Enable the notifications corresponding to the bits set in mask parameter.

This function enables the notifications corresponding to the bits set in mask parameter.

Parameters

in	<i>u32Instance</i>	- the CTU instance number
in	<i>u16NotificationMask</i>	- bit-mask selecting which notifications to be enabled

Returns

void

6.3.6.17 Ctu_Ip_DisableNotifications()

```
void Ctu_Ip_DisableNotifications (
    const uint32 u32Instance,
    const uint16 u16NotificationMask )
```

Disable the notifications corresponding to the bits set in mask parameter.

This function disables the notifications corresponding to the bits set in mask parameter.

Parameters

in	<i>u32Instance</i>	- the CTU instance number
in	<i>u16NotificationMask</i>	- bit-mask selecting which notifications to be disabled

Returns

void

6.3.6.18 Ctu_Ip_EnableTrigger()

```
void Ctu_Ip_EnableTrigger (
    const uint32 u32Instance,
    const uint8 u8TrigIdxMask )
```

Enable the triggers corresponding to the bits set in mask parameter.

This function enables the triggers corresponding to the bits set in mask parameter. This is done by modifying the Tx_E bits of THCRx register corresponding to the triggers set in the mask parameter.

Parameters

in	<i>u32Instance</i>	- the CTU instance number
in	<i>u8TrigIdxMask</i>	- bit-mask selecting which triggers to be enabled

Returns

void

6.3.6.19 Ctu_Ip_DisableTrigger()

```
void Ctu_Ip_DisableTrigger (
    const uint32 u32Instance,
    const uint8 u8TrigIdxMask )
```

Disable the triggers corresponding to the bits set in mask parameter.

This function disables the triggers corresponding to the bits set in mask parameter. This is done by modifying the Tx_E bits of THCRx register corresponding to the triggers set in the mask parameter.

Parameters

in	<i>u32Instance</i>	- the CTU instance number
in	<i>u8TrigIdxMask</i>	- bit-mask selecting which triggers to be disabled

Returns

void

6.3.6.20 Ctu_Ip_ClearErrorFlags()

```
void Ctu_Ip_ClearErrorFlags (
    const uint32 u32Instance,
    const uint32 u32FlagsMask )
```

Clear the error flags corresponding to the bits set in mask parameter.

This function clears the error flags corresponding to the bits set in mask parameter. The mask parameter can be set using the defines provided in the driver header file that have the pattern CTU_IP_ERROR_FLAG_... e.g. CTU_IP_ERROR_FLAG_INVALID_CMD.

Parameters

in	<i>u32Instance</i>	- the CTU instance number
in	<i>u32FlagsMask</i>	- bit-mask selecting which error flags to be cleared

Returns

void

6.3.6.21 Ctu_Ip_GetErrorFlags()

```
uint16 Ctu_Ip_GetErrorFlags (
    const uint32 u32Instance )
```

Return bit-mask with values for all the CTU error flags.

This function returns bit-mask with values for all the CTU error flags. Values for individual flags may be retrieved by masking the result with defines provided in the driver header file that have the pattern CTU_IP_ERROR_FLAG_... e.g. CTU_IP_ERROR_FLAG_MRS_RE.

Parameters

in	<i>u32Instance</i>	- the CTU instance number
----	--------------------	---------------------------

Returns

uint32 - bit-mask with values for all the CTU error flags.

6.3.6.22 Ctu_Ip_GetListStatus()

```
Ctu_Ip_StatusType Ctu_Ip_GetListStatus (
    const uint32 u32Instance,
```

```
const uint32 u32ListIdx,
Ctu_Ip_ListStatusType *const pListStatus )
```

Get the list status information.

This function gets the list status information. Information for multiple lists is available when executing in parallel mode.

Parameters

in	<i>u32Instance</i>	- the instance number
in	<i>u32ListIdx</i>	- index of the list
out	<i>u32ListStatus</i>	- pointer to list status information

Returns

status:

- CTU_IP_STATUS_SUCCESS: returned information is valid
- CTU_IP_STATUS_ERROR: list status information is not meaningful, because EFR.LIST_BE is not set.

6.3.6.23 Ctu_Ip_SwInternalTrigger()

```
void Ctu_Ip_SwInternalTrigger (
    const uint32 u32Instance,
    const uint32 u32TrigIdx )
```

Generates from software an event on the selected internal trigger.

Generates from software an event on the selected internal trigger, provided as input to the Scheduler subunit (SU). This bypasses any delays configured in Trigger Generator Subunit (TGS) and can be used when the TGS Counter is running or stopped.

Parameters

in	<i>u32Instance</i>	- the CTU instance number
in	<i>u32TrigIdx</i>	- index of the internal trigger

Returns

void

6.3.6.24 Ctu_Ip_SwMasterReload()

```
void Ctu_Ip_SwMasterReload (
    const uint32 u32Instance )
```

Software trigger the Master Reload Signal.

Software trigger the Master Reload Signal.

Parameters

in	<i>u32Instance</i>	- the CTU instance number
----	--------------------	---------------------------

Returns

void

6.3.6.25 Ctu_Ip_EnableOutput()

```
void Ctu_Ip_EnableOutput (
    const uint32 u32Instance )
```

Enable CTU output.

This function enables CTU output. The output can be a pulse, an ADC command or a stream of ADC commands.

Parameters

in	<i>u32Instance</i>	- the CTU instance number
----	--------------------	---------------------------

Returns

void

6.3.6.26 Ctu_Ip_DisableOutput()

```
void Ctu_Ip_DisableOutput (
    const uint32 u32Instance )
```

Disable CTU output.

This function disables CTU output. The output can be a pulse, an ADC command or a stream of ADC commands.

Parameters

in	<i>u32Instance</i>	- the CTU instance number
----	--------------------	---------------------------

Returns

void

6.3.6.27 Ctu_Ip_SetGeneralReloadEnable()

```
Ctu_Ip_StatusType Ctu_Ip_SetGeneralReloadEnable (
    const uint32 u32Instance,
    const boolean bEnable )
```

Set the Enable General Reload flag (CR.GRE)

This function sets the Enable General Reload flag (CR.GRE). It needs to be called before MRS, after the last write to a double-buffered register. It marks that the user has completed writing all the double-buffered registers for the current control cycle, so an MRS event shall load all the new values in the double-buffered registers. If CR.GRE is not set and at least a double-buffered register has been written during the current control cycle (i.e. after the last MRS), the next MRS will not load the new values and an error flag will be raised (EFR.MRS_RE). For more details please see reload mechanism described in device reference manual.

Parameters

in	<i>u32Instance</i>	- the CTU instance number
in	<i>bEnable</i>	- value of the enable bit to be set

Returns

status:

- CTU_IP_STATUS_SUCCESS: Set successful
- CTU_IP_STATUS_TIMEOUT: Set timed out

6.3.6.28 Ctu_Ip_SetUserAccessAllowed()

```
void Ctu_Ip_SetUserAccessAllowed (
    const CTU_Type *const CtuBasePtr )
```

Sets the UAA bit in REG_PROT to make the instance accessible in user mode.

Sets the UAA bit in REG_PROT to make the instance accessible in user mode

Parameters

in	<i>CtuBasePtr</i>	Peripheral CTU base pointer. (Eg: IP_CTU getting from Base header files)
----	-------------------	--

Return values

<i>none</i>	
-------------	--

Precondition

The user mode is enabled

6.3.6.29 Ctu_Ip_ClrUserAccessAllowed()

```
void Ctu_Ip_ClrUserAccessAllowed (
    const CTU_Type *const CtuBasePtr )
```

Clears the UAA bit in REG_PROT to make the instance inaccessible in user mode.

Clears the UAA bit in REG_PROT to make the instance inaccessible in user mode.

Parameters

in	<i>CtuBasePtr</i>	Peripheral CTU base pointer. (Eg: IP_CTU getting from Base header files)
----	-------------------	--

Return values

<i>none</i>	
-------------	--

Precondition

The user mode is enabled

How to Reach Us:

Home Page:

nxp.com

Web Support:

nxp.com/support

Information in this document is provided solely to enable system and software implementers to use NXP products. There are no express or implied copyright licenses granted hereunder to design or fabricate any integrated circuits based on the information in this document. NXP reserves the right to make changes without further notice to any products herein.

NXP makes no warranty, representation, or guarantee regarding the suitability of its products for any particular purpose, nor does NXP assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation consequential or incidental damages. "Typical" parameters that may be provided in NXP data sheets and/or specifications can and do vary in different applications, and actual performance may vary over time. All operating parameters, including "typicals," must be validated for each customer application by customer's technical experts. NXP does not convey any license under its patent rights nor the rights of others. NXP sells products pursuant to standard terms and conditions of sale, which can be found at the following address: nxp.com/SalesTermsandConditions.

NXP, the NXP logo, NXP SECURE CONNECTIONS FOR A SMARTER WORLD, COOLFLUX, EMBRACE, GREENCHIP, HITAG, I2C BUS, ICODE, JCOP, LIFE VIBES, MIFARE, MIFARE CLASSIC, MIFARE DESFire, MIFARE PLUS, MIFARE FLEX, MANTIS, MIFARE ULTRALIGHT, MIFARE4MOBILE, MIGLO, NTAG, ROADLINK, SMARTLX, SMARTMX, STARPLUG, TOPFET, TRENCHMOS, UCODE, Freescale, the Freescale logo, Altivec, C-5, CodeTEST, CodeWarrior, ColdFire, ColdFire+, C-Ware, the Energy Efficient Solutions logo, Kinetis, Layerscape, MagniV, mobileGT, PEG, PowerQUICC, Processor Expert, QorIQ, QorIQ Qonverge, Ready Play, SafeAssure, the SafeAssure logo, StarCore, Symphony, VortiQa, Vybrid, Airfast, BeeKit, BeeStack, CoreNet, Flexis, MXC, Platform in a Package, QUICC Engine, SMARTMOS, Tower, TurboLink, and UMEMS are trademarks of NXP B.V. All other product or service names are the property of their respective owners. ARM, AMBA, ARM Powered, Artisan, Cortex, Jazelle, Keil, SecurCore, Thumb, TrustZone, and Vision are registered trademarks of ARM Limited (or its subsidiaries) in the EU and/or elsewhere. ARM7, ARM9, ARM11, big.LITTLE, CoreLink, CoreSight, DesignStart, Mali, mbed, NEON, POP, Sensinode, Socrates, ULINK and Versatile are trademarks of ARM Limited (or its subsidiaries) in the EU and/or elsewhere. All rights reserved. Oracle and Java are registered trademarks of Oracle and/or its affiliates. The Power Architecture and Power.org word marks and the Power and Power.org logos and related marks are trademarks and service marks licensed by Power.org.

© 2023 NXP B.V.

